



**BAD SMELLS EN BASES DE DATOS RELACIONALES:
UNA PROPUESTA TAXONÓMICA**

ANGELA PAOLA RAMÍREZ PINILLA

**ASESORA
SILVIA TAKAHASHI RODRIGUEZ, Ph.D**

**UNIVERSIDAD DE LOS ANDES
FACULTAD DE INGENIERIA
MAESTRIA EN INGENIERIA DE SISTEMAS Y
COMPUTACIÓN
BOGOTA D.C.
AGOSTO 2006**

CONTENIDO

Pág.

INTRODUCCIÓN	12
OBJETIVO GENERAL DEL PROYECTO	14
OBJETIVOS ESPECÍFICOS DEL PROYECTO	14
MARCO TEÓRICO	15
1. EVOLUCIÓN DEL SOFTWARE	15
2. REINGENIERÍA COMO DISCIPLINA	16
2.1. Guía para el proceso de Reingeniería	17
2.1.1 ¿Qué tipo de transformación se desea realizar? (<i>¿Qué?</i>)	17
2.1.2 ¿Cuál es el objetivo de la reingeniería? (<i>¿Por qué?</i>).....	19
2.1.3 ¿Sobre qué artefactos se llevará a cabo la transformación? (<i>¿Sobre qué?</i>).....	19
2.1.4 ¿A través de cuál técnica de reingeniería? (<i>¿Cómo?</i>)	20
2.2. Enfoques de la Reingeniería	20
2.3. Modelos de Reingeniería	21
2.3.1 Ingeniería en Reversa de software y datos en sistemas legados	21
2.3.2 Modelo de herradura.....	21
2.3.3 Modelo de OAR (Options Analysis for Reengineering)	24
3. REFACTORING	26
3.1. ¿Qué es refactoring?	26
3.1.1 Características	26
3.1.2 ¿Por qué se hace necesario realizar refactoring?	27
3.1.3 Proceso de Refactoring.....	27
3.2. Mantenimiento de Software, Modelos de Reingeniería y Proceso de Refactoring ..	27
3.3. Refactoring a partir de Bad Smells	29
3.3.1 Detección de Bad Smells a partir de métricas o procesos de deducción	33
4. REINGENIERÍA EN BASES DE DATOS RELACIONALES	35
4.1. Objetivos de la reingeniería de bases de datos	35
4.2. Comprensión de las bases de datos relacionales	35

4.3.	Refactoring en Bases de datos.....	36
4.3.1	Complejidad del refactoring en bases de datos	37
4.3.2	Aplicación de refactoring en bases de datos relacionales.....	38
4.4.	Fundamentos de diseño de bases de datos relacionales	42
4.4.1	Pautas para el diseño de esquemas ER en BDR	42
4.4.2	Dependencia funcional y formas normales	44
4.4.3	Patrones en BDR	46
5.	<i>PROPUESTA BAJO LA CUAL SERÁN DEFINIDOS BAD SMELLS PARA BASES DE DATOS RELACIONALES.</i>	52
5.1.	Proceso de definición de Bad Smells para BDR.....	53
5.2.	Bad Smells propuestos para BDR	55
5.3.	Organización Taxonómica	82
5.3.1	Categorías relacionadas	82
5.3.2	Aspectos arquitectónicos.....	82
5.3.3	Aspectos de integridad	84
5.3.4	Aspectos de Calidad	85
6.	<i>ESTRATEGIA PARA DETECCIÓN DE BAD SMELLS EN BASES DE DATOS RELACIONALES</i>	86
6.1.	Mecanismo de Operación.....	86
6.1.1	Detalle Técnico del Proceso de Detección de Bad Smells	87
6.1.2	Detalle Visual del Proceso de Detección de Bad Smells.....	89
6.2.	Cómo incluir un nuevo Bad Smell a BadSmellsBDR	90
6.2.1	Ajustes al Modelo.....	90
6.2.2	Ajustes al Controlador	90
6.2.3	Ajustes a la Vista	91
7.	<i>VALIDACIÓN DE LA ESTRATEGIA DE DETECCIÓN</i>.....	92
7.1.	Caso de Estudio.....	92
7.1.1	Marco Conceptual del caso seleccionado	92
7.1.2	Caso de estudio seleccionado	93
7.2.	Mecanismo de detección por intuición humana.....	94
7.2.1	Descripción de la encuesta	94
7.2.2	Descripción de los casos de evaluación.....	94
7.3.	Resultados Obtenidos	94
7.3.1	Caso – Tabla gruesa.....	95
7.3.2	Caso – Tabla gruesa.....	96
7.3.3	Caso – Tabla aislada.....	96
7.3.4	Caso – Tabla aislada.....	97
7.3.5	Caso – Llave primaria larga	97
7.3.6	Caso – Llave primaria larga	98
7.3.7	Caso – Llave primaria larga	98
7.3.8	Caso – Detección Bad Smells.....	99
7.3.9	Caso – Detección Bad Smells.....	99
7.3.10	Caso – Detección Bad Smells	100

7.4. Resultados Obtenidos	101
8. CONCLUSIONES.....	102
9. TRABAJO FUTURO	103
BIBLIOGRAFÍA.....	104
ANEXOS.....	106

LISTA DE FIGURAS

Pág.

Figura 1 Dimensiones de la reingeniería	17
Figura 2 Qué técnicas se debe utilizar para la comprensión del sistema.....	18
Figura 3 Modelo de herradura para el ciclo de la reingeniería	22
Figura 4 Propuesta para ingeniería en reversa de sistemas legados	23
Figura 5 Actividades Principales del modelo OAR.	24
Figura 6 Etapas que componen cada actividad del modelo OAR	25
Figura 7 Estándar propuesto por ANSI/IEEE para soportar el mantenimiento de software	28
Figura 8 Correlación de las etapas del ciclo de vida del desarrollo de software y ciclo para mantenimiento de software.....	29
Figura 9 Taxonomía propuesta por William Wake dentro de clases [WAKE00].....	31
Figura 10 Taxonomía propuesta por William Wake entre clases [WAKE00].....	32
Figura 11 Tipos de métricas	34
Figura 12 Proceso de reingeniería de bases de datos de Agarwal, Sarangi y Das.	36
Figura 13 Proceso de refactoring en bases de datos planteado por Ambler.....	38
Figura 14 Descripción de las tareas asociadas al proceso de refactoring en Bases de Datos (1)	39
Figura 15 Descripción de las tareas asociadas al proceso de refactoring en Bases de Datos (2)	39
Figura 16 Descripción de las tareas asociadas al proceso de refactoring en Bases de Datos (3)	40
Figura 17 Propuesta y Clasificación de refactorings para bases de datos de Ambler	41
Figura 18 Plantilla base para la definición de los bad smells	54
Figura 19 Taxonomía de bad smells BDR arquitectónicos- dentro de tablas.....	83
Figura 20 Taxonomía de bad smells BDR arquitectónicos- entre tablas.....	83
Figura 21 Taxonomía de bad smells BDR de integridad- dentro de tablas	84
Figura 22 Taxonomía de bad smells BDR de integridad - entre tablas	85
Figura 23 Taxonomía de bad smells BDR de calidad	85
Figura 24 Arquitectura de “BadSmellsBDR”	86
Figura 25 Patrón MVC en BadSmellsBDR	87
Figura 26 Parámetros de configuración para conexión a bases de datos –Archivo XML ..	87
Figura 27 Diagrama de secuencia para detección de bad smells en BadSmellsBDR	88
Figura 28 Orden en el cual visualmente se ejecuta la aplicación BadSmellsBDR	89
Figura 29 Modelo eTOM - TM Forum ®	93
Figura 30 Descripción del proceso completo de reingeniería.....	107
Figura 31 Fases a evaluar al iniciar un proyecto de reingeniería	108
Figura 32 Actividades de la etapa de valoración	109
Figura 33 Etapas en el análisis de la decisión.....	110

Figura 34 Etapas en el desarrollo de la solución propuesta en reingeniería.....	110
Figura 35 Actividades en la etapa de transición	111
Figura 36 Actividades de la etapa de mejora.	112
Figura 37 Paquetes marco de BadSmellsBDR.....	115
Figura 38 Estructura del paquete src.co.edu.uniandes.badsmells de BadSmellsBDR	116
Figura 39 Estructura del paquete general de BadSmellsBDR.....	117
Figura 40 Estructura del paquete logica de BadSmellsBDR	117
Figura 41 Interfaz Principal de BadSmellsBDR	123
Figura 42 Parámetros de configuración para conexión a bases de datos –Archivo XML	123
Figura 43 Solicitud de archivo de configuración para conexión a la base de datos.	124
Figura 44 Solicitud de parámetros para bad smell “tabla gruesa”	125
Figura 45 Resultado análisis bad smell tabla gruesa	125
Figura 46 Resultado análisis bad smell tablas aisladas	125
Figura 47 Solicitud de parámetros para bad smell “llave primaria larga”	126
Figura 48 Resultado análisis bad smell tabla gruesa	126
Figura 49 Resultado análisis bad smell tablas aisladas	126
Figura 50 Resultados de análisis de bad smell	127
Figura 51 Página principal de la encuesta.....	128
Figura 52 Página de presentación de la encuesta	130
Figura 53 Página de indicaciones de la encuesta	132

LISTA DE TABLAS

	Pág.
Tabla 1 Componentes de bajo nivel para hacer reingeniería.....	19
Tabla 2 Componentes de alto nivel para hacer reingeniería.....	20
Tabla 3 Cómo implementar las técnicas de reingeniería.....	20
Tabla 4 Puntos de intersección entre el ciclo de vida de mantenimiento de software, modelos de reingeniería y el proceso de refactoring.....	29
Tabla 5 Técnicas de aproximación para el descubrimiento del diseño de bases de datos relacionales.....	36
Tabla 6 Pautas para validar la calidad de un diseño de BDR.	44
Tabla 7 Desarrollo en reingeniería para comprensión, mantenibilidad y optimización en POO.....	52
Tabla 8 Desarrollo en reingeniería para comprensión, mantenibilidad y optimización en BD Relacionales.	53

LISTA DE ANEXOS

	Pág.
Anexo 1 Proceso de reingeniería en sistemas legados.....	107
Anexo 2 Propuesta de Bad Smells de Fowler	113
Anexo 3 Diseño de la aplicación BadSmellsBDR.....	115
Anexo 4 Requerimientos de la aplicación BadSmellsBDR.....	119
Anexo 5 Manual de usuario de la aplicación BadSmellsBDR	122
Anexo 6 Encuesta aplicada como elemento de validación.....	128
Anexo 7 Explicación gráfica de bad Smells.....	140

GLOSARIO

Bad Smells: Conjunto de 22 malas prácticas de codificación en aplicaciones orientadas a objetos, realizadas por Fowler [MFOW99]. Para el objeto de éste trabajo, también hace referencia a malas prácticas de diseño en bases de datos relacionales.

Garantizar el Comportamiento semántico: Propiedad que permite garantizar que al realizar *refactoring* se mantenga la semántica del código sobre el cual se está trabajando; esto implica que si se aplica sobre un código que contiene errores de lógica y tiene fallas funcionales la aplicación, éstas fallas no deben ser corregidas, pues se alteraría el principio de inocuidad funcional.

Ingeniería en reversa: Proceso que resulta de la comprensión de un sistema, a partir de su representación, como resultado se tiene una abstracción del sistema.

Mantenibilidad: Atributo del software que corresponde a la facilidad con la que se puede detectar errores, corregir o mejorar una aplicación.

Patrones de Bases de datos relacionales: Estructuras que independientemente del contexto, se repiten con frecuencia y permiten establecer similitudes en la estructura de los diseños a través del tiempo, como consecuencia permiten mejorar los tiempos y la calidad de los diseños ER que se elaboran evitando caer en los mismos errores repetidamente.

Problema de detección: Primera etapa del proceso de refactoring, cuyo objetivo es identificar las partes del sistema que presentan fallas o malas prácticas de diseño, y deben ser modificadas por etapas posteriores del proceso de refactoring.

Proceso de refactoring: Conjunto de actividades encaminadas a la transformación estructural de una aplicación, para mejorar su mantenibilidad, sin modificar su funcionalidad

Refactoring: Disciplina controlada que tiene como objeto reestructurar el código, fundamentalmente introduciendo pequeños cambios en la estructura del mismo.

Reingeniería: Conjunto de actividades que permiten la comprensión total del software, con el objeto de disminuir la complejidad de su mantenimiento, para una reconstrucción total o parcial, con el fin de incrementar su mantenibilidad, permitir que evolucione o se reutilice.

Sistemas legados: Son sistemas que se encuentran en funcionamiento durante muchos años, y envuelven gran parte del conocimiento corporativo, incluyendo requerimientos, decisiones de diseño y reglas del negocio.

Tabla base: Tablas del diseño ER, que se utilizan para administrar, consultar y actualizar información necesaria para poder establecer relaciones **padre** – hijo, **maestro** – detalle. Se caracterizan por que su información es general.

Taxonomía: Colección de términos organizados en una estructura jerárquica, donde es posible encontrar relaciones padre e hijo.

RESUMEN

El propósito de esta investigación surge ante la falta de claridad de cómo detectar estructuras mal formadas en bases de datos relacionales, a través de patrones definidos en los procesos de ingeniería en reversa. En la actualidad, el determinar si una base de datos se encuentra mal estructurada queda a juicio y conocimiento de quienes estén realizando esta evaluación, pues no se cuenta con una herramienta que les permita tener una guía confiable e imparcial.

Este documento se presenta como resultado de un detallado estudio, en los avances de la reingeniería, propone un conjunto de 29 bad smells, planteando una estrategia formal que permite fortalecer el análisis del modelo de bases de datos relacionales. Con esta propuesta, se busca complementar la ingeniería en reversa, reconociendo que éste tipo de persistencia es de uso masivo y fundamental.

Esta investigación permite concluir que, no solo es factible proponer técnicas de detección de malas prácticas de diseño OO, también es posible fortalecer el proceso de detección para esquemas de persistencia como lo son: las bases de datos relacionales. Éstas malas prácticas, no solo pueden ser detectadas por la experiencia humana, también pueden ser identificadas a través de herramientas que permitan con base en métricas una automatización de ésta actividad.

INTRODUCCIÓN

En la actualidad, el principal objetivo de la industria del software es la búsqueda constante de calidad. La preocupación de tener un equipo bien formado y contar con desarrolladores que codifiquen adecuadamente ya no es una inquietud. También es indispensable conocer como diseñar arquitecturas que cumplan con características de la industria como: estandarización, flexibilidad y modularidad, pues se ha comprendido que no es suficiente entregar un producto de software que satisfaga todos los requerimientos. Las casas de software deben preocuparse por entregar un producto completo y además mantenible.

Se ha identificado en la industria del software que el 20% del tiempo se invierte en el ciclo de vida hasta la etapa de implantación; el 80% restante se invierte en el mantenimiento, sea por corrección de errores o por adición de nuevas funcionalidades. Dentro de éste contexto, no es casualidad que en las primeras etapas del ciclo de vida se invierta el tiempo suficiente para construir un adecuado diseño de alto y bajo nivel, pues éstas características tendrán un impacto en costos y recursos en el momento de realizar mantenimiento al software: costos reflejados en el tiempo que se debe utilizar para la comprensión de software y recursos para mantener la estabilidad y operatividad de las aplicaciones al intervenir los diferentes componentes que la conforman.

Actualmente, el tiempo estimado para el desarrollo de software es considerable debido a la procura en la calidad. Sin embargo, es necesario complementar las etapas del ciclo de vida con herramientas y metodologías que permitan, en el momento de realizar mantenimiento, una comprensión más ágil del diseño del software. Con base en esta necesidad, surge la ingeniería en reversa y la reingeniería como áreas que permiten abordar de forma ordenada el mantenimiento del software. La primera se enfoca en la comprensión total de los sistemas existentes. La segunda proporciona lineamientos para rescatar componentes importantes de los aplicativos, prever los impactos por cambios, detectar defectos y corregirlos, procurando mantener la vida útil de las aplicaciones que apoyan y/o complementan la operación normal de la organización.

Para la ingeniería en reversa y reingeniería, es una constante generar metodologías y procedimientos que permitan obtener los mejores resultados en la etapa de mantenimiento del software, pues la continua implantación de nuevas funcionalidades y la corrección de errores pueden degenerar el diseño inicialmente concebido para las aplicaciones. Actualmente existen herramientas que permiten, por ejemplo, en el caso de la programación orientada a objetos, tener una comprensión dinámica y estática de cómo están conformados los sistemas y su comportamiento. Existen esfuerzos enfocados a la detección de malas prácticas de programación presentadas por Martin Fowler [MFOW99] denominadas *Bad Smells*. Con base en su propuesta, se han desarrollado herramientas que permiten su detección de una manera más formal, como es la propuesta de Beatriz Florián [BFLO05] quien presenta modelos de detección con base en métricas y la de

Carlos Angarita [CANG05] quien planea estrategias de detección a partir de representaciones XML.

Los avances en la reingeniería, en cuanto a la arquitectura y diseño de las aplicaciones OO, ha sido importante. Sin embargo, no se ha profundizado en la detección de malas prácticas en el diseño de las estructuras de almacenamiento, siendo éste un factor de vital importancia al realizar mantenimiento a los aplicativos: la persistencia, entonces, se identifica como un área donde no es del todo claro cómo apoyar la reingeniería.

El problema de detección de malas prácticas, bad smells, en el diseño de bases de datos relacionales, se convierte en un reto importante para complementar el proceso de reingeniería.

OBJETIVO GENERAL DEL PROYECTO

- ⌘ Proponer un conjunto de *Bad Smells* que pueden ser detectados en el momento de realizar reingeniería a bases de datos relacionales, clasificarlos en una taxonomía por categorías relacionadas que permita en el momento de realizar reingeniería ser detectados, clasificados y comprendidos con mayor agilidad.

OBJETIVOS ESPECÍFICOS DEL PROYECTO

- ⌘ Apropiación del estado del arte de conceptos tales como:
 - Ingeniería en reversa y reingeniería.
 - Patrones y *Bad Smells*.
 - *Bad Smells* definidos para la comprensión en la programación orientada a objetos.
 - Sanas prácticas en el diseño lógico de bases de datos relacionales.
- ⌘ Caracterizar *Bad Smells* para bases de datos relacionales.
- ⌘ Proponer la taxonomía bajo la cual serán catalogados los *Bad Smells*.
- ⌘ Desarrollar una o varias estrategias para la detección de los *Bad Smells* definidos en bases de datos relacionales
- ⌘ Efectuar un proceso de validación de una de las estrategias propuestas para la detección de uno o varios *Bad Smells* propuestos en la taxonomía.

MARCO TEÓRICO

1. EVOLUCIÓN DEL SOFTWARE

Los productos de software sin importar cuál es su objeto están llamados a permanecer en constante evolución, gracias a los cambios tecnológicos ante los cuales se encuentran expuestos, no es suficiente tener un software completo y robusto, si no se cuenta con la tecnología en hardware requerida para que las aplicaciones mantengan la eficacia bajo las cuales fueron construidas, a la vez cuando la tecnología en hardware evoluciona se exige de los aplicativos mayor portabilidad, la cual debe ser tomada en cuenta en el momento del diseño de las soluciones de software.

Ahora bien, no solo los productos de software deben permanecer en constante evolución por los cambios tecnológicos, también deben adaptarse a la organizaciones a través del tiempo. Puede que en un momento dado, una solución de software hecha a la medida o comprada cubra todas las necesidades de una organización sin embargo, los productos, servicios y procesos varían con el tiempo y esto hace que las aplicaciones deban ser adaptadas o renovadas.

Por lo anterior, el mantenimiento del software ha sido históricamente una etapa costosa, que exige especial cuidado en el momento de ser realizado, puesto que cambios pequeños en los aplicativos no solo tienen un gran impacto en el producto de software (entiéndase la descripción de requerimientos, definición de la arquitectura, diseño y documentación), sino que también pueden alterar el comportamiento y los resultados normales esperados en el ambiente en el que se desempeñan. Cualquiera que sea la necesidad de mantenimiento, sea por: corrección de errores, adición de nuevas funcionalidades o mejora en el entendimiento del aplicativo (documentación), las organizaciones buscan mejorar la productividad de las aplicaciones a un menor costo, en un menor tiempo en comparación al requerido del momento en que se implementó la solución inicial y mantener la documentación completa.

Esta evidencia hace que se exija no solo a la academia sino a la industria, que investigue y apoye la etapa de mantenimiento a través de metodologías y procedimientos, buscando aumentar la productividad con mejores tiempos de respuesta, menor esfuerzo y un máximo aseguramiento en la calidad del proceso, para mantener el producto de software consistente en el tiempo.

Como respuesta a esta necesidad, surge la *reingeniería* como una disciplina en donde se requiere un alto nivel de planeación y un conjunto de técnicas estructuradas, orientadas

no solo a la actualización de sistemas legados, sino también a sistemas desarrollados con enfoques modernos que requieren evolucionar.

2. REINGENIERÍA COMO DISCIPLINA

Son varios los conceptos que se pueden encontrar de *reingeniería* bajo una cuidadosa revisión bibliográfica, sin embargo existen definiciones que tienen un gran respaldo por parte de la comunidad académica y la industria del software que se complementan.

Definición [BATI92]: Es la reconstrucción de un sistema de software o un componente con el objeto de satisfacer un nuevo propósito.

Definición [ARNO93]: Es un conjunto de actividades que: (1) Mejora el entendimiento del software, (2) Mejora el software en si mismo, normalmente para incrementar su mantenibilidad y permitir que evolucione o (3) reutilizarlo.

Lo anterior apoyado de un conjunto de actividades según el propósito.

Definición [CHIK92]: Consiste en inspeccionar un sistema para reestructurarlo de una nueva forma seguido por su implementación de esta nueva forma.

Para el objeto de este proyecto, se entenderá *reingeniería* como el conjunto de actividades que permiten la comprensión total del software, con el objeto de disminuir la complejidad de su mantenimiento, para una reconstrucción total o parcial, con el fin de incrementar su mantenibilidad, permitir que evolucione o se reutilice.

Todo proceso de reingeniería, sin importar cual es su fin, debe cubrir al menos una de las siguientes características:

- ☞ Mejorar la eficiencia, en el consumo de recursos especialmente tiempo y dinero, por parte del sistema.
- ☞ Efectividad, en el acoplamiento a los cambios en los procesos organizacionales.

No en todos los casos en los que se requieren renovar los sistemas aplica la reingeniería, adjunto se encuentra una lista ocasiones en las cuales es posible aplicar un proceso de reingeniería:

- ☞ Migrar la aplicación escrita de un lenguaje obsoleto a un lenguaje más completo.
- ☞ Generar documentación de las aplicaciones
- ☞ Conocer en profundidad la aplicación que soporta el core de una organización.
- ☞ Obtener el código fuente de una aplicación que ha sido comprada y en el momento de requerir un cambio no cuenta con el soporte necesario.
- ☞ Adicionar nuevas funcionalidades a la aplicación o mejorar las ya existentes.
- ☞ Valorar la calidad del sistema

- 3 Extraer/convertir la información.
- 3 Administrar la información.
- 3 Reutilizar componentes.

2.1. Guía para el proceso de Reingeniería

Robert Arnold [ARNO93][ARNO96] proporciona un marco de trabajo bajo el cual se puede guiar un proceso de reingeniería, según el tipo de actividad que se desea realizar y el sistema que se desea intervenir.

Las actividades de reingeniería se pueden definir con base en una serie de interrogantes (Figura 1) que al ser resueltos guían el proceso, éstos son:

- 3 ¿Qué tipo de transformación se desea realizar? (*¿Qué?*)
- 3 ¿Cuál es el objetivo de la reingeniería? (*¿Por qué?*)
- 3 ¿Sobre qué artefactos se llevará a cabo la transformación? (*¿Sobre qué?*)
- 3 ¿A través de cuál técnica de reingeniería? (*¿Cómo?*)



Figura 1 Dimensiones de la reingeniería

2.1.1 ¿Qué tipo de transformación se desea realizar? (*¿Qué?*)

Este enfoque se ocupa de la técnica que se debe utilizar según el objetivo de reingeniería y las transiciones a través de las diferentes etapas del ciclo de vida del sistema, dentro de éstas se puede identificar (Figura 2):

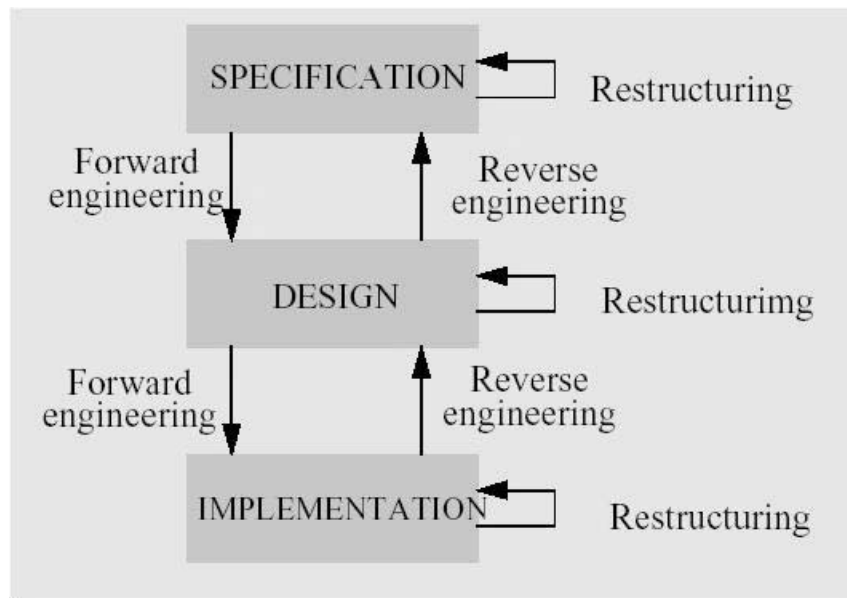


Figura 2 Qué técnicas se debe utilizar para la comprensión del sistema.

Ingeniería hacia delante (Forward engineering): Proceso que involucra: la abstracción (comprensión), especificación del sistema para realizar el diseño y con base en éste, realizar la implementación (representación).

Ingeniería en reversa (Reverse engineering): Proceso que resulta de la comprensión de un sistema a partir de su representación, como resultado se tiene una abstracción del mismo.

La ingeniería en reversa puede enfocarse en dos técnicas:

- ⌘ **Redocumentación:** Genera una vista alternativa del sistema, que se analiza a partir de la generación de diagramas (UML, control de flujo) con base en el código fuente.
- ⌘ **Recuperación del Diseño:** A partir del código fuente y el conocimiento de la operación del sistema, se extrae el diseño y las reglas de negocio del sistema.

Reingeniería: A partir de un sistema (representación) existente, se realizan cambios y se obtiene el mismo sistema con modificaciones y/o adiciones en su funcionalidad.

Puede enfocarse en dos técnicas:

- ⌘ **Reestructuración (Refactoring):** Se mantiene la funcionalidad intacta del sistema y sin embargo, se modifica su arquitectura obteniendo así sistemas equivalentes.
- ⌘ **Reingeniería (Reengineering):** A partir del sistema existente, se rescatan las reglas de negocio y funcionalidades core. A través de cambios y

transformaciones se obtiene un nuevo sistema, con funcionalidades adicionales y reutilización de componentes.

2.1.2 ¿Cuál es el objetivo de la reingeniería? (¿Por qué?)

Normalmente la reingeniería se realiza con diferentes objetivos, dentro de los cuales podemos encontrar:

- ☞ *Mantenibilidad:* Facilita el mantenimiento de software existente, su entrada es un sistema no estructurado que posteriormente se convierte en un sistema fácilmente mantenible.
- ☞ *Reutilización:* Extrae del sistema componentes que son reutilizables, para vincularlos a sistemas existentes o nuevos productos de software.
- ☞ *Documentación:* Permite generar vistas alternativas del producto, es útil cuando se está dentro de una etapa de implementación. Se extrae la información que depende de un nuevo desarrollo y chequea que la descripción de la arquitectura este conforme.
- ☞ *Integración:* Útil en el momento en que se desea integrar una aplicación con otras dentro o fuera de la organización, para permitir la comunicación entre diversos sistemas.

2.1.3 ¿Sobre qué artefactos se llevará a cabo la transformación? (¿Sobre qué?)

El reconocer sobre qué tipos de artefactos que componen el software se realizará la reingeniería, es un factor clave para realizar una planeación adecuada del proceso. Para detectar los componentes primero se debe identificar si corresponden a un detalle de bajo o de alto nivel.

Dentro de los componentes con un nivel de detalle bajo, se puede especificar (Tabla 1):

Algoritmos	Estructuras de Datos
Instrucciones Operaciones Procedimientos	Constantes Variables Tipos de Datos Clases
Flujo de Control	Jerarquía de tipos

Tabla 1 Componentes de bajo nivel para hacer reingeniería

Dentro de los componentes con un nivel de detalle alto, se puede identificar (Tabla 2):

Arquitectura	Implementación	Variación	Evolución
Entendimiento de: Módulos Interfaces Sub. sistemas	Objetos fuente Objetos Derivados (Entidades)	Parámetros del sistema	Revisiones Información Histórica Releases

<i>Relaciones de dependencia</i>	<i>Relaciones entre entidades</i>	<i>Compatibilidad entre sistemas</i>	<i>Relaciones con sucesores</i>
----------------------------------	-----------------------------------	--------------------------------------	---------------------------------

Tabla 2 Componentes de alto nivel para hacer reingeniería

2.1.4 ¿A través de cuál técnica de reingeniería? (¿Cómo?)

Este enfoque se ocupa de cómo se va a aplicar la técnica de reingeniería seleccionada. Dentro de las opciones se puede identificar si la técnica se va a aplicar manual, asistida o automática, cómo se realizará el proceso de extracción y su correspondiente representación.

Manual	Asistido por Computador					
	Análisis Estático	Análisis Dinámico	Consultas	Explorar	Visualización	Animación
A partir de la lectura del código fuente se adicionan comentarios	Análisis del flujo de datos.	Ejecución paso a paso.	Cuales procedimientos varían con el cambio de una variable X.	Explorar el código fuente	Presentación de diagrama de clases o diferentes diagramas.	Presentando la pila de ejecución
		Perfilamiento				
	Evaluación parcial					
	Extracción de la información		Presentación de la información			

Tabla 3 Cómo implementar las técnicas de reingeniería

En general, para cada uno de los componentes identificados sobre los cuales se debe realizar reingeniería, se debe identificar: por qué se le va a hacer, qué se le va a hacer y cómo se va a implementar el proceso de reingeniería. Una vez identificados todos estos componentes, se debe proceder a realizar un plan que permita hacer seguimiento al proceso.

2.2. Enfoques de la Reingeniería

La disciplina de la reingeniería presenta cuatro enfoques, como sigue:

- ☞ *Reingeniería de procesos:* Se enfoca en el análisis y reestructuración de los procesos organizacionales.
- ☞ *Reingeniería de software:* Se enfoca al mantenimiento de programas y artefactos de software.
- ☞ *Reingeniería de datos:* Se enfoca en el análisis y reestructuración de bases de datos, tanto en su estructura lógica y física. Pues ellas proveen la información e infraestructura necesaria para soportar requerimientos, eventualmente puede cambiar el diseño, y los tipos de datos requeridos para soportar el negocio.
- ☞ *Híbrida:* Se enfoca en los procesos de negocio, liderando cambios en los modelos, la estructura de las aplicaciones y la base de datos.

De acuerdo con las dimensiones (Figura 1) y enfoques de la reingeniería (2.2), ésta investigación se enfoca en la reingeniería de software y datos. Orientando la comprensión del diseño de bases de datos y posterior planteamiento de la taxonomía de los bad smells propuestos, que podrán apoyar las técnicas de *Reestructuración (Refactoring)* y *Reingeniería (Reengineering)*.

2.3. Modelos de Reingeniería

El SEI (Software Engineering Institute) es tradicionalmente la organización que proporciona los lineamientos formales para la ingeniería de software, dentro de ésta, la reingeniería se articula como un área importante de investigación y fortalecimiento del proceso de mantenimiento del software.

En la evolución de la reingeniería y los modelos de reingeniería planteados por el SEI, podemos identificar la evolución histórica, así:

El primer planteamiento se presenta en 1995, con “Perspectives on Legacy System Reengineering”, un breve resumen del enfoque se presenta a continuación.

2.3.1 Ingeniería en Reversa de software y datos en sistemas legados

Los sistemas legados son un caso interesante, no solo por la reingeniería del software, también por el proceso de reingeniería de sus datos; el interés especial por los sistemas legados, se fundamenta en la dificultad de actualización debido a la alta probabilidad de estar escritos en lenguajes no comerciales y tener persistencia en bases de datos sobre las cuales actualmente no se brinda soporte, o el conocimiento se encuentra concentrado en unas pocas personas, por lo que se hace especialmente costoso.

Estos sistemas presentan una alta probabilidad de haber sufrido modificaciones poco estructuradas, que por la poca cultura de la calidad, probablemente no se encuentren documentadas, como consecuencia es probable que la documentación y diseño tampoco lo estén.

El SEI ha desarrollado un banco de trabajo para la reconstrucción de arquitecturas, con el perfil anteriormente planteado, que busca la integración entre herramientas y técnicas existentes para su reconstrucción efectiva.

En 1998 a través de la presentación de “Reengineering: The Horseshoe Model”, se realiza un modelo que permite identificar cada una de las etapas del proceso de reingeniería.

2.3.2 Modelo de herradura

Es un modelo conceptual [KAZM98] que permite caracterizar diferentes niveles de análisis en reingeniería y proporciona una base para transformaciones arquitectónicas en cada nivel.

La propuesta realizada por Kazman está compuesta por tres etapas (Figura 3):

- 3 Análisis del sistema existente.
- 3 Transformación lógica
- 3 Desarrollo de un nuevo sistema.

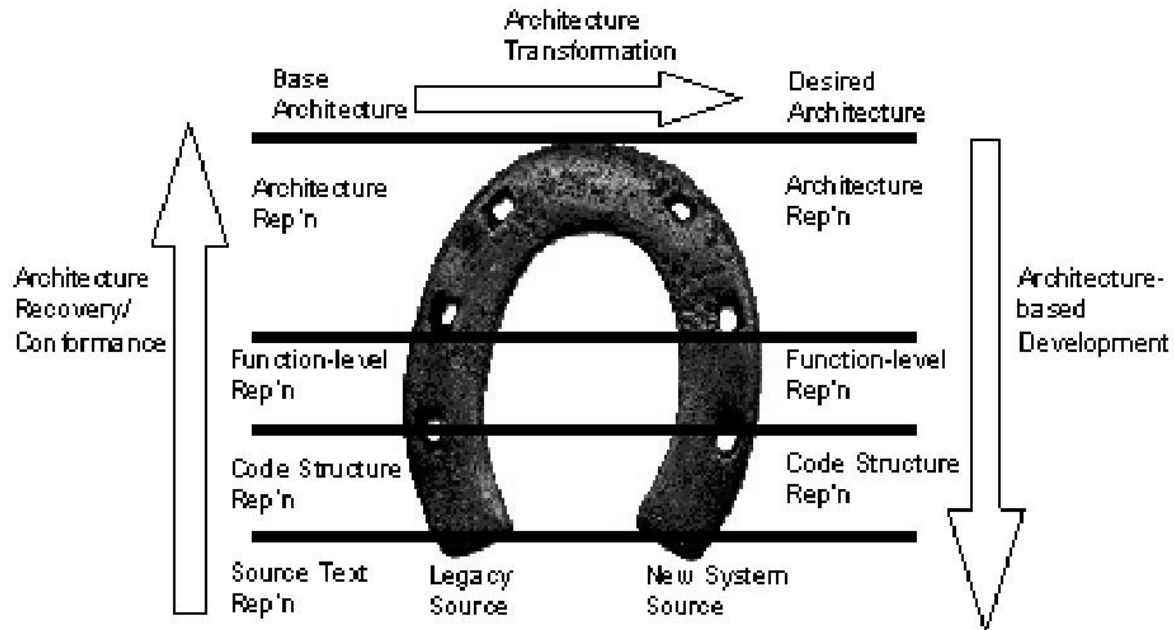


Figura 3 Modelo de herradura para el ciclo de la reingeniería

3 Análisis del sistema existente:

Recupera la arquitectura extrayendo los artefactos del código fuente. Con ésta, se determina si está conforme al diseño inicialmente concebido, analizando requerimientos de calidad como desempeño, fácil modificación, seguridad, y/o confiabilidad.

3 Transformación lógica:

Una vez se cuenta con la arquitectura, se realiza un proceso de transformación lógica planteando puntos y características que deben ser modificados para obtener la arquitectura deseada. Se evalúan también algunos requisitos como calidad, restricciones económicas y el objeto de la organización en donde se implantará el nuevo sistema.

3 Desarrollo de un nuevo sistema.

Una vez se cuenta con la arquitectura deseada, se agrupan los problemas detectados y se deciden estrategias de interconexión entre ellos de acuerdo a los tipos de problemas y la complejidad con que se deben tratar.

El modelo "herradura" describe un conjunto de opciones técnicas que los ingenieros pueden aplicar. Al ser un modelo conceptual no reemplaza técnicas de reingeniería o refactoring, en cambio permite complementar el proceso de comprensión y evolución del software, conformándose como carta de navegación para el mantenimiento del mismo.

En 1999, se realiza una mejora a la propuesta de ingeniería en reversa, se propone el proceso en tres etapas como se ve en la Figura 4.[KAZM99].

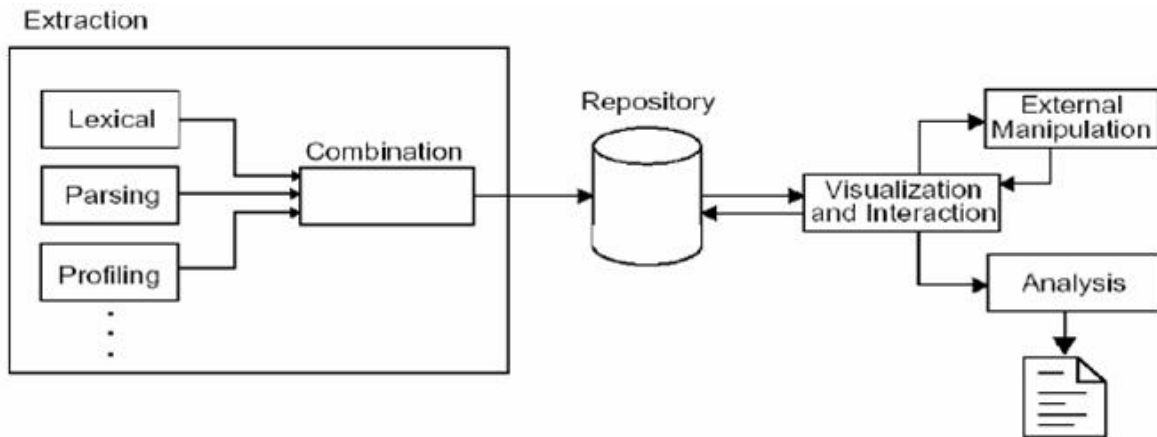


Figura 4 Propuesta para ingeniería en reversa de sistemas legados

Fases:

- ☞ *Extracción*, de la arquitectura a partir de artefactos de bajo nivel como código fuente y makefiles.

El modelo que se obtiene, es un conjunto de elementos y relaciones (funciones, archivos, variables, objetos, llamado a funciones, acceso a archivos, entre otros) que deben reflejar la estructura estática y dinámica del sistema.

- ☞ *Definición de patrones de diseño*, de ésta forma se permite realizar una correlación entre la arquitectura real y la diseñada inicialmente, el objetivo es hacer explícitos los enlaces entre la arquitectura de diseño con la extraída, a partir de los artefactos de bajo nivel.
- ☞ *Visualización* de la arquitectura resultante, donde se presenta la información extraída según los patrones definidos en la etapa anterior y se valida.

La reingeniería y la ingeniería se pueden ver como un proceso para resolver un problema donde hay que entender lo que se quiere desarrollar o en su defecto lo que existe, realizar un plan para llevar a cabo el desarrollo o adecuación, en este caso, de los sistemas legados, y proceder a desarrollar la solución. Con la existencia de sistemas legados existen restricciones sobre las actividades que se deben realizar para llegar a la solución que se desea, lo que inevitablemente requiere un proceso de reingeniería, no es suficiente con aplicar ingeniería en reversa. Para mayor documentación Anexo 1 Proceso de reingeniería en sistemas legados.

En 2001, al comprender la limitada aceptación del modelo de herradura por ser una propuesta conceptual, éste se tomó como punto de partida para construir una propuesta estructurada, que aplicara con mayor claridad a diversas situaciones del mundo real, la

propuesta recibió el nombre de “Options Analysis for Reengineering OAR” el modelo se es una ayuda para apoyar decisiones de reingeniería y refactoring de forma sistemática.

2.3.3 Modelo de OAR (Options Analysis for Reengineering)

Modelo conceptual [BERG01] sistemático, centrado en la arquitectura, permite tomar decisiones para identificar y detectar componentes de software, inclusive en sistemas de software complejos. La identificación de componentes involucra la rehabilitación para una posterior reutilización.

OAR permite identificar componentes de la arquitectura relevantes y el análisis de los campos que se requieren, para ser utilizados en productos de software o nuevos sistemas, todo esto se logra a través de estimación de costos, esfuerzos y riesgos asociados al proceso de reingeniería.

El modelo consiste en 5 grandes actividades, como sigue (ver Figura 5):

3 Determinar el contexto:

Busca comprender las necesidades de la organización, el sistema existente, sus fundamentos, y las expectativas/ lineamientos para el descubrimiento de componentes sobre los cuáles se realizará reingeniería.

3 Inventario de componentes:

Se identifican los componentes del sistema que son potencialmente útiles para ser utilizados como core para el proceso de reingeniería

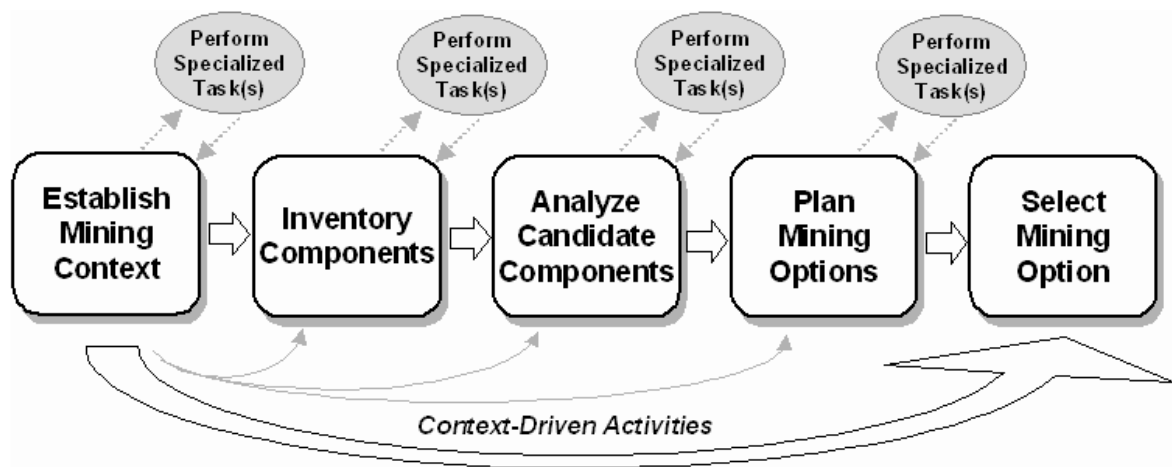


Figure 1. Overview of OAR Activities

Figura 5 Actividades Principales del modelo OAR.

3 Análisis de componentes candidatos:

De acuerdo con el inventario realizado, se toma cada uno de los componentes y se evalúa su potencial para ser utilizado como un elemento, para el producto de software que se está trabajando.

3 *Plan de opciones:*

Con base en los componentes candidatos previamente analizados, se determinan las alternativas de desarrollo bajo las consideraciones de tiempo, costo, esfuerzo, riesgo y recursos disponibles.

3 *Selección de las mejores opciones:*

Con base en el plan y análisis realizado anteriormente, se seleccionan las mejores opciones o combinación de opciones, de acuerdo a un balance entre programación requerida y las consideraciones técnicas.

Cada una de las anteriores actividades están compuestas por tareas y subtareas de diseño, que responden a un conjunto de interrogantes que permiten definir las actividades necesarias, y sirven como lista de chequeo para determinar los criterios de salida de cada una de estas actividades.

La estructura de cada actividad responde a un formato EITVOX (Criterios de entrada, entradas, Tareas, Validaciones, Resultados, Criterios de salida), como sigue (Figura 6):

Table 1. EITVOX Structure of OAR Activities



E	Entry Criteria	Criteria for beginning activity
I	Inputs	Artifacts required during execution of activity
T	Tasks	Tasks to carry out the prescribed activity
V	Validation	Data collected to validate the activity
O	Outputs	Artifacts produced during execution of activity
X	Exit Criteria	Criteria defining completion of activity

Figura 6 Etapas que componen cada actividad del modelo OAR

3. REFACTORING

Como se ha mencionado en 2.1.1, el *refactoring* hace parte de los tipos de transformaciones que se puede aplicar cuando se emplea la reingeniería como metodología para el mantenimiento, reutilización y/o integración de las aplicaciones.

Históricamente, el *refactoring* se ha conocido como reestructuración del software. Sin embargo, no hay una gran distinción entre estos términos, por lo general la reestructuración hace referencia a programación o procedimientos mientras que *refactoring* hace referencia a la programación orientada a objetos.

3.1. ¿Qué es *refactoring*?

Se define como el uso de diferentes técnicas para mejorar la integridad estructural de las aplicaciones, manteniendo la funcionalidad intacta de los sistemas para obtener así sistemas equivalentes.

En una definición formal *Martin Fowler* [MFOW99] describe *Refactoring* (verbo) como una disciplina controlada que tiene como objeto reestructurar el código, fundamentalmente introduciendo pequeños cambios en la estructura del código.

La anterior definición describe *refactoring* como verbo, sin embargo *Martin Fowler* [MFOW99] describe *Refactoring* como sustantivo, así:

- ↯ Cambio en la estructura interna del código para un entendimiento más fácil.
- ↯ Cambio que permite modificar con mayor facilidad la estructura del código, sin cambiar el comportamiento observable

En conjunto se entiende *refactoring* como reestructurar el código aplicando una serie de *refactorings*.

3.1.1 Características

- ↯ Permite hacer cambios en el tiempo, para realizar así un proceso iterativo e incremental.
- ↯ Mantiene el comportamiento semántico del sistema.
- ↯ No se adiciona funcionalidad cuando se realiza el proceso de *refactoring*.
- ↯ Realizar *refactoring* y adicionar funcionalidad son dos actividades complementarias en la labor de reingeniería.
- ↯ Al introducir una serie de pequeños *refactoring* se puede tener un rediseño significativo de las aplicaciones.
- ↯ Al realizar una serie de *refactoring* se reduce el riesgo de introducir errores.

Cuando se “Mantiene el comportamiento semántico del sistema”, se debe garantizar que al realizar *refactoring* se mantenga la semántica del código sobre el cual se está trabajando; esto implica que si se aplica sobre un código que contiene errores de lógica, que genera fallas funcionales en el comportamiento de la aplicación, éstas fallas no deben ser corregidas, pues se alteraría el principio de inocuidad funcional.

3.1.2 ¿Por qué se hace necesario realizar refactoring?

- ☞ Mejora el diseño del software a través del tiempo.
- ☞ Hace el código más fácil de entender
- ☞ Permite encontrar bugs.
- ☞ Permite una programación más ágil.

3.1.3 Proceso de Refactoring

Si se profundiza en las técnicas de *refactoring* y *reingeniería*, se puede observar que se pueden utilizar conjuntamente, el primero es una actividad dentro del proceso de reingeniería. Adicionalmente, si se tienen presente las actividades que se requieren para realizar un adecuado *refactoring*, es más conveniente hacer referencia al *proceso de refactoring*.

En [MENS04], se plantean las actividades en las cuales se debe trabajar un proceso de refactoring, **et al**:

1. Identificar los puntos donde se debe realizar *refactoring*.
2. Determinar cuáles refactoring se deben aplicar en los puntos antes identificados.
3. Asegurar la calidad de los refactoring que se van a aplicar, y preservar la inocuidad funcional.
4. Aplicar refactoring seleccionados.
5. Evaluar efectos de los refactoring aplicados en términos de calidad de software (complejidad, mantenibilidad), y también evaluar efectos sobre el proceso (costos, esfuerzo).
6. Realizar mantenimiento a los artefactos de software (documentación, diseño) para mantener la integridad del producto.

Para realizar un proceso de refactoring, adicional a la serie de actividades que se deben tener presentes, anteriormente expuestas, es indispensable que exista un plan y casos de prueba que permitan identificar en cualquier momento si el desarrollo cumple con el principio de inocuidad funcional.

3.2. Mantenimiento de Software, Modelos de Reingeniería y Proceso de Refactoring

En general, a la etapa de mantenimiento del ciclo de vida del software, se han asociado estándares y lineamientos que permiten definir en grandes rasgos las fases por las que debe pasar el software cuando se realiza ésta labor, en [AIEM92] se evidencia el estándar

proporcionado por ANSI/IEEE para seguir el proceso de mantenimiento de software. Ver Figura 7.

Ciclo de Vida del Mantenimiento de Software

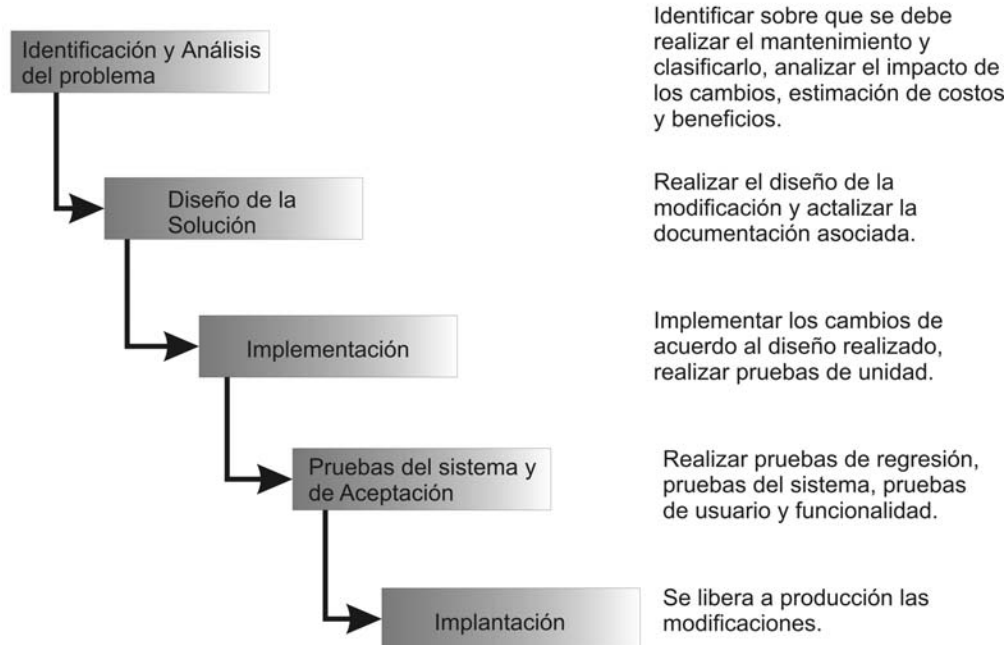


Figura 7 Estándar propuesto por ANSI/IEEE para soportar el mantenimiento de software

Al realizar una comparación entre el ciclo de vida para el mantenimiento propuesto, con respecto al ciclo de vida del software (modelo cascada), existe una gran correlación entre las etapas, como se puede observar en la Figura 8.

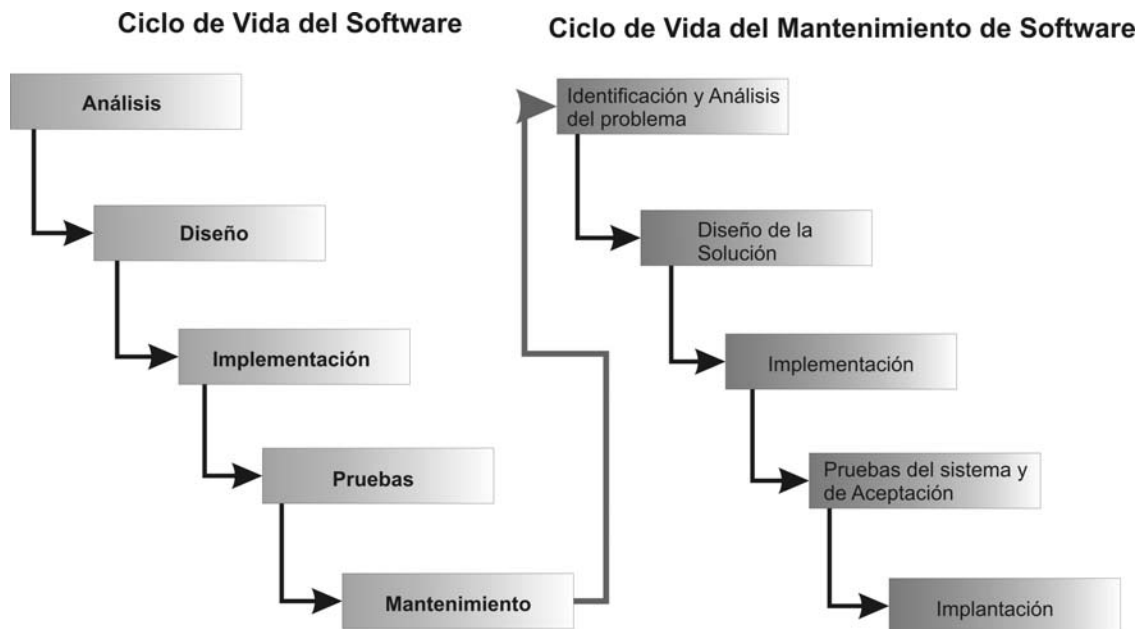


Figura 8 Correlación de las etapas del ciclo de vida del desarrollo de software y ciclo para mantenimiento de software.

Aunque este estándar dado por ANSI/IEEE se publicó en el año de 1992, la reingeniería ha estado presente desde el inicio de la ingeniería de software, no obstante se ha fortalecido de forma acelerada en las últimas décadas dadas las necesidades del mercado. Como una disciplina, la reingeniería siempre ha buscando responder con altos estándares de calidad a todas aquellas actividades que permiten asegurar la integridad y funcionalidad de los productos de software, por tal razón no es coincidencia que los modelos planteados en 2.3 soporten parte de las etapas que conforman el ciclo de vida para el mantenimiento de software.

Al identificar que el proceso de refactoring hace parte de los modelos de reingeniería, es posible plantear un escenario en el cual se pueden identificar cuáles son los puntos donde intervienen actividades del proceso. En la Tabla 4 se presenta un cuadro que permite la ubicación conceptual de los puntos de intersección entre el ciclo de vida, los modelos de reingeniería y el proceso de refactoring.

Ciclo de Vida del Mantenimiento de Software	Modelo Herradura	Modelo OAR	Proceso Refactoring
Identificación y Análisis del problema	- Análisis del Sistema Existente	- Determinar el contexto. - Inventario de Componentes.	
Diseño de la Solución	- Transformación lógica de la arquitectura	- Análisis Componentes Candidatos - Plan de Opciones. - Selección Mejor Opción	- Id. Puntos donde se debe hacer <i>refactoring</i> . - Determinar que <i>refactorings</i> se deben aplicar en los puntos antes identificados. - Asegurar la calidad del <i>refactoring</i> a aplicar.
Implementación	- Desarrollo de la nueva arquitectura		- Aplicar <i>Refactorings</i> .
Pruebas del sistema y de Aceptación			- Evaluar efectos del <i>refactoring</i> en calidad y procesos. - Mantenimiento artefactos de software.
Implantación			

Tabla 4 Puntos de intersección entre el ciclo de vida de mantenimiento de software, modelos de reingeniería y el proceso de refactoring.

3.3. Refactoring a partir de Bad Smells

La “*identificación de puntos donde se debe realizar refactoring*” es una de las actividades¹ que demanda más tiempo, puesto que es en ésta etapa en la que se deben detectar los puntos en el código donde existen fallas del diseño y posteriormente determinar que refactorings deben ser aplicados. De manera similar al diseño de software, ésta es una actividad donde se debe invertir la mayor parte del tiempo. Para mitigar el impacto y las

¹ Actividades asociadas al proceso de refactoring.

demoras en esta etapa, se han realizado varios esfuerzos para automatizar esta fase, sin embargo, no es fácil por medio de herramientas, detectar partes de código mal estructuradas debido a que cada arquitecto y diseñador de software maneja su propio estilo y refleja su criterio de diseño en los proyectos en los cuales participa.

Un logro importante para fortalecer la arquitectura y diseño en el desarrollo de software, se ha dado en la programación orientada a objetos a través de los *patrones de diseño*, cuando en 1977 el arquitecto Christopher Alexander [ALEX77] presentó ésta propuesta como un método sistémico, a través del cual ante problemas de diseño recurrentes se propone la mejor práctica, en 1994 Erich Gamma consolida nuevos patrones de diseño en su libro Design patterns [GAMM94].

Al aplicar reingeniería a proyectos de software, con diseños guiados por patrones, se ha mejorado el tiempo necesario para detectar los puntos donde se debe aplicar *refactoring*, sin embargo, el problema de detección continúa teniendo ciertos inconvenientes, listados a continuación:

- ⌘ No es un proceso repetible, al no ser totalmente automático.
- ⌘ El tiempo que se debe invertir para detectar problemas de diseño continúa siendo alto.
- ⌘ No es viable aplicar el mismo procedimiento de detección de problemas, en varios proyectos de ingeniería, por la complejidad en los diseños.

El *refactoring* como actividad se ha desarrollado desde los años 70's, sin embargo, el entusiasmo reciente se genera gracias al planteamiento de mejores formas para detectar dónde aplicarlo, a través de la propuesta de *Bad Smells* realizada por Fowler [MFOW99]. Ésta propuesta, surge como respuesta al uso de patrones de diseño en la ingeniería hacia delante.

En su propuesta Fowler [MFOW99], define el término *Bad Smell* como aquella mala práctica de programación, en total en su libro plantea 22 *bad smells* orientados a la programación orientada a objetos, ver Anexo 2. Posteriormente Wake [WAKE00] fortalece la definición de bad smell, como aquella señal ante problemas potenciales localizados en el código, que aunque en ocasiones son fáciles de identificar no necesariamente son fáciles de solucionar.

La propuesta de Fowler, es cuestionada a través de trabajos posteriores, como los realizados por Mantyla [MANT03] y Wake [WAKE00]. Dentro de las principales críticas se encuentran:

- ⌘ La forma en que se definieron los bad smells, pues se presentan como una lista única que hace difícil comprender el alcance de la propuesta.
- ⌘ No es fácil comprender los bad smells presentados ni las relaciones entre ellos.
- ⌘ Se plantea que la única forma de identificar los bad smells es a través de la intuición humana, lo que hace obsoleta la propuesta en el tiempo.
- ⌘ Si la responsabilidad de detección de bad smells está en la intuición humana, el proceso se puede corromper puesto que no todo el mundo tiene el mismo juicio para detectar los bad smells

- 3 El que se plantee que ningún conjunto de métricas, puede identificar la necesidad de refactoring.

A continuación se presenta el trabajo de Wake [WAKE00], donde realiza una propuesta taxonómica al organizar los bad smell, propuestos por Fowler, en categorías relacionadas y con nuevas adiciones (Figura 9, Figura 10).

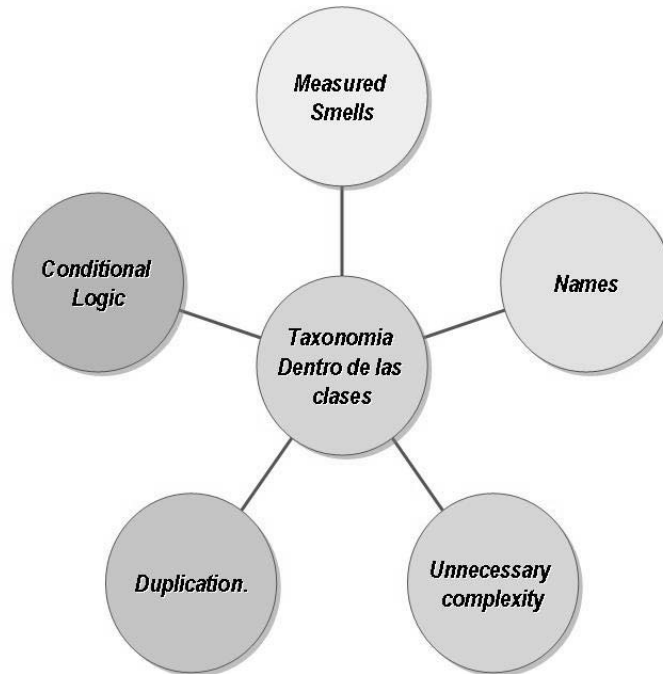


Figura 9 Taxonomía propuesta por William Wake dentro de clases [WAKE00]

- 3 Measured smells → Agrupa bad smells que son fácilmente detectables a través de métricas de software. Donde cada métrica tiende a capturar diferentes aspectos que permiten localizar los siguientes bad smells.
 - o Comments
 - o Long Method
 - o Large Class
 - o Long Parameter List
- 3 Names → Agrupa bad smells que permiten detectar nombres asociados a las estructuras que no permiten un fácil entendimiento y comunicación
 - o Type embedded in name
 - o Uncommunicative name
 - o Inconsistent names.
- 3 Unnecessary Complexity → Agrupa bad smells que permiten identificar partes del código que solucionan problemas simples de forma demasiado complicada.
 - o Dead Code
 - o Speculative Generality

- 3 Duplication → Agrupa bad smells asociados a estructuras de código o semántica duplicada a través de la aplicación.
 - o Duplicated Code
 - o Magic Number
 - o Alternative classes with different interfaces
- 3 Conditional Logic → Agrupa bad smells que detectan uso excesivo de condicionales.
 - o Null Check
 - o Complicated Boolean Expression
 - o Special Case
 - o Simulated Inheritance (Switch Statement)

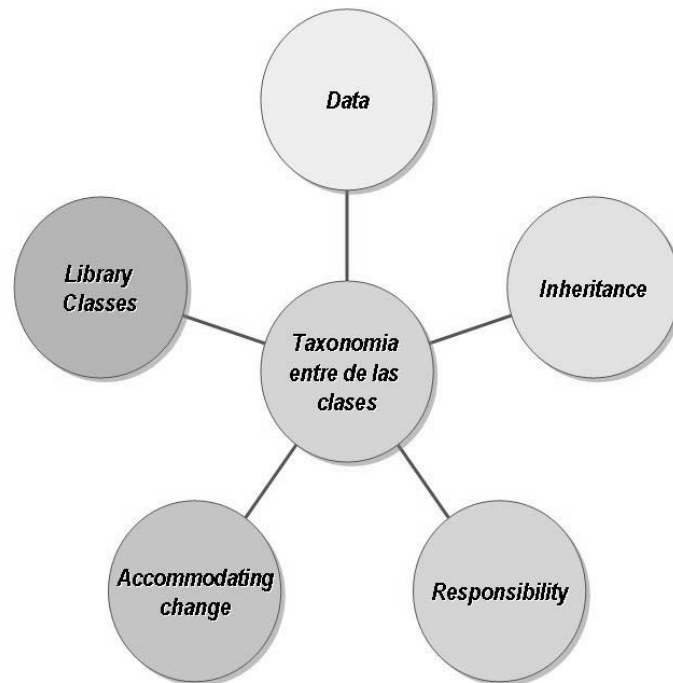


Figura 10 Taxonomía propuesta por William Wake entre clases [WAKE00]

- 3 Data → Agrupa bad smells que proporcionan indicios de clases diseñadas de forma inadecuada respecto a sus estructuras de datos.
 - o Primitive Obsession
 - o Data Class
 - o Data Clump
 - o Temporary Field
- 3 Inheritance → Agrupa bad smells que permite detectar herencia dentro de las clases innecesaria.
 - o Refused Bequest
 - o Inappropriate Intimacy (subclass form)
 - o Lazy class.

- ⌘ Responsibility → Agrupa bad smells que permiten detectar un balance incorrecto en la responsabilidad entre los objetos.
 - o Feature Envy
 - o Inappropriate intimacy
 - o Message Chains
 - o Middle Man

- ⌘ Accommodating change → Agrupa bad smells que permite detectar al realizar un cambio en el código en que otros sitios el cambio tiene impacto.
 - o Diverting change
 - o Shogun surgery
 - o Parallel inheritance hierarchies
 - o Combinatorial explosion

- ⌘ Library classes → Agrupa bad smells que permiten identificar librerías de clases que por cambios que se han realizado pueden afectar a otras aplicaciones y por lo tanto generar riesgos en su funcionamiento.
 - o Incomplete library class

El aporte más importante de la propuesta taxonómica para los bad smells, consiste en proporcionar un marco de referencia para iniciar el proceso de refactoring e identificar que se desea aplicar y en que orden resolviendo, de ésta manera, la primera critica fuerte que se realiza a la propuesta de Fowler.

3.3.1 Detección de Bad Smells a partir de métricas o procesos de deducción

La crítica sobre la responsabilidad de detección de bad smells a la intuición humana, y la afirmación que no existen métricas que permitan identificar necesidades de refactoring, ha sido revaluada en trabajos posteriores.

Se han planteado mecanismos que permiten automatizar el proceso de detección de bad smells, apoyados en conceptos de métricas o procesos de deducción, logrando convertir los bad smells en una propuesta no obsoleta, que permite detectar con mayor claridad los puntos en el diseño donde se debe aplicar *refactoring*.

Las métricas proporcionan información cuantitativa, por lo que indicadores no absolutos de: extensión, cantidad, dimensión, capacidad o tamaño de los atributos de un producto o proceso, permiten evaluar la calidad de los mismos. En el proceso de desarrollo de productos de software hay dos grandes conjuntos de indicadores (Figura 11):

- ⌘ De administración de proyectos
- ⌘ Del producto de software.



Figura 11 Tipos de métricas

La clasificación de las métricas para evaluar el producto de software, es la siguiente: [PSEI88]:

- ⌘ Métricas de tamaño → Análisis estático que permite obtener información básica sobre el código.
- ⌘ Métricas de complejidad → Análisis estático que permite obtener información acerca de la complejidad del código, acoplamiento, cohesión, reutilización, entre otros.
- ⌘ Métricas de estructura → Proporciona información acerca del grado de acoplamiento y herencia del código.
- ⌘ Métricas de calidad → Permiten medir características de calidad del producto, eficiencia, portabilidad.

Otro enfoque a través del cual se puede determinar la presencia de un bad smell, es la aplicación de procesos de deducción donde a través de grafos, diagrama de estados, entre otros se puede determinar patrones de comportamiento dinámico/estático y así detectar posibles problemas localizados para un posterior proceso de refactoring.

4. REINGENIERÍA EN BASES DE DATOS RELACIONALES

4.1. Objetivos de la reingeniería de bases de datos

En general, la reingeniería de datos tiene como objeto, uno o varios de los siguientes aspectos:

- ⌘ Mejorar la accesibilidad de los datos.
- ⌘ Mejorar la comprensión, consistencia y confiabilidad de los datos.
- ⌘ Ajustar los sistemas de bases de datos para soportar cambios en las reglas de negocio, estos cambios por lo general son de sintaxis y semántica de los datos, pues éstas varían con el tiempo ya sea porque:
 - ⌘ Los datos almacenados son accedidos por un gran número de usuarios, lo que incrementa el tamaño de la base de datos, como consecuencia con el tiempo muchos datos son redundantes y pueden llegar a ser inconsistentes.
 - ⌘ Intención de adicionar campos por cambios en el negocio o inclusive cambiar el formato de los campos.
- ⌘ Soportar nuevas herramientas que se integrarán para soportar el negocio
- ⌘ Transferir los datos a un contenedor que los soporte mejor.

4.2. Comprensión de las bases de datos relacionales

La comunidad académica y de investigación, se ha dedicado a trabajar el tema de reingeniería en bases de datos, particularmente para profundizar en la comprensión a partir del análisis de su estructura y extraer el diseño y reglas de negocio que están soportando; esto dentro de las dimensiones identificadas de la reingeniería 2.1.1, se conoce como *ingeniería en reversa*, que orientado a las bases de datos, ha sido una práctica de relativamente pocos años [BATI92], [CHIA94].

Las aproximaciones trabajadas se pueden clasificar en dos áreas:

- ⌘ Comprensión Física:
Consiste en el descubrimiento del diseño a partir del diccionario de la base de datos, extensiones de la base de datos y los DLL.
- ⌘ Comprensión Conceptual:
Consiste en el descubrimiento de la base de datos a partir de la comprensión del comportamiento de los datos y la interacción de la base de datos con factores externos.

En éstas áreas se identifican técnicas reconocidas y utilizadas en el proceso de descubrimiento de la base de datos, ver Tabla 5.

Comprensión Física	Comprensión Conceptual
Descubrimiento de llaves primarias y foráneas. ²	Atributos comunes entre relaciones ³
Descubrimiento de constraints	Análisis de sentencias SQL ⁴ A través de usuarios expertos en las bases de datos.

Tabla 5 Técnicas de aproximación para el descubrimiento del diseño de bases de datos relacionales

Aunque estas técnicas apoyan el proceso de reingeniería, no es suficiente contar con una comprensión completa del diseño de la base de datos, pues esto es solo un pequeño eslabón de lo que realmente implica un proceso de reingeniería en bases de datos.

En su trabajo, [AGSD03] Agarwal, Sarangi y Das, plantean una propuesta de las diferentes etapas que se deben presentar en un proceso de reingeniería de bases de datos, cuando se requieren realizar cambios en las estructuras por cambios en las reglas de negocio, Figura 12.

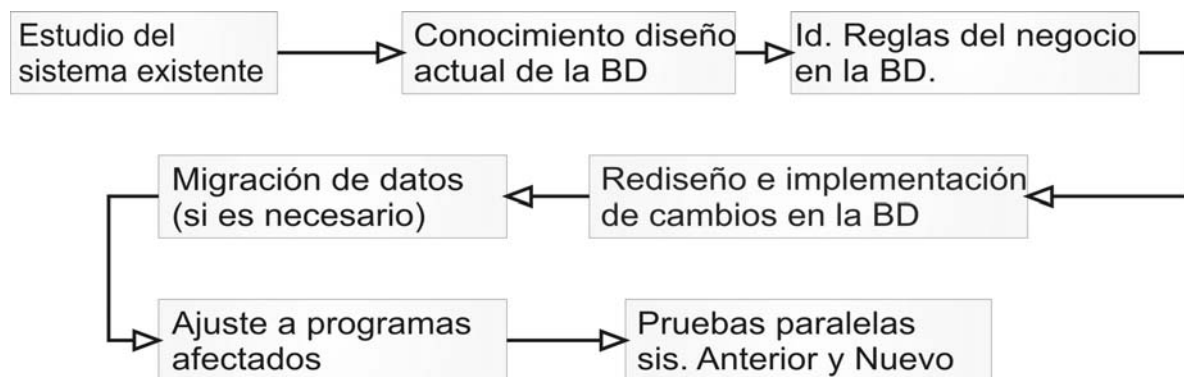


Figura 12 Proceso de reingeniería de bases de datos de Agarwal, Sarangi y Das.

En esta propuesta, la etapa de rediseño de la base de datos se trata como un paso que se debe realizar para obtener sistemas empresariales ajustados a los nuevos requerimientos de negocio y en procesos de migración, sin embargo, no se proporcionan lineamientos claros de cómo debe ser el proceso de rediseño ni las variables a tener en cuenta en el momento de plantear cambios a la estructura y comportamiento de la base de datos.

4.3. Refactoring en Bases de datos

Para reforzar los objetivos de la reingeniería en bases de datos, se debe trabajar en fortalecer aspectos dentro del proceso planteado por Agarwal, Sarangi y Das [AGSD03], y

² Para más información sobre esta técnica se puede hallar en [ANDE94]

³ Para más información sobre esta técnica se puede hallar en [HAIN93].

⁴ Para más información sobre esta técnica se puede hallar en [BARB00].

apoyar así otros objetivos como lo son: el mejoramiento de la accesibilidad de los datos, mejoramiento de comprensión, consistencia y confiabilidad de los mismos.

Ambler [AMBL02], realiza una *propuesta de refactoring* de bases de datos, apoyado en trabajos contemporáneos como los de *Fowler 3.1*; en una definición formal Ambler [AMBL02], lo describe como cambios en los esquemas que permiten mejorar el diseño mientras mantiene la semántica del comportamiento y de la información.

Su propuesta se apoya además en los mismos principios de refactoring planteados por Fowler, lo que evidencia no solo un trabajo en refactoring de aplicaciones sino también en las estructuras de almacenamiento de datos que las soportan.

4.3.1 Complejidad del refactoring en bases de datos

Según Ambler [AMBL02], el refactoring modifica el diseño del esquema de la base de datos y tiene como premisa mantener la semántica del comportamiento sin embargo, tiene como componente adicional el mantener la semántica de la información, ésta última, hace referencia a conservar la misma funcionalidad desde el punto de vista de cualquier fuente de datos, es decir que siga funcionalmente idéntica inclusive con los aspectos modificados, por lo cual se requiere un trabajo detallado en cada uno de los elementos que interactúan con la base de datos, para mantener la misma funcionalidad que antes de haber aplicado refactoring.

Para mantener el comportamiento semántico de la base de datos, se requiere escribir scripts que migren la información y mantenga, frente a cambios de la estructura, los procedimientos y el comportamiento de las aplicaciones consistente.

No solo es difícil realizar refactoring a la base de datos por los aspectos planteados anteriormente, también se hace difícil por el acoplamiento entendiéndolo como el grado de dependencia entre variables; entre más acoplamiento exista, mayor probabilidad habrá de realizar un cambio al haber afectado algún componente de la estructura.

Se pueden identificar como fuentes de acoplamiento:

- o Integración con diferentes aplicaciones → Cuando se realiza un cambio en la base de datos también se debe realizar un cambio en el código de la aplicación integrada. El peor escenario se puede identificar si existe una amplia variedad de aplicaciones acopladas a una misma base de datos.
- o Carga de datos hacia varias fuentes → Si la base de datos es la encargada de enviar información a diferentes fuentes, es necesario realizar una revisión del mecanismo de carga.
- o Extracción de datos de varias fuentes → Si la base de datos toma información de diferentes fuentes, con seguridad se deben revisar los mecanismos de extracción y carga.

- o Frameworks o capas de persistencia → Frameworks que encapsulan la lógica y mapean la información de las clases a la base de datos para hacerla persistente.
- o Dentro de la base de datos → Con elementos como: procedimientos almacenados, tablas que utilizan columnas como llaves foráneas y vistas que hacen referencia a una o más tablas, se debe tener cuidado con pequeños cambios que impacten también otros elementos a través de la base de datos.

Otras fuentes dependientes de la estructura de la base de datos son: scripts de migración de datos, código de pruebas, documentación, formato de los datos, interfaces, administración de recursos/adopción de las nuevas tecnologías, manejo de errores.

4.3.2 Aplicación de refactoring en bases de datos relacionales

En “*Refactoring Databases: Evolutionary Database Design*”, trabajo propuesto por Ambler en el 2002, plantea la forma como se debe manejar el proceso de refactoring aplicado a las bases de datos. Su propuesta se fundamenta en tres actividades principales cada una con tareas relacionadas.

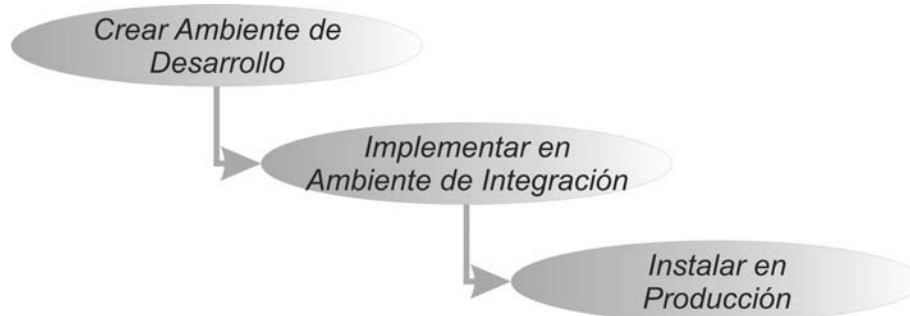


Figura 13 Proceso de refactoring en bases de datos planteado por Ambler

Como se observa en la Figura 13, las tres actividades se deben ejecutar una seguida de otra sin trabajo paralelo, puesto que los resultados de la actividad anterior genera las entradas estables para la siguiente actividad.

1. Verificar que el proceso de refactoring realmente se requiere.
2. Seleccionar el refactoring más apropiado para la base de datos.
3. Escribir las pruebas de unidad que posteriormente permitan garantizar la consistencia de la información ante los cambios.
 - ✍ Las pruebas deben ser a nivel de base de datos o de aplicación.
 - ✍ Búsqueda de herramientas que permitan asegurar la ejecución de las pruebas.
 - ✍ Crear un ambiente con datos de prueba.
4. Sobre el esquema original establecer un periodo de transición

** se mantiene un espacio compartido de la nueva estructura y la estructura anterior. De tal forma que se trabaje con el esquema nuevo y antiguo de manera paralela, es importante en el periodo de transición establecer estrategias que permitan mantener los datos entre los dos esquemas sincronizados.
5. Modificar el esquema de bd.

Se deben aplicar los refactoring en el ambiente de desarrollo tanto para la base de datos como para la aplicación. Se debe arrancar por refactorign simples, y mantener dos registros o bitácoras actualizadas así:

 - ✍ Log de los cambios en la base de datos: Mantener un registro histórico de los cambios que son realizados sobre la base de datos. Cuando se implementan refactoring, se debe incluir en el log y si es posible incluir el DDL y estructuras de sincronización implementadas en la base de datos.
 - ✍ Log de actualizaciones: mantener el código que deberá ser alterado luego del periodo de sincronización para dejar la base de datos definitiva luego de haber aplicado los refactorings.
6. Migrar los datos, Al aplicar refactoring es posible que se requiera realizar una migración de datos cuidadosa de la antigua versión al nuevo esquema.
7. Actualizar las aplicaciones y programas que acceden a la bd
8. Ejecutar las pruebas de regresión

Una vez se han aplicado los refactoring en la base de datos y las aplicaciones han sido ajustadas para soportar el cambio, se debe ejecutar el plan de pruebas de regresión realizado en el punto dos.
9. Realizar el control de versiones apropiado.

Figura 14 Descripción de las tareas asociadas al proceso de refactoring en Bases de Datos (1)
En la Figura 14, Figura 15 y Figura 16, respectivamente se observa en su orden las tareas asociadas a cada una de las actividades necesarias para realizar un proceso de refactoring apropiado, para mayor información se puede acudir a [AMBL02].

1. Una vez se han implementado los refactoring en bases de datos se debe mantener el ambiente de integración **donde todas las aplicaciones son modificadas, probadas de acuerdo al nuevo esquema**. El objetivo de este paso es validar el trabajo conjunto tanto a nivel de base de datos como a nivel de los aplicativos.
2. Se debe realizar un proceso de aseguramiento de calidad.

Figura 15 Descripción de las tareas asociadas al proceso de refactoring en Bases de Datos (2)



Figura 16 Descripción de las tareas asociadas al proceso de refactoring en Bases de Datos (3)

En la Figura 17 se representa la propuesta de Ambler [AMBL02], quien plantea refactorings para bases de datos clasificándolos por categorías relacionadas. Se escogió vincular esta categorización a la investigación pues provee una guía para apoyar posteriormente la propuesta taxonómica de los bad smells para las bases de datos relacionales.

Cabe resaltar que la propuesta tiene en cuenta refactorings para bases de datos enfocadas a aplicaciones orientadas por objetos, sin embargo, el valor agregado que puede proporcionar ésta propuesta a la investigación continúa siendo válido.

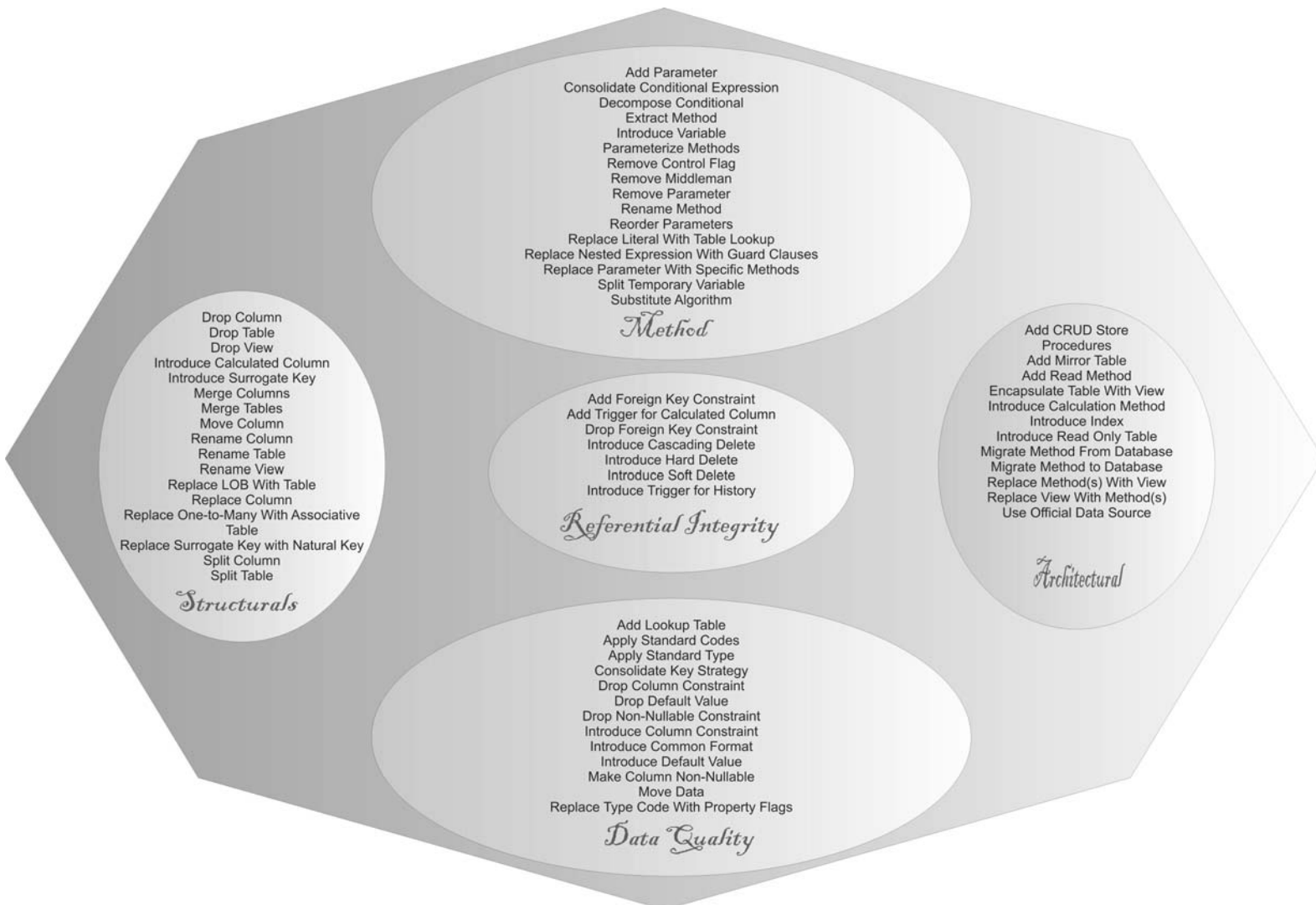


Figura 17 Propuesta y Clasificación de refactorings para bases de datos de Ambler

4.4. Fundamentos de diseño de bases de datos relacionales

Alrededor del diseño de bases de datos relacionales (BDR) se ha realizado gran cantidad de investigaciones y avances tecnológicos, por lo que se puede encontrar una amplia variedad de bibliografía, propuestas y marcos de referencia acerca de los fundamentos necesarios para realizar un adecuado diseño de BDR.

Para proponer bad smells con bases adecuadas y organizar una posterior taxonomía es necesario identificar y apropiar los fundamentos en el diseño de BDR; a continuación se realiza una breve descripción.

4.4.1 Pautas para el diseño de esquemas ER en BDR

Al evaluar si un diseño de bases de datos es “bueno”, se puede realizar el análisis con base en dos niveles:

- ✂ Nivel lógico: O conceptual que se refiere a cómo los usuarios interpretan las relaciones y el significado de los atributos.
- ✂ Nivel de implementación: O de almacenamiento que se refiere a la forma como se realizan las actualizaciones de registros entre las relaciones y la forma en que se almacena la información.

Sea cual sea el análisis que se realice, es importante tener en cuenta ciertas pautas cuando se diseña o evalúa un esquema de relaciones de una base de datos, conceptualmente las pautas son⁵ (Tabla 6):

Pauta	Descripción.
<i>Semántica de los atributos</i>	<i>Los atributos deben tener cierto significado a través del diseño del modelo ER, se debe especificar claramente cómo se deben interpretar los valores de los atributos almacenados, de tal forma que permita reflejar la relación entre los valores de una tupla.</i> <i>Se debe evitar combinar atributos de varios tipos de entidades en una misma relación.</i>
<i>Información redundante</i>	<i>Uno de los objetivos en el diseño de bases de datos es minimizar el espacio de almacenamiento que ocupan las relaciones en la base de datos.</i> <i>Se debe evitar almacenar información redundante a través de las relaciones de las bases de datos, en caso que sea necesario violar estas pautas para mejorar el rendimiento, se debe documentar ésta decisión y las implicaciones que se tienen al momento de realizar actividades de CRUD sobre la información a través de las aplicaciones que operarán los</i>

⁵ Se realiza una descripción conceptual puesto que en la sección 4.4.2 se presentan los fundamentos formales de éstas pautas.

	datos. Frente a la decisión de violar esta pauta se debe asegurar al máximo a través de restricciones y verificaciones en la base de datos que la información va a mantener la integridad a través del tiempo. Adicionalmente se deben tener en cuenta implicaciones como: poca facilidad y el alto costo, de una posterior actualización de la información.
Anomalías de actualización	Estrechamente ligada con la pauta anterior, se deben diseñar esquemas donde no existan anomalías de actualización, inserción y eliminación. En acciones de inserción de datos, asegurarse que la información que se requiere sea registrada completamente y evitar utilizar valores nulos como acción temporal mientras que se tienen datos completos. En acciones de eliminación, evitar que se pierda información relacionada por borrado de un último registro que mantiene la información de interés de manera temporal. En acciones de modificación, evitar ante actualización de un dato tener que actualizar masivamente la misma información relacionada en diferentes entidades. Donde se determine la necesidad de violar ésta pauta, es necesario evaluar si una vista es una opción válida para presentar al usuario la información y evitar que el usuario realice operaciones que posiblemente puede provocar que la base de datos quede inconsistente.
Valores nulos en las tuplas	Hasta donde sea posible se debe evitar definir relaciones cuyos atributos pueden ser nulos. Si no es posible evitar los valores nulos, asegúrese que solo se apliquen en casos excepcionales.
Tuplas espurias	Las tuplas espurias, son tuplas adicionales resultantes de una operación de reunión natural, que originalmente no existen en los datos registrados en la base de datos. Las relaciones deben diseñarse de tal forma que puedan reunirse mediante condiciones de igualdad sobre atributos que sean llave primaria o foránea, y no genere este tipo de tuplas. No se deben incluir relaciones que contengan atributos coincidentes que no estén definidos como llaves primarias o foráneas.
Imposibilidad de representación de cierta información	Se evidencia al identificar puntos donde información, que por su naturaleza es independiente de los datos que se manipulan a través del tiempo, se encuentra vinculada con valores nulos (por la falta de datos relacionados) o solo aparece en las relaciones cuando se debe vincular datos a través del tiempo. Lo que provocaría, en determinadas situaciones, eliminar información fundamental por falta de relación con información

	<i>detallada o asociarla con valores nulos para mantenerla a través del tiempo.</i>
--	---

Tabla 6 Pautas para validar la calidad de un diseño de BDR.

4.4.2 Dependencia funcional y formas normales

El diseño de BDR se fundamenta en la teoría de conjuntos, [SILB02] por lo que temas como redundancia de la información, entre otros, se valida a través de verificaciones de las dependencias funcionales⁶ de la base de datos y posteriormente de las formas normales.

La utilidad principal de las dependencias funcionales, es describir un esquema de relaciones de los atributos de la base de datos, mediante la especificación de restricciones que deben cumplirse sobre los atributos en todo momento. Éstas no pueden definirse automáticamente, aunque se pueden inferir, se debe partir de un conjunto de dependencias base especificadas por alguien que conozca de los atributos de la base de datos, sus relaciones, semántica y dependencias.

La importancia de identificar la totalidad de las dependencias funcionales que se encuentran implícitas o explícitamente definidas en las bases de datos, radica en que a partir de éstas dependencias se pueden validar decisiones de diseño e identificar las pautas mencionadas en la Tabla 6.

Las reglas de inferencia para las dependencias funcionales implícitas, se establecen a través de axiomas, conocidos como axiomas de Armstrong [SILB02]^{7 8}:

- | | |
|---|--|
| 1. Regla de reflexiva | Si $X \supseteq Y$, entonces $X \rightarrow Y$. |
| 2. Regla de aumento | $\{X \rightarrow Y\} \models XZ \rightarrow YZ$. |
| 3. Regla transitiva | $\{X \rightarrow Y, Y \rightarrow Z\} \models X \rightarrow Z$. |
| 4. Regla de descomposición o proyectiva | $\{X \rightarrow YZ\} \models X \rightarrow Y$ |
| 5. Regla de unión o aditiva | $\{X \rightarrow Y, X \rightarrow Z\} \models X \rightarrow YZ$. |
| 6. Regla pseudotransitiva | $\{X \rightarrow Y, WY \rightarrow Z\} \models WX \rightarrow Z$. |

4.4.2.1 Proceso de normalización

Al conocer las dependencias funcionales totales, cierre F+, sobre un conjunto de relaciones (modelo ER) y al suponer que cada relación en el modelo cuenta con una llave primaria, ésta información y la combinación con las pruebas necesarias permiten establecer en qué forma normal se encuentra una relación, lo que conduce al proceso de normalización.

⁶ Restricción entre conjunto de atributos de una base de datos

⁷ Para ampliar la información se puede acudir a [SILB02]

⁸ Para estudiar la demostración de las reglas

La normalización se entiende entonces como el proceso de aplicar una serie de pruebas para validar si un esquema se encuentra en cierta forma normal, es un proceso descendente donde se descomponen relaciones hasta tanto no cumplan la forma normal deseada por el diseñador. Éstas se fundamentan en las dependencias funcionales, por lo que un factor importante a tener presente es que las formas normales vistas de manera aislada no garantizan un buen diseño de datos, adicional a esto se debe asegurar que durante el proceso de normalización las relaciones mantienen un conjunto de propiedades adicionales como:

- ⌘ La propiedad de reunión sin pérdida o reunión no aditiva.
Garantiza que no se generen tuplas espurias cuando se aplique una operación de reunión natural a las relaciones de la descomposición.
- ⌘ La propiedad de la conservación de las dependencias
Asegura que todas las dependencias funcionales estén representadas en algunas de las relaciones individuales resultantes del proceso de descomposición.

La forma normal en que un diseñador define el modelo e implementación de la base de datos no necesariamente debe ir hasta un alto nivel, por razones de rendimiento se puede decidir dejar las relaciones en formas normales inferiores, la industria sin embargo, considera que un buen proceso de normalización puede variar entre 3FN y FNBC.

A continuación se menciona brevemente la definición de cada una de las formas normales y las pruebas que se deben realizar para verificar la forma normal de una relación:

Nombre	Primera Forma Normal (1FN)
Descripción	⌘ Establece que el dominio de un atributo debe incluir valores atómicos (simples e indivisibles). ⌘ Se prohíben las relaciones dentro de relaciones o las relaciones como tuplas.
Pruebas	Validar que no existan atributos multivaluados en las relaciones.
Técnicas para normalizar en 1FN	⌘ Eliminar el atributo que viola el principio de valores atómicos de la relación y colocarlo en una relación aparte. (abrir la relación en dos o más relaciones) ⌘ Ampliar la llave primaria de tal forma que exista una tupla distinta por cada uno de los valores que están asociados al atributo que viola el principio de valores atómicos. Desventaja: redundancia de tuplas. ⌘ Si se conoce el número máximo de los valores para el atributo, reemplazar el atributo que viola el principio de valores atómicos por n-atributos atómicos. Desventaja: Introducir valores nulos a las relaciones.

Nombre	Segunda Forma Normal (2FN)
Descripción	Se basa en el concepto de dependencia funcional total, lo que restringe a que todos los atributos que no hacen parte de la llave primaria dependan por completo de la llave primaria.
Pruebas	Para relaciones donde la llave primaria está compuesta de múltiples atributos, determinar si hay dependencias funcionales donde atributos que no hacen parte de la llave primaria

	<i>dependan de forma parcial de ella.</i>
Técnicas para normalizar en 2FN	⌘ <i>Descomponer y crear una nueva relación para cada atributo parcial que determina la dependencia en la llave primaria con su atributo o atributos dependientes.</i> ⌘ <i>Se debe asegurar que se mantiene la relación con la llave primaria original y todos los atributos dependen funcionalmente de manera total de la llave primaria</i>

Nombre	<i>Tercera Forma Normal (3FN)</i>
Descripción	<i>Se basa en el concepto de dependencia transitiva, donde un conjunto de atributos no pueden depender transitivamente de la llave primaria, en otras palabras, no se deben detectar atributos que a través de algún otro atributo se determine su valor.</i>
Pruebas	<i>Una relación no deberá tener un atributo que no se determine a través de la llave primaria. Se debe validar que no existan dependencias transitivas por parte de un atributo no clave.</i>
Técnicas para normalizar en 3FN	<i>Los atributos que determinan su valor transitivamente dentro de una relación deben abrirse a una nueva relación y asegurarse que por reunión natural la información se mantenga consistente</i>

Nombre	<i>Forma Normal Boyce-Cood (FNBC)</i>
Descripción	<i>La definición original de 3NF de Cood, tiene ciertas deficiencias al manejar relaciones en las cuales:</i> ⌘ <i>Hay varias llaves foráneas.</i> ⌘ <i>Las llaves foráneas son compuestas.</i> ⌘ <i>Las llaves candidatas se traslapan (tienen atributos en común).</i> <i>Posteriormente Boyce y Cood definieron la forma normal FNBC, para solucionar las deficiencias de la 3FN.</i>
Pruebas	<i>Se busca los atributos determinantes que no se comportan como una llave candidata de la relación y se separan éstos atributos en una nueva relación.</i>
Técnicas para normalizar en FNBC	<i>Determinar los atributos de las relaciones que no se comportan como una llave candidata, de ésta forma los atributos deben de nuevo abrirse en una nueva relación, esto con el objeto de evitar redundancia de datos.</i>

Las formas normales 4FN y 5FN no se presentan, puesto que como fundamento de éste trabajo de investigación, se realizará el análisis hasta la 3NF y FNBC, al ser éstas formas las consideradas por la industria como adecuadas.

4.4.3 Patrones en BDR

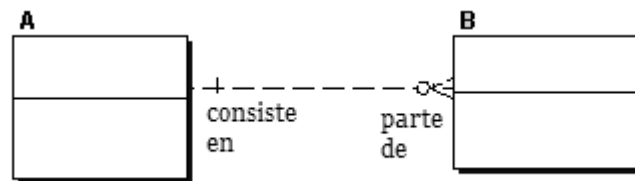
Los patrones son modelos de estructuras que independientemente del contexto, se repiten con frecuencia y permiten establecer similitudes en la estructura de los diseños a través del tiempo, como consecuencia permiten mejorar los tiempos y la calidad de los diseños que se elaboran evitando caer en los mismos errores repetidamente, actualmente la principal preocupación no es cómo realizar el diseño, sino encontrar soluciones a situaciones verdaderamente complejas de un caso particular.

A continuación se presenta una relación de los patrones que se han identificado para modelos de relaciones de BDR.

⌘ Patrón Maestro – Detalle

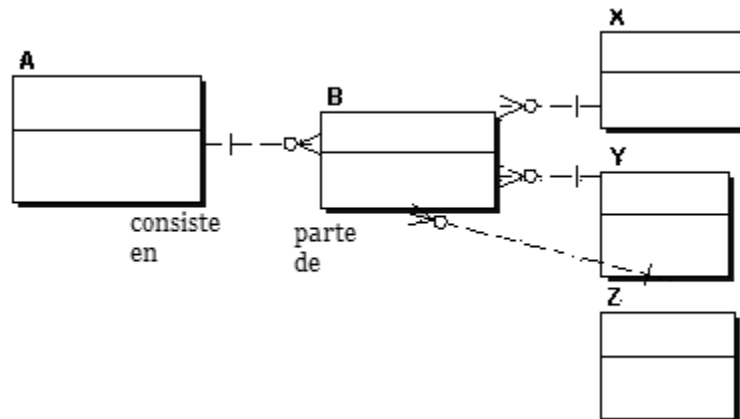
El más común de los patrones, se utiliza para establecer relaciones 1:m, característico por el hecho que el maestro A es detallado en entidades B, y ésta última no existe sola, siempre se entienden en el contexto de su relación maestra A. Típicamente éstas relaciones responden a aserciones como:

- o Consiste de...
- o Dividida dentro....
- o Hecha de...



⌘ Basket

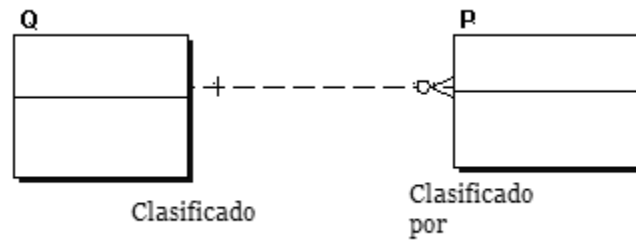
Caso especial de maestro – detalle, una relación basket puede contener una o más cosas, pero estas cosas pueden ser de diferentes tipos. Cada ítem que compone la relación puede ser solamente de un tipo.



⌘ Clasificación

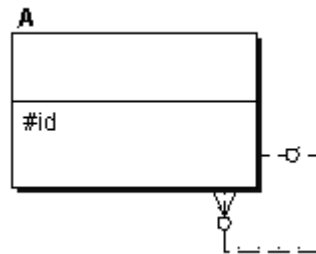
De nuevo es un patrón que expresa relaciones 1:m, pero su orientación es inversa de P a Q. Típico caso cuando Q puede existir independientemente de P. Q actúa como un tipo o clase particular de P y P establece agrupaciones. Estas relaciones responden a aserciones como:

- o Clasificado por...
- o Agrupado por...
- o Asignado a...



3 Jerarquía

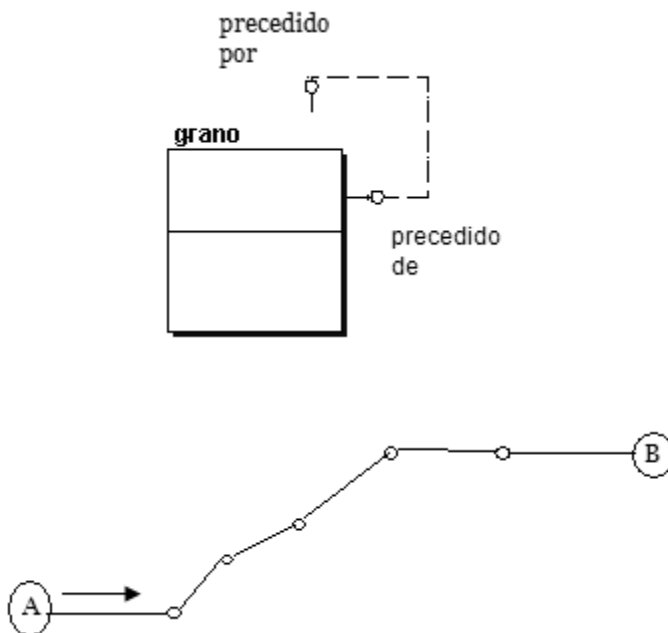
Las jerarquías normalmente tienen un límite de niveles, cuando el número de niveles es conocido y pequeño usualmente se utiliza el éste modelo. Sin embargo, se pueden utilizar jerarquías recursivas donde no se conoce el número de niveles a manejar o el número de niveles conocido, es alto.

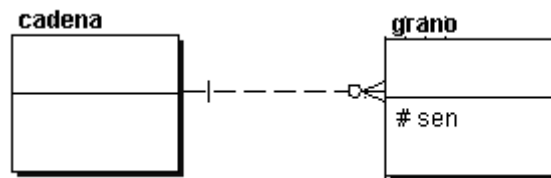


3 Cadena

Es una forma especial de jerarquía, una cadena es una relación recursiva de una entidad, las relaciones de este patrón es de 1:1 caracterizada por el hecho de que un objeto en la cadena es precedido del siguiente y así sucesivamente.

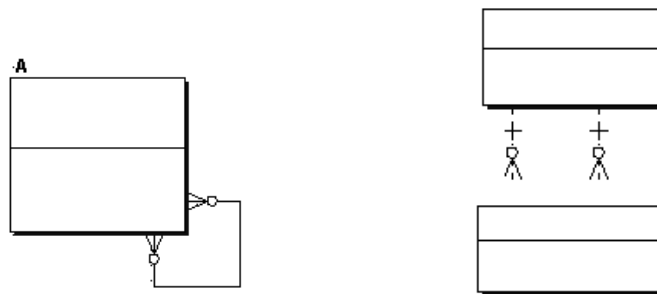
Es una estructura donde la secuencia es importante, permite modelar relaciones maestro detalle, el modelo recursivo permite insertar fácilmente relaciones en una cadena.



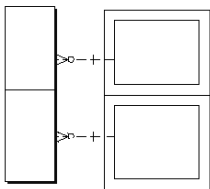


3 Red

Típicamente describe pares de cosas del mismo tipo. Como características de este patrón, las relaciones m:m deben ser resueltas a través de información específica acerca del par de información relacionado, dos relaciones tienen como intersección un identificador único, constraints relacionados aplican en la red como guardas.



Como ejemplo, dos relaciones hacen referencia a diferentes subtipos de una entidad.

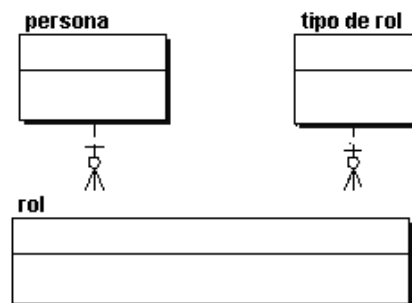
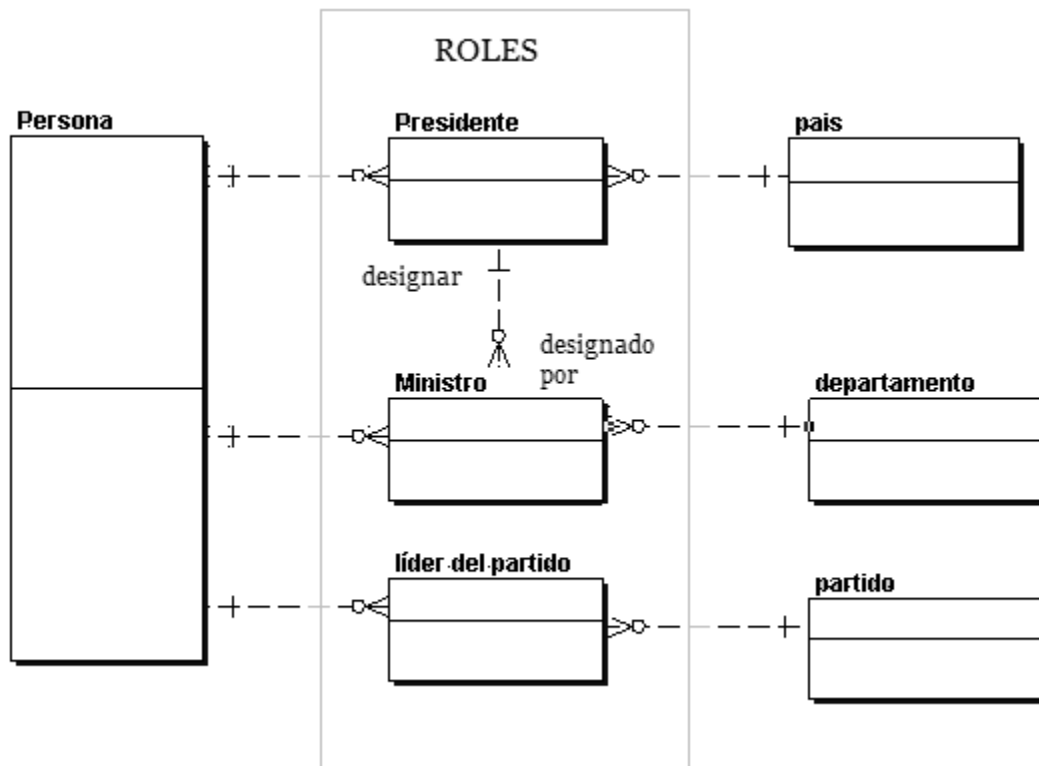
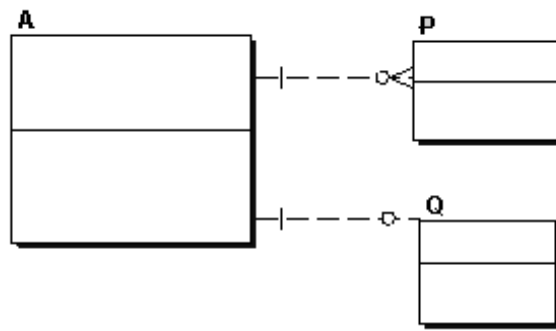


Patrones Simétricos

Las relaciones recursivas y simétricas son un tipo especial de problema el cual es más complejo de lo que se podría suponer. Son relaciones que se pueden leer en ambas direcciones y no pierden su significado por ejemplo “la persona de la familia o la familia de la persona”, los patrones asociados son:

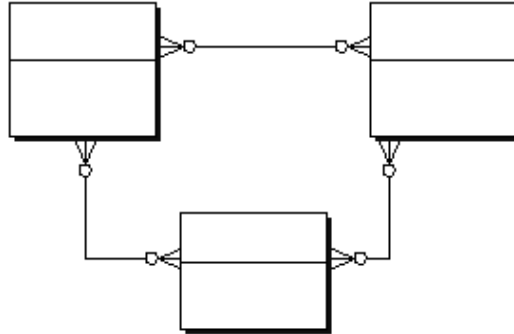
3 Roles

Se presentan cuando se requiere conocer más información de la básica asociada a una cosa, la información adicional se modela a través de entidades separadas que ofrecen la posibilidad de mostrar cuales atributos son mandatorios para un rol o atributo en particular. Y si es necesario presentar relaciones entre varios roles.



3 Fan Trap

Se presenta cuando tres o mas entidades están relacionadas a través de relaciones m:m en forma de campana, usualmente se reemplazan las relaciones m:m con una entidad central para dividir las relaciones y tener relaciones m:1



5. PROPUESTA BAJO LA CUAL SERÁN DEFINIDOS BAD SMELLS PARA BASES DE DATOS RELACIONALES.

Como se ha observado en los capítulos 3 y 4, el proceso de reingeniería utiliza el refactoring para mejorar la estructura del código existente (ver Tabla 4), adicional a esto la actividad se apoya en la propuesta de bad smells de Fowler para la identificación de puntos donde se debe aplicar el proceso, su propuesta, gracias a investigaciones, [BFLO05] [CANG05] se ha optimizado a través de métricas y estrategias para automatizar su detección, todo esto en POO. La visualización de ésta explicación se puede observar en la Tabla 7.

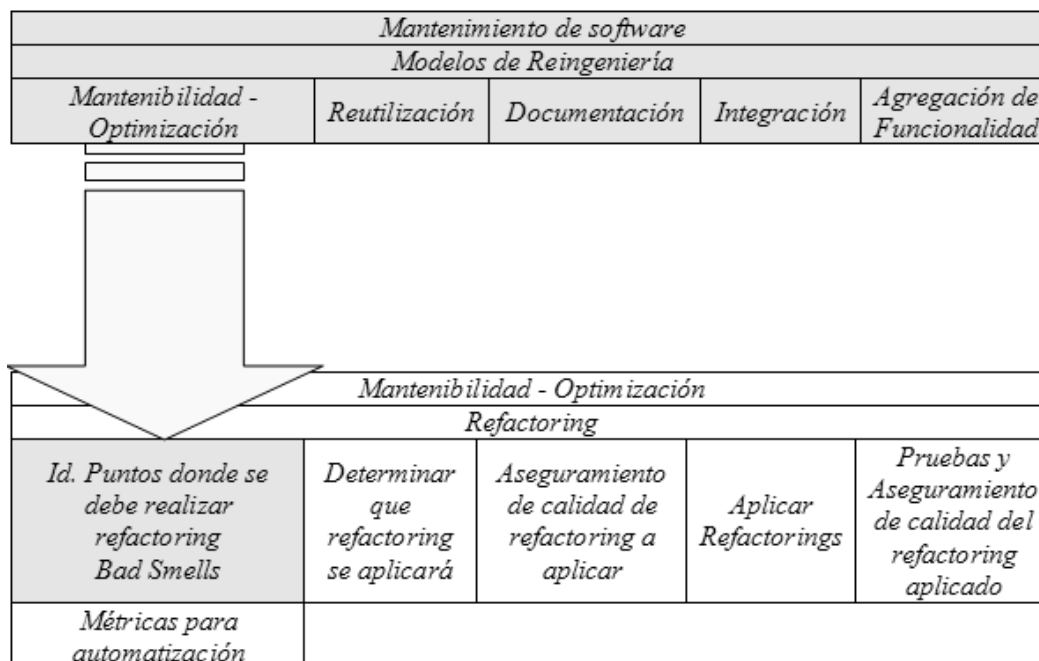


Tabla 7 Desarrollo en reingeniería para comprensión, mantenibilidad y optimización en POO.

El tema de bases de datos relacionales se ha trabajado en los modelos de reingeniería (ver 4.2), los refactorings a aplicar y su calidad en la implementación (ver 4.3), como se observa en la Tabla 8. Sin embargo, aún no se ha explorado el tema de bad smells.

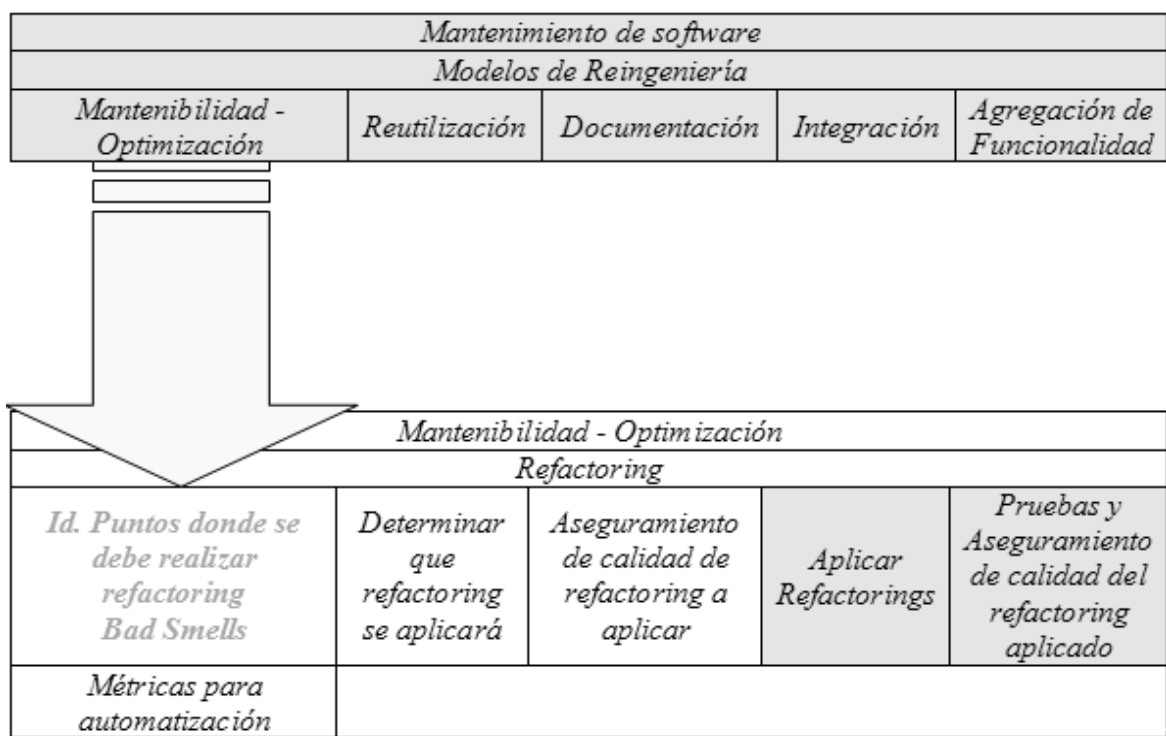


Tabla 8 Desarrollo en reingeniería para comprensión, mantenibilidad y optimización en BD Relacionales.

El objeto de ésta investigación es realizar un estudio del problema de detección, primer paso en el proceso de refactoring, enfocado en el planteamiento de bad smells para BDR y posteriormente la definición de la taxonomía por categorías relacionadas de los mismos.

5.1. Proceso de definición de Bad Smells para BDR

Los bad smells identificados y definidos en éste trabajo, se establecen con base en la investigación realizada acerca de los fundamentos de bases de datos (proceso de normalización, formas normales), la comprensión de las pautas para el diseño de esquemas relacionales y los patrones de BDR, descritos en la sección 4.4.

Con el objeto de mantener uniformidad a través de la propuesta, para la definición de los bad smells, permitir una fácil comprensión y que la propuesta se preste como base para futuras mejoras, a continuación se presenta la plantilla (Figura 18) que será aplicada para la definición de cada uno de los bad smells.

Nombre, Busca caracterizar brevemente el bad smell propuesto con el objeto de permitir una fácil identificación de su objetivo.	
Descripción	Breve descripción del bad smell
Síntomas	Descripción conceptual básica, de la forma como el bad smell puede ser descubierto.
Qué hacer	Descripción conceptual detallada, de la forma como el bad smell puede ser identificado. Describe posibles acciones que pueden ser tenidas en cuenta para mejorar el bad smell una vez identificado.
Ventajas de su corrección	Describe las mejoras que la corrección de éste bad smell implica para el desempeño y/o para el esquema relaciones de la BDR.
Notas, espacio que permite describir comentarios relevantes acerca del bad smell, como que elementos del diseño adicionales son necesarios, descripción de situaciones (si las hay) donde el bad smell no debe ser arreglado, entre otros.	

Figura 18 Plantilla base para la definición de los bad smells

Los bad smells se definen, partiendo del hecho que en la industria y academia un nivel de normalización óptimo es la tercera forma normal, para modelos relacionales. Para bad smells puntales se expresará gráficamente su representación, con el objeto de apoyar su explicación

5.2. Bad Smells propuestos para BDR

1. Tabla Gruesa	
Descripción	Apoya en la detección de malos diseños, donde la tabla cuenta con un gran número de atributos, por lo que es probable que exista un alto acoplamiento de la información o indicar que se encuentra mal normalizada.
Síntomas	Tabla que tiene un alto número de atributos.
Qué hacer	☞ Sobre los atributos no clave, identificar aquellos que no dependen totalmente de la llave primaria, en caso que dependan parcialmente, los atributos deben ser desvinculados. Los atributos identificados pueden provocar anomalías de actualización, se debe validar si ésta situación se presenta lo que indica fallas en el proceso de normalización en 2FN, se deben desvincular de la tabla a través de un proceso válido de normalización.
Posibles problemas	☞ Anomalías de actualización, por causa de un inadecuado diseño. ☞ Mantenibilidad, de la información en el tiempo.
Ventajas de su corrección	☞ Se elimina anomalías de actualización, si existen. ☞ Reduce el tamaño de las tablas. ☞ La mantenibilidad de la información en el tiempo es más eficiente.
Notas: ☞ Si la tabla se encuentra normalizada adecuadamente, es posible que tenga una gran responsabilidad sobre la información altamente relacionada, se debe continuar entonces con la validación del bad smell Tabla con alta responsabilidad. ☞ Si se detectan anomalías de actualización y se busca retirar atributos a través de un proceso válido de normalización, es deseable tener presente las dependencias funcionales si las hay y validarlas con el nuevo esquema propuesto de normalización.	

2. Tabla Larga	
Descripción	<i>Apoya en la detección de malos diseños donde la tabla tiene almacenado un alto número de registros.</i>
Síntomas	<i>Tabla con un gran alto de registros.</i>
Qué hacer	☞ <i>Se debe determinar si la tabla es conceptualmente o no, una tabla base⁹, si la tabla es base: Detectar si hay redundancia no justificada de la información o atributos no atómicos, lo que indica un inadecuado proceso de normalización con base en 1FN. Se debe proceder a identificar cuales son los atributos que están generando redundancia o no son atómicos y desvincularlos de la tabla a través de un proceso de normalización asegurando 1FN y 2FN.</i> <i>Se debe tener presente las relaciones que utilice la tabla original para establecer y actualizar las relaciones con las nuevas tablas.</i>
Posibles problemas	☞ <i>Anomalías de actualización, por causa de un inadecuado diseño.</i> ☞ <i>Redundancia de información.</i> ☞ <i>Tuplas espurias.</i>
Ventajas de su corrección	☞ <i>Se elimina anomalías de actualización, si existen.</i> ☞ <i>Elimina redundancia de la información.</i> ☞ <i>Optimiza el rendimiento en el acceso a la información donde la tabla se requiere para consulta de información.</i>
Notas: ☞ <i>Si la tabla se encuentra normalizada adecuadamente y no se detectan los problemas relacionados, se debe encontrar documentado e identificado el rango posible de valores para los atributos, para así controlar el crecimiento de la tabla.</i>	

⁹ Las tablas o entidades bases también se conocen como fuertes o maestras, se considera que su información es punto de referencia para las transacciones de la BDR.

3. Tabla con alta responsabilidad	
Descripción	<i>Apoya en la detección de tablas que tienen relacionado un gran número de atributos y posiblemente responsabilidades diferentes, lo que indica que la tabla tiene una gran responsabilidad en cuanto a la información que almacena. Y no se encuentra enfocada en el manejo de un solo tema y sus relaciones.</i>
Síntomas	<i>Se ha verificado el bad smell Tabla Gruesa, y existe aún un gran número de atributos.</i>
Qué hacer	⌘ <i>Se deben realizar validaciones sobre los atributos para detectar si hacen referencia a diferentes responsabilidades, que aunque están relacionadas no deben ser administradas a través de una única tabla.</i> <i>Se deben agrupar por responsabilidades relacionadas los atributos y determinar cuáles deben ser organizados en tablas diferentes a través de un proceso válido de normalización.</i> ⌘ <i>Detectar si existen atributos que aunque dependen de forma total de la llave primaria, a través de otro atributo no clave se puede llegar a él.</i> <i>Esto indica fallas en el proceso de normalización en 3NF, por lo que se deben tomar estos atributos detectados, vincularlos en una tabla diferente y relacionarlos según corresponda.</i>
Posibles problemas	⌘ <i>Alta probabilidad de generar inconvenientes de desempeño (ejecución de consultas o transacciones) en las aplicaciones con las cuales tiene relación la BDR.</i>
Ventajas de su corrección	⌘ <i>Optimiza el rendimiento en el acceso a la información.</i> ⌘ <i>Mantenibilidad de la información a través del tiempo.</i>
Notas: ⌘ <i>Si es requerido desvincular atributos de la tabla por la diferencia de responsabilidades es importante tener presente las dependencias funcionales, si las hay, y validarlas con el nuevo esquema propuesto de normalización.</i> ⌘ <i>En caso que el análisis sobre atributos identifique un diseño donde los atributos hacen referencia a diferentes responsabilidades sobre la información, y se deben mantener juntos por condiciones de rendimiento, ésta decisión se debe encontrar documentada.</i>	

4. Tablas con alto acoplamiento	
Descripción	Apoya en la detección de malos diseños donde hay tablas que se encuentran altamente acopladas
Síntomas	Tabla que sus atributos dependen o hacen referencia a otras tablas y existe al menos una tabla haciendo referencia a tuplas de la tabla que se está analizando.
Qué hacer	Se debe determinar si: ⌘ La tabla debe ser conceptualmente base ⌘ Se establece que la relación que está haciendo referencia a ella es una relación 1:n. ⌘ Al menos existe un atributo de la tabla que hace referencia a otra tabla. Esto indica la presencia de anomalías de actualización, es decir que se presentaron fallas en el proceso de normalización en 2NF, puesto que si, se actualiza una referencia de un registro y existen varias referencias a éste, pueden presentarse inconsistencias en la información a través del tiempo. Se deben tomar estos atributos detectados, vincularlos en una tabla diferente y relacionarlos según corresponda.
Posibles problemas	⌘ Anomalías de actualización. ⌘ Alto acoplamiento de la información.
Ventajas de su corrección	⌘ Se elimina la posibilidad que con el tiempo se genere inconsistencia o pérdida de información por la actualización no adecuada de información.
Notas: ⌘ Al desvincular atributos de la tabla, es importante tener presente las dependencias funcionales, si las hay, y validarlas con el nuevo esquema propuesto de normalización.	

5. Ausencia de comentarios	
Descripción	Apoya en la detección de estructuras de la base de datos no documentadas.
Síntomas	No se cuenta con comentarios asociados a las tablas, columnas, vistas y estructuras PL/SQL.
Qué hacer	Se debe determinar si las tablas, las vistas y sus columnas no cuentan con comentarios asociados. Para estructuras PL/SQL la documentación se encuentra dentro del código, por lo cual se debe determinar si existen en él estructuras como: <i>/* texto */</i> → Que documenta más de una línea <i>--</i> → documenta una línea dentro del código Los comentarios son particularmente útiles para: ☞ Cuando el autor del código identifica fragmentos que no son claros, esto permitirá aumentar la mantenibilidad del código cuando una persona diferente estudie lo estudie. ☞ Describir porque fragmentos específicos del código están escritos de una forma particular. ☞ Describir algoritmos que no son obvios. Se deben incluir progresivamente comentarios a estructuras de datos o código PL/SQL que se estudie y/o afine.
Posibles problemas	☞ Cuando se realizan mantenimientos o mejoras a la base de datos, es posible que la comprensión de funcionalidades sea en el mejor de los casos un proceso lento, o erróneo. Lo que no permitirá una viabilidad rápida de este tipo de actividades sobre la base de datos.
Ventajas de su corrección	☞ Permite una ágil comprensión de la base de datos y de la razón de ser de las estructuras de datos o código PL/SQL. Da viabilidad a actividades de mantenimiento o mejoras de la base de datos.
Notas: ☞ Los comentarios que se incluyen deben ser claros y concretos, y tener presente que deben ser incluidos una vez se han realizado análisis cuidadosos de las estructuras que conforman la base de datos.	

6. Estructuras de datos internas duplicadas	
Descripción	Apoya en la detección de estructuras de datos duplicadas dentro de una misma tabla.
Síntomas	Tabla que cuenta con atributos que en su estructura de definición son idénticos (mismo tipo y tamaño) y entre ellos son opcionales.
Qué hacer	Se debe determinar si: ⌘ Dentro de la tabla existen dos o más atributos que son idénticos en su definición de tipo y tamaño. ⌘ Si los atributos antes identificados entre ellos opcionales. ⌘ Si los atributos conceptualmente están relacionados y son excluyentes. Las estructuras de datos duplicadas tienen dos comportamientos: 1. Pueden ser utilizadas para tipificar estados conocidos, es decir, se asocia cada estado conocido a un atributo y en el momento de registrar una tupla solo se relaciona información (por lo general booleana o de un dominio muy pequeño conocido) a un atributo de los definidos, es decir que sus valores son excluyentes entre sí para una misma tupla. En este caso se debe identificar los estados asociados a estos atributos, relacionarlos en una tabla base, eliminar los atributos identificados de la tabla, crear o mantener uno de los atributos para relacionar valores, definirlo como obligatorio y crear una relación de dependencia del atributo con la tabla que ha sido creada. 2. Pueden ser utilizados para relacionar valores de tipos conocidos, es decir, se asocia cada tipo conocido a un atributo y en el momento de registrar una tupla solo se relaciona valor a uno de los atributos definidos (por lo general éste valor no es conocido, ni se puede establecer un dominio), los atributos son excluyentes entre sí. En general, el valor de estos atributos no está relacionado con otras tablas. En este caso se debe identificar los tipos asociados a estos atributos, relacionarlos en una tabla base, eliminar los atributos identificados de la tabla, crear o mantener uno de los atributos para relacionar valores, definirlo como obligatorio, crear una relación de dependencia del atributo con la tabla que ha sido creada y crear otro atributo para relacionar los valores asociados al tipo relacionado con el atributo que se definió como obligatorio.
Posibles problemas	⌘ Estructuras redundantes y desaprovechamiento del espacio
Ventajas de su corrección	⌘ Reduce el espacio utilizado de la base de datos. ⌘ Reduce estructuras de datos redundantes. ⌘ Permite mayor abstracción de tipos o estados. ⌘ Hace las estructuras de datos más flexibles.
Notas: ⌘ Validar si estructuras de código PL/SQL estén tomando información de las tablas para proceder a ajustar las operaciones sobre las tablas.	

7. Estructuras de datos parciales duplicadas a través del modelo	
Descripción	Apoya en la detección de estructuras de datos parciales duplicadas a través de la base de datos.
Síntomas	Tablas que cuentan con atributos que en su definición son idénticos (mismo nombre(o), tipo y tamaño) y entre ellos son opcionales. En éste bad smell los atributos no hacen que las tablas sean idénticas, sólo que compartan parcialmente atributos.
Qué hacer	Se debe determinar si: ⌘ Existe a través de la base de datos dos o más tablas que cuentan con atributos que son idénticos en su definición de tipo, tamaño y opcionalmente nombre pues, los atributos pueden hacer referencia a la misma información conceptual y sin embargo, tener nombres diferentes (esto implica un mayor cuidado en la identificación de los atributos)¹⁰. ⌘ Si los atributos antes identificados son opcionales entre ellos a través de todas las estructuras de datos identificadas. ⌘ Si los atributos conceptualmente están relacionados y son excluyentes. ⌘ Los atributos identificados no hacen parte de llaves foráneas Extraer de todas las tablas los atributos y tratarlos de la forma como se trató el bad smell Estructuras de datos internas duplicadas.
Posibles problemas	⌘ Estructuras redundantes y desaprovechamiento del espacio ⌘ Posibilidad de incluir información aislada en una de las tablas y generar tuplas espurias, a través del tiempo, por causa de actualizaciones incompletas de la información.
Ventajas de su corrección	⌘ Reduce el espacio utilizado de la base de datos. ⌘ Reduce estructuras de datos redundantes. ⌘ Permite mayor abstracción de tipos o estados a través de la base de datos. ⌘ Hace las estructuras de datos más flexibles. ⌘ Elimina la posibilidad de generar tuplas espurias.
Notas: ⌘ Posterior a la extracción de las estructuras de datos a través de las tablas, se debe prestar especial atención a validar que permanezcan las dependencias funcionales de las tablas existentes de la base de datos con la nueva tabla. ⌘ Al realizar la extracción de la información común de las tablas se debe validar que permanezca la misma definición de los atributos, y validar si estructuras de código PL/SQL estén tomando información de las tablas para proceder a ajustar las operaciones sobre las tablas.	

¹⁰ Los atributos también pueden ser idénticos aunque no cuenten con el mismo nombre en su definición, lo que se debe validar es que tienen el mismo concepto y efecto sobre la información de la cual son responsables.

8. Estructuras de código PL/SQL duplicadas.	
Descripción	Apoya en la detección de estructuras de código duplicadas a través de la base de datos.
Síntomas	Se detecta código PL/SQL idéntico sintáctico y/o semánticamente a través de dos o más estructuras de código de la base de datos.
Qué hacer	Las estructuras de código PL/SQL que componen las bases de datos son: paquetes, procedimientos, funciones y triggers; dentro de éstas estructuras es posible identificar dos tipos de duplicidad: ⌘ Duplicidad sintáctica: Segmentos de código idéntico (o con diferencias mínimas) que se encuentran a través de varios objetos de la base de datos. ⌘ Duplicidad semántica: Segmentos de código que aunque sintácticamente no son idénticos, tienen una cercanía conceptual bastante pronunciada en cuanto a su funcionalidad se refiere. Se debe determinar si: ⌘ Existe duplicidad (sintáctica o semántica) dentro de un mismo procedimiento o entre dos o más procedimientos, éstos pueden estar ubicados en paquetes diferentes. ⌘ Existe duplicidad (sintáctica o semántica) en las responsabilidades y/o definición de los triggers. ⌘ Existe duplicidad (sintáctica o semántica) en las responsabilidades y/o definición de las funciones. Frente a estas situaciones, se debe extraer el código duplicado, identificando antes su complejidad, para determinar si se debe extraer a través de un procedimiento o función independiente. Una vez extraído el código se deben eliminar los segmentos duplicados en los lugares donde se identificó el bad smell y colocar llamados a procedimientos o funciones extraídos según corresponda.
Posibles problemas	⌘ Duplicidad del código innecesario. ⌘ Complejidad en actividades de mantenimiento de la base de datos.
Ventajas de su corrección	⌘ Al eliminar la duplicidad de las estructuras de código de la base de datos, disminuye la complejidad de actividades de mantenimiento de la base de datos, reduciendo así la posibilidad de incluir errores. ⌘ Aumenta el nivel de abstracción y comprensión en cuanto al comportamiento de la base de datos.
Notas: ⌘ Si existen funciones, procedimientos o triggers con parámetros de entrada idénticos y/o con un mínimo grado de diferencia en cuanto a longitud se refiere, es recomendable analizar el comportamiento de los datos dentro de estas estructuras y analizar si con breves validaciones estas estructuras se pueden unificar. PL/SQL con alto número de parámetros	

9. Vistas redundantes	
Descripción	<i>Apoya en la detección de vistas que por su información son redundantes.</i>
Síntomas	<i>Vistas definidas en la base de datos que por la información que presentan conceptualmente son redundantes.</i>
Qué hacer	<i>Las vistas cumplen un papel importante en el desempeño de las bases de datos, puesto que son una herramienta que evita realizar consultas complejas repetidamente, en cambio son estructuras que reflejan este tipo de consultas y permiten al usuario obtener información en un menor tiempo. Por otro lado, evitan que usuarios con pocos conocimientos de la base de datos y ante estructuras complejas de la misma, realicen consultas erróneas y entregar al usuario tuplas espurias no esperadas.</i> <i>El mantenimiento de las vistas también es costoso para la base de datos, por lo que se debe determinar si existen vistas con un alto grado de similitud y conceptualmente semejantes y unificar su información en una sola vista.</i>
Posibles problemas	⌘ <i>Disminución en el desempeño de la base de datos.</i>
Ventajas de su corrección	⌘ <i>Al consolidar en estructuras únicas y no duplicadas la información a través de las vistas se aumenta el tiempo de respuesta de consultas que pueden hacer los usuarios a la base de datos.</i>
Notas: ⌘	

10. Tablas aisladas	
Descripción	Apoya en detección de tablas que no tienen relaciones a través de la base de datos.
Síntomas	Tablas que no hacen referencia a información almacenada a través de la base de datos y tampoco son utilizadas como referencia.
Qué hacer	Normalmente las tablas con éstas características son utilizadas con propósitos de auditoria, es por esto, que es importante determinar cual es la razón de ser de la tabla. Se debe determinar si: ⌘ Si la tabla es utilizada con propósitos de auditoria. ⌘ Si se evidencia dependencias funcionales no relacionadas con el resto del modelo. En caso que a través de la tabla se pueda modelar dependencias funcionales, se debe verificar que dichas dependencias no se estén reflejadas a través de otras tablas y dependencias en la base de datos. Se debe entonces verificar cuales son los atributos que deberían estar relacionados con el resto de la base de datos y establecer dichas relaciones ó si falta algún atributo para completar la dependencia funcional incluirlo donde sea apropiado y establecer la dependencia funcional adecuadamente. En caso que se determine que la tabla no se está utilizando con propósitos de auditoria y tampoco refleja dependencias funcionales de la base de datos la tabla debe ser eliminada.
Posibles problemas	⌘ Ausencia de dependencias funcionales completas y/o necesarias a través de la base de datos. ⌘ Dependencias funcionales necesarias en la base de datos que no están reflejándose en el modelo de datos.
Ventajas de su corrección	⌘ Completitud en la información que almacena y administra la base de datos al reflejar las dependencias funcionales consistentes y completas en la base de datos.
Notas: ⌘ Para trabajar éste bad smell es necesario contar con la definición completa de las dependencias funcionales de la base de datos.	

11. Ausencia de restricciones principales	
Descripción	Apoya en detección de tablas con posible duplicidad de información.
Síntomas	Tablas que no tienen asociada a su estructura llaves primarias.
Qué hacer	Las llaves primarias son un elemento fundamental en la definición de la base de datos puesto que evitan a nivel de tabla información redundante y a nivel del modelo de la base de datos permite establecer relaciones únicas evitando así tablas espurias. Se debe determinar si: ☞ Existen tablas que no tienen relacionadas llaves primarias. Se debe proceder a definir la llave primaria para la tabla con el objeto de garantizar que no se almacena información redundante, para esto se identifica cual es el objetivo de la tabla, determinar cual será la llave primaria (atributo o conjunto de atributos que identifica ocurrencias individuales de información en una tabla), manejar adecuadamente la información hasta el momento almacenada en la tabla eliminando tuplas cuya llave primaria puede repetirse en la base de datos, definir la llave primaria para la tabla.
Posibles problemas	☞ Redundancia de datos. ☞ Tablas colgantes. ☞ Tablas espurias.
Ventajas de su corrección	☞ Se elimina la posibilidad de almacenar información redundante en la tabla. ☞ Frente a operaciones de uniones, se evita aparezcan tablas colgantes o espurias. ☞ Aumenta el tiempo de respuesta de consultas realizadas sobre esta tabla.
Notas:	
☞	

12. Dependencias funcionales	
Descripción	<i>Apoya en la identificación de dependencias funcionales ausentes en el modelo de la base de datos.</i>
Síntomas	<i>En el mantenimiento de la base de datos no existe claridad de la totalidad de las dependencias funcionales que se modelaron y se requerían modelar a través de la base de datos.</i>
Qué hacer	<i>Aunque no se conforma en su totalidad como un bad smell, al realizar mantenimiento a una base de datos es bueno contar con las dependencias funcionales definidas para la base de datos.</i> <i>Se debe identificar si:</i> ⌘ <i>Se cuenta con la definición de dependencias funcionales.</i> ⌘ <i>Si no se cuenta con la definición de un mínimo de dependencias funcionales, es necesario acudir con los usuarios de la base de datos y empezar a levantar información básica para determinar las dependencias funcionales.</i> ⌘ <i>Si se cuenta con un mínimo de dependencias definidas y/o se obtuvieron consultando con los usuarios. Se debe a través del uso de los axiomas de Armstrong determinar el cierre de F (determinar la totalidad de las dependencias funcionales de la base de datos).</i>
Posibles problemas	⌘ <i>No se cuentan con elementos formales para determinar si las relaciones realmente están reflejando las dependencias funcionales que se esperaban de la base de datos.</i>
Ventajas de su corrección	⌘ <i>Al contar con la completitud de las dependencias funcionales se tienen elementos de juicio para determinar y validar la presencia o no de diferentes bad smells que se apoyan de éstas definiciones.</i>
Notas: ⌘ <i>Al acudir a usuarios, se debe buscar personas o equipos de trabajo que tienen amplia experiencia en el manejo de aplicativos que soporta la base de datos, o del manejo de la misma base de datos, para contar así con información confiable.</i>	

13. Llave primaria con gran tamaño	
Descripción	<i>Permite detectar llaves primarias que hacen lenta la consulta sobre tablas.</i>
Síntomas	<i>Tabla con una llave primaria compuesta por un 50% o más de atributos del total de atributos que componen la tabla.</i>
Qué hacer	<i>Las llaves primarias permiten acceder más rápido a la información asociada a las tuplas de una tabla, sin embargo, si están compuestas por un gran número de atributos con relación al total de atributos que están relacionados a una tabla, las consultas se hacen lentas a medida que el tamaño de la base de datos aumenta.</i> ☞ <i>Se debe identificar la tabla según su función (tablas base, tablas transaccionales)</i> <i>Si la tabla es base, es de esperar que la llave primaria esté compuesta por el 50% de los atributos del total de atributos que compone la tabla.</i> <i>Si la tabla es transaccional, esperar que la llave primaria esté compuesta por el 50% del total de los atributos que componen la tabla resulta exagerado, por lo que es necesario definir el % aceptable para las tablas de la base de datos.</i>
Posibles problemas	☞ <i>Bajo desempeño de la base de datos en la ejecución de tablas con éste bad smell.</i>
Ventajas de su corrección	☞ <i>Disminuye el tiempo de respuesta de las consultas que relacionan dentro de su fuente de datos tablas con ésta característica</i>
Notas: ☞	

14. Estructuras de datos duplicadas a través del modelo	
Descripción	Apoya en la detección de estructuras de datos duplicadas a través de la base de datos ¹¹ .
Síntomas	Tablas que en sus atributos son casi idénticas (varían en atributos mandatorios o particulares de acuerdo a información adicional que se desea modelar), conceptualmente manejan los mismos datos y sin embargo su función es administrar información con significados semejantes.
Qué hacer	Se debe examinar el nombre y contenido de cada tabla y agruparlas en el siguiente orden: ⌘ Unir tablas con estructuras idénticas y que cuenten con la misma definiciones en sus llaves primarias. Generalmente tablas que manejan la misma llave primaria representan un mismo concepto en el mundo real y por lo tanto pueden ser unidas. Se debe tener especial cuidado con información duplicada. ⌘ Unir tablas con estructuras idénticas que tienen asociadas diferentes llaves primarias. Es posible encontrar dos o más entidades idénticas que manejan definiciones de llaves primarias diferentes pero cuentan con llaves candidatas similares. En éste caso es posible unir las tablas, sin embargo es necesario seleccionar una llave primaria o bien definiendo una nueva llave primaria o revisando las llaves candidatas comunes. ⌘ Unir tablas que aunque no tienen estructuras idénticas y manejan la misma o diferente llave primaria Tablas con estructuras casi idénticas pero que manejan el mismo propósito, es posible unificarlas de dos formas: • Si la estructura es idéntica conceptualmente, más no por su espacio de nombres, es necesario determinar si los atributos aunque con diferentes nombres cumplen con un mismo propósito, en éste caso se deben unir atributos comunes y unificar el nombre de los atributos diferentes. • Si las estructuras no son idénticas porque cuentan con atributos particulares adicionales, se debe unificar los atributos comunes en una nueva tabla o verificar si existe una tabla maestra que permita vincular ésta información y determinar si: los atributos adicionales son mandatorios y están relacionados con información particular para ciertos registros, entonces se recomienda vincular los atributos particulares en nuevas tablas y crear en la tabla que contiene la información natural un atributo que permita realizar referencia a las nuevas tablas como corresponda.

Posibles problemas	⌘ Estructuras redundantes y desaprovechamiento del espacio
Ventajas de su corrección	⌘ Reduce el espacio utilizado y estructuras redundantes. ⌘ Permite mayor abstracción de tipos o estados. ⌘ Hace las estructuras de datos más flexibles.
Notas: ⌘ Validar si estructuras de código PL/SQL estén tomando información de las tablas para proceder a ajustar las operaciones sobre las tablas. ⌘ Unificar o ajustar las llaves foráneas que pudieran estar referenciándose.	

15. Ausencia de índices	
Descripción	Apoya en detección de tablas que con el tiempo pueden presentar tiempos altos de respuesta ante operaciones CRUD.
Síntomas	Tablas que no tienen asociada a su estructura índices.
Qué hacer	Los índices son un elemento necesarios para optimizar el desempeño de las bases de datos puesto que indexan y permiten acceder a la información almacenada en las tablas de una forma más rápida, sobre todo ante operaciones de consulta y join con otras tablas. Se debe determinar si: ⌘ Existen tablas que no tienen relacionados índices. ⌘ La cantidad de atributos o registros es grande. Las tablas deben contar al menos con un índice asociado, a parte de la llave primaria, se debe identificar si: ⌘ La tabla tiene un campo o conjunto de campos que son consultados con frecuencia. ⌘ Si se utiliza algún campo o conjunto de campos en particular para realizar operaciones de unión. Si se identifica alguna de las situaciones anteriores es recomendable crear índices sobre el campo o el conjunto de campos que presentan las anteriores características.
Posibles problemas	⌘ Altos tiempos de respuesta ante operaciones CRUD sobre las tablas.
Ventajas de su corrección	⌘ Se optimiza la base de datos y los tiempos de respuesta ante operaciones realizadas sobre tablas particulares.
Notas: ⌘	

16. Atributo Repetitivo	
Descripción	Apoya en la detección de malos diseños donde la tabla con atributos con valores repetidos.
Síntomas	Tabla que el valor de uno o varios de sus atributos tienen una alta frecuencia de repetición.
Qué hacer	Se debe determinar si la tabla analizada es transaccional: Por cada atributo de dato, es decir que no pertenezca a llaves foráneas o primarias, identificar: ⌘ Si se conoce el dominio del atributo¹² ⌘ Si no se conoce el dominio del atributo determinar si es posible definir su dominio. ⌘ Si el dominio se mantiene en el tiempo o es variable. Para los casos anteriores, se debe validar que se presente una de las situaciones siguientes: 1. Que por el tamaño, alta transaccionalidad y si el dominio no varia en el tiempo, sea conveniente mantener la información vinculada directamente con la tabla, sin embargo para éste caso se debe establecer un constraint que valide el dominio de la información que se ingresa y asociar al atributo un índice. 2. Si por las características de transaccionalidad de la tabla, y si el dominio varia en el tiempo es conveniente, crear una nueva tabla base con la información del dominio del atributo y ajustar la tabla que se está analizando para establecer la relación con la nueva tabla, verificar la información y establecer la integridad referencial apropiadamente.
Posibles problemas	⌘ Redundancia de la información. ⌘ Anomalías de actualización.
Ventajas de su corrección	⌘ Para cualquiera de los casos, se asegura la integridad de la información. ⌘ Se eliminan problemas de anomalías de actualización.
Notas:	
⌘ Si se implementa la opción 2, validar si estructuras de código PL/SQL estén relacionada con la tabla para proceder a ajustar las operaciones sobre la tabla.	

17. Atributos co-relacionados	
Descripción	Apoya en la detección de dos o más atributos que pertenecen a una misma tabla y tienen un registro de valores estrechamente relacionados.
Síntomas	Dentro de una tabla, dos o más atributos que presentan un comportamiento co-relacionados en sus valores.
Qué hacer	Se debe determinar si la tabla a analizar es transaccional y presenta alguno de los siguientes patrones: ⌘ Si la llave primaria es compuesta y se puede identificar que parte de la llave tiene una alta ocurrencia. ⌘ Si se cumple la anterior condición, y existe uno o más atributos no claves o de datos que sus valores son conocidos y frecuentemente acompañan a la llave primaria. ⌘ Si existen dos o más atributos no claves o de datos que son altamente recurrentes¹³ y éstos atributos en sus valores están co-relacionados, es decir ante la presencia de X en un atributo_1 repetitivo existe un atributo_2 con dominio conocido que frecuentemente acompaña el valor X. Si el dominio de los datos co-relacionados (identificados anteriormente) es conocido, el grado del dominio es pequeño, y la tabla transaccional no es significativamente grande se recomienda mantener la información correlacionada en la tabla, vinculando a la información un índice. Si se presenta al menos uno de los patrones descritos anteriormente, se debe considerar la posibilidad de vincular el conjunto de atributos antes identificados de la tabla en una nueva entidad y ajustar la tabla para que reciba un campo que relacione la información que se desvinculó con la nueva tabla, especialmente si la tabla transaccional es significativamente grande.
Posibles problemas	⌘ Redundancia de información. ⌘ Anomalías de actualización, si existe información altamente correlacionada es posible que al actualizar información se modifiquen datos que no debían tocarse, lo que puede provocar tablas colgantes.
Ventajas de su corrección	⌘ Integridad de información co-relacionada a través de la base de datos. ⌘ Se evitan problemas en anomalías de actualización. ⌘ Se optimizan los tiempos de respuesta de la base de datos ante operaciones CRUD sobre la tabla que se esta analizando.
Notas: ⌘ Validar si estructuras de código PL/SQL estén relacionada con la tabla para proceder a ajustar las operaciones sobre la tabla.	

18. Obsesión por las relaciones	
Descripción	<i>Permite detectar tablas muy pequeñas que por los datos que almacena, la tabla y sus relaciones son poco utilizadas.</i>
Síntomas	<i>Tabla que tiene relaciones que son utilizadas con poca frecuencia en otra tabla.</i>
Qué hacer	<i>Se debe determinar si la tabla a analizar es base y satisface las siguientes condiciones:</i> <i>☞ Almacena como máximo tres atributos de datos.</i> <i>☞ Si tiene una única relación y ésta es utilizada para hacer referencia a los valores almacenados en la tabla que se está analizando.</i> <i>☞ Si la frecuencia con que los datos almacenados son utilizados en la tabla que le hace referencia, es baja.</i> <i>☞ Si los registros que hasta el momento se mantienen se utilizan como referencia y no serán eliminados en el tiempo (Si se eliminan no es recomendable pues es posible que se pierda información)</i> <i>Si cumple las anteriores condiciones, es conveniente considerar la posibilidad de eliminar la tabla base y permitir que la información quede almacenada en la tabla que le hace referencia, teniendo presente que los atributos dato de la tabla base deben ser vinculados a la tabla que le hace referencia, agregar constraint para asegurar que los valores que se ingresen o modifiquen a través del tiempo sean consistentes e indexar los atributos.</i> <i>Es importante resaltar que esta es una consideración de diseño puesto que al aplicar la corrección a éste “bad smell” se está violando el principio de la forma normal FNBC.</i>
Posibles problemas	<i>☞ La obsesión por las relaciones en general no presentan ningún problema.</i>
Ventajas de su corrección	<i>☞ Disminuye el tiempo de respuesta de las operaciones sobre las tablas que intervienen.</i> <i>☞ Evita al usuario realizar join innecesarios.</i>
Notas: <i>☞</i>	

19. Relaciones cíclicas	
Descripción	<i>Permite detectar relaciones cíclicas no justificadas a través de la base de datos.</i>
Síntomas	<i>Tablas que entre sus atributos hacen referencia a atributos de terceras tablas que podrían ser recuperados por a través de atributos relacionados inicialmente.</i>
Qué hacer	<i>Las relaciones cíclicas son válidas cuando se están modelando patrones de jerarquía, cadena o red, útiles para modelar relaciones de precedencia, donde la secuencia es importante o donde se habla de pares de cosas del mismo tipo.</i> <i>Se identifican fácilmente cuando existen relaciones 1:1 a través de varias entidades, 1:m, m:m sobre una misma entidad.</i> <i>Para determinar si hay fallas en modelos de relaciones cíclicas se debe verificar las siguientes condiciones:</i> ⌘ <i>Una tabla no debe tener dos atributos no clave que conduzcan a la misma información ya sea por transitividad o por relación directa.</i> <i>En caso que se presente la anterior condición, se debe analizar cuál atributo debe ser retirado de la tabla, por lo general se debe permitir la presencia del atributo que por transitividad conduce a la misma información, puesto que a través de las diferentes relaciones se puede obtener más información.</i>
Posibles problemas	⌘ <i>Duplicidad de la información.</i> ⌘ <i>Anomalías de actualización.</i>
Ventajas de su corrección	⌘ <i>Se mantiene un mayor nivel de abstracción de la información de la cual es responsable la tabla analizada.</i> ⌘ <i>Se elimina la información redundante.</i>
Notas: ⌘	

20. PL/SQL con alto número de parámetros	
Descripción	Apoya en la detección de estructuras de código que en su declaración cuentan con un alto número de parámetros
Síntomas	Se detecta código PL/SQL donde la cantidad de valores de entrada o salida es alta.
Qué hacer	Las estructuras de código PL/SQL que componen las bases de datos son: paquetes, procedimientos, funciones y triggers. Se debe determinar sí: ⌚ Existe otro objeto PL/SQL, en la base de datos, posiblemente del mismo tipo, que tenga una lista de parámetros semejantes o idéntica semánticamente. Si se detecta ésta situación, es posible que se trate del bad smell Estructuras de código PL/SQL duplicadas. ⌚ Si existen estructuras que no tienen una duplicidad semántica, en su comportamiento y sin embargo, tienen parámetros de entrada y/o salida semejantes, es posible que aunque hagan cosas distintas se pueda realizar una generalización de ellas, de tal forma que: parámetros de entrada, se incluyan dentro del código como llamados a funciones, que dada la necesidad de consulta entregue resultados según lo que se requiera. Y ajustar el código, para que dado los resultados por las nuevas funciones, realice las operaciones necesarias para entregar los resultados necesarios. Con los parámetros de salida, determinar cuáles realmente son responsabilidad del objeto y cuales pueden ser consultados a través de elementos, igualmente genéricos.
Posibles problemas	⌚ Duplicidad de código innecesario ⌚ Complejidad en actividades de mantenimiento de la base de datos.
Ventajas de su corrección	⌚ Al eliminar la duplicidad de las estructuras de código de la base de datos, disminuye la complejidad de actividades de mantenimiento de la base de datos, reduciendo así la posibilidad de incluir errores. ⌚ Aumenta el nivel de abstracción y comprensión en cuanto al comportamiento de la base de datos.
Notas: ⌚ Al realizar los análisis de duplicidad sintáctica o semántica de las estructuras, se debe tener presente no solo parámetros de entrada, la forma en que se trata la información, también se debe tener presente los parámetros de salida y su tipo.	

21. Atributos relacionados inadecuadamente	
Descripción	<i>Tablas que sus atributos, tienen más relación con otra tabla, que con la que se encuentra relacionado</i>
Síntomas	<i>Atributos que por el significado y responsabilidad de la cual son responsables, deben pertenecer a una tabla diferente a la que se encuentran definidos.</i>
Qué hacer	<i>Se debe determinar si:</i> ⌘ <i>Existen consultas, que realizan las aplicaciones a la base de datos, donde siempre que se pregunta por la información de una tabla particular, siempre se esté preguntando por un atributo que se encuentre en otra tabla.</i> ⌘ <i>Existen operaciones CRUD a través de los objetos PL/SQL, que presenten el mismo comportamiento descrito anteriormente.</i> <i>Para los casos anteriores, se debe determinar si el atributo el atributo identificado, realmente pertenece a la tabla donde se encuentra relacionado. O si por el contrario, es un atributo que debe retirarse de la tabla donde se encuentra y relacionarse en la tabla donde tiene mayor incidencia.</i>
Posibles problemas	⌘ <i>Altos tiempos de respuesta ante operaciones CRUD sobre las tablas.</i>
Ventajas de su corrección	⌘ <i>Se optimiza la base de datos y los tiempos de respuesta ante operaciones realizadas, donde la(s) tablas y atributo(s) se encuentre relacionado.</i>
Notas:	
⌘	

22. Inapropiada intimidad entre tablas	
Descripción	<i>Tablas que en sus datos están demasiado acoplados.</i>
Síntomas	<i>Tupla de una tabla, que tiene dependencia funcional con una tupla localizada en otra tabla. Y existe correspondencia entre tuplas de las tablas identificadas</i>
Qué hacer	<i>Específicamente, éste bad smell tiene mayor alcance que el bad smell Atributos relacionados inadecuadamente</i> <i>Se debe determinar si:</i> ⌘ <i>Las operaciones CRUD que se realizan sobre una tabla, sea por PL/SQL o consultas de aplicaciones, siempre que se realiza una operación particular sobre la tupla de una tabla (P), se debe realizar la misma operación y/o actualizar información de una tupla particular de otra tabla(S).</i> <i>Lo que puede indicar que la información de las dos tablas está altamente acoplada y es posible. Si cumple las anteriores condiciones, es conveniente considerar la posibilidad de mantener la tabla(P) por dónde siempre inicia la operación y permitir que la información de la tabla (S) se almacene tabla(P), teniendo presente agregar los atributos dato, y revisar la validez de la llave primaria.</i>
Posibles problemas	⌘ <i>Altos tiempos de respuesta ante operaciones CRUD sobre las tablas.</i>
Ventajas de su corrección	⌘ <i>Se optimiza la base de datos y los tiempos de respuesta ante operaciones realizadas, donde la(s) tablas y atributo(s) se encuentre relacionado.</i>
Notas:	
⌘	

23. Valores nulos	
Descripción	<i>Apoya en la detección de diseños donde los atributos definidos para una tabla son opcionales.</i>
Síntomas	<i>Atributos con opción de ser nulos según la definición de tabla en la que se encuentra.</i>
Qué hacer	<i>Los valores nulos ocasionan desperdicio de espacio y dificulta el entendimiento del significado de los atributos, adicional a esto genera problemas en operaciones de reunión.</i> <i>Un valor nulo, puede dar espacio a diferentes interpretaciones:</i> ◦ <i>Que el atributo no aplica al registro.</i> ◦ <i>Se desconoce el valor asociado a un registro</i> ◦ <i>Valor puede ser conocido pero está ausente.</i> <i>Se debe determinar sí:</i> ↯ <i>Existen tablas en las que el % de registros opcionales es mayor que los registros obligatorios.</i> ↯ <i>Existen atributos que por reglas de negocio y/o dependencias funcionales acepta valores nulos.</i> ↯ <i>Atributos numéricos que permiten valores nulos.</i> <i>Para los atributos identificados anteriormente, se debe determinar el valor agregado de éstos valores nulos y evaluar la posibilidad de:</i> ↯ <i>Establecer sobre el atributo un constraint check, modificando el atributo a NOT NULL...</i> ↯ <i>Separar la información opcional a otra tabla y relacionarla como es debido.</i> ↯ <i>Para atributos numéricos, se recomienda, siempre mantener un valor conocido por defecto en caso que no se pueda establecer su valor en un momento dado.</i>
Posibles problemas	↯ <i>Problemas en operaciones de reunión natural.</i> ↯ <i>Desperdicio de espacio en la base de datos.</i> ↯ <i>Hace compleja la comprensión del significado de un valor nulo en los atributos.</i>
Ventajas de su corrección	↯ <i>Permite una comprensión más clara del significado de un atributo, del cual todavía no se conoce su valor, o no aplica para una tupla determinada.</i> ↯ <i>Evita problemas ante operaciones de reunión natural.</i>
Notas:	
↯	

24. Tuplas espurias	
Descripción	Al realizar operaciones de reunión natural, se genera más información de la que normalmente es debida.
Síntomas	Tablas que no tienen llave primaria y/o foráneas.
Qué hacer	Se debe determinar sí: ⌘ Existen tablas, que no cuenten con llave primaria y se encuentran relacionadas con otras tablas, pues es posible que ante operaciones de reunión natural se genere más información de la debida. ⌘ Existen relaciones que contengan atributos coincidentes que no sean combinaciones de llaves foráneas o llaves primarias. Ante estos casos, se debe definir sobre las tablas o bien llaves foráneas o constraints que permitan validar la información que se registra, actualiza o elimina en el tiempo. Y garantizar que las relaciones contengan atributos coincidentes por reunión a través de llaves primaria o foráneas.
Posibles problemas	⌘ Tuplas espurias ⌘ Anomalías de actualización
Ventajas de su corrección	⌘ Integridad y mantenibilidad de la información a través del tiempo. ⌘ Se elimina la posibilidad que con el tiempo se genere inconsistencia o pérdida de información por la actualización no adecuada de información. ⌘ Reduce espacio utilizado en la base de datos.
Notas: ⌘	

25. Valores no atómicos	
Descripción	<i>Atributos que relacionan valores no atómicos.</i>
Síntomas	<i>Tablas que cuentan con atributos no atómicos.</i>
Qué hacer	<i>Se debe determinar sí:</i> ☞ <i>Existen atributos que permiten registrar valores compuestos y divisibles.</i> <i>Si se presenta éste caso:</i> ☞ <i>Se debe identificar, si es posible, y si el número máximo de valores para los atributos no es alto (máx. 3) el dominio de valores; posteriormente reemplazar el atributo no atómico por n-atributos, de acuerdo con el dominio identificado. Se debe tener presente los valores que se relacionarán con los atributos para no caer en el bad smell Valores nulos</i> ☞ <i>Si el rango de valores es alto, se debe evaluar la posibilidad de llevar éste atributo a una tabla diferente y asociarla con información relacionada.</i>
Posibles problemas	☞ <i>Violación de la 1FN.</i>
Ventajas de su corrección	☞
Notas:	
☞	

26. Dependencia a la llave primaria	
Descripción	<i>Atributos dato que no dependen de la llave primaria.</i>
Síntomas	<i>Atributos de dato, que no dependen totalmente de la llave primaria</i>
Qué hacer	<i>Se debe determinar sí:</i> ☞ <i>Existen tablas que cuentan con llaves primarias compuestas, sobre éstas tablas validar: Que todos los atributos dato, dependen de manera total de la llave primaria.</i> <i>Para aquellas tablas, donde existen atributos que dependen de la llave primaria de forma parcial, se deben normalizar, creando nuevas tablas normalizadas en 2FN, en las que los atributos no primos estén asociados sólo a la parte de la llave primaria de la que dependen de forma total.</i>
Posibles problemas	☞ <i>Violación de la 2FN</i>
Ventajas de su corrección	☞ <i>Se evitan problemas de anomalías de actualización.</i>
Notas:	
☞ <i>Contar con las dependencias funcionales totales de la base de datos resulta útil para éste bad smell.</i>	

27. Sintaxis de los atributos	
Descripción	<i>Duplicidad semántica de los atributos.</i>
Síntomas	<i>Atributos definidos a través de la base de datos con distintos nombres pero con responsabilidades idénticas.</i>
Qué hacer	<i>Se debe determinar sí:</i> ⌘ <i>Existe duplicidad semántica sobre los atributos definidos a través del diseño de la base de datos, es decir, se debe analizar los atributos para establecer si existe más de un atributo con la misma responsabilidad y sin embargo, tienen diferente nombre.</i> <i>Se debe determinar cuál es el nombre característico del atributo y reemplazar la descripción de los diferentes atributos con el mismo nombre.</i>
Posibles problemas	⌘ <i>Redundancia de información.</i> ⌘ <i>Por falta de un enfoque consolidado de los atributos, es posible se presente fallas en el proceso de normalización de la base de datos.</i>
Ventajas de su corrección	⌘ <i>Evita redundancia de información innecesaria.</i> ⌘ <i>Comprensión consolidada de la responsabilidad de un atributo sobre la base de datos.</i> ⌘ <i>Permite comprender de una forma más limpia el diseño y la normalización efectuada sobre la información que administra la base de datos.</i>
Notas: ⌘ <i>Es importante, tener cuidado al realizar estos cambios, pues puede hacer evidentes bad smells que hasta el momento no se habían identificado.</i>	

28. Dependencias transitivas	
Descripción	Atributos dato que dependen de manera transitiva de la llave primaria.
Síntomas	Atributo dato dependiente funcionalmente de otro atributo dato de la tabla
Qué hacer	Se debe determinar sí: ☞ Existen tablas con atributos datos determinados funcionalmente por uno o varios atributos dato, pues esto genera dependencias transitivas. Para aquellas tablas, donde se determina la situación antes descrita, se debe descomponer la tabla y crear una nueva que incluya los atributos no clave que se determinen funcionalmente a otro u otros atributos no clave.
Posibles problemas	☞ Viola la forma normal 3FN.
Ventajas de su corrección	☞ Se evitan problemas de anomalías de actualización.
Notas:	
☞	

29. Constraints duplicados	
Descripción	Apoya en la detección de constraints duplicados dentro del modelo.
Síntomas	Constraint duplicados que aunque se encuentran definidos bajo diferentes nombres, tienen el mismo propósito.
Qué hacer	Se debe determinar si: ☞ Existe duplicidad semántica sobre constraints definidos sobre una misma tabla. ☞ Existe duplicidad semántica sobre constraints definidos sobre relaciones entre tablas. Si se presentan los anteriores casos, se debe determinar cuál es el nombre definitivo del constraint y eliminar los constraint duplicados.
Posibles problemas	☞ Altos tiempos de respuesta ante operaciones sobre las tablas, puesto que si se tiene más de un constraint con la misma funcionalidad, se tomará mas tiempo para dar respuesta a las operaciones solicitadas.
Ventajas de su corrección	☞ Se optimiza la base de datos y los tiempos de respuesta ante operaciones realizadas, donde la(s) tablas, atributo(s) y relaciones entre tablas.
Notas:	
☞	

5.3. Organización Taxonómica

A continuación se presenta una propuesta taxonómica, con base en los bad smells de BDR definidos.

La organización por categorías relacionadas, tiene por objeto facilitar la tarea de detección de bad smells de BDR, de acuerdo al interés de refactoring con el que se inicien proyectos de reingeniería a bases de datos relacionales.

5.3.1 Categorías relacionadas

En general, los bad smells de BDR hacen referencia a aspectos detectados que influyen en el diseño ER de la base de datos, al clasificar los bad smells se quiere enfatizar en los conceptos de diseño susceptibles de ser mejorados en mayor grado.

La clasificación de los bad smells de BDR se realiza a través de tres aspectos:

- ☞ Aspectos arquitectónicos:
Agrupa bad smells de BDR presentes en aspectos estructurales de los objetos que componen la base de datos.
- ☞ Aspectos de Integridad
Agrupa bad smells de BDR que detectan aspectos que afectan la integridad de los datos almacenados en la base de datos.
- ☞ Aspectos de Calidad
Agrupa bad smells de BDR que detectan aspectos del diseño que afectan negativamente la calidad de los datos almacenados y/o el diseño ER.

5.3.2 Aspectos arquitectónicos

Los aspectos arquitectónicos se catalogan teniendo en cuenta su presencia en el modelo, así:

- ☞ Dentro de tablas
- ☞ Entre tablas

La catalogación de los aspectos dentro de tablas se constituye como se observa en la Figura 19.

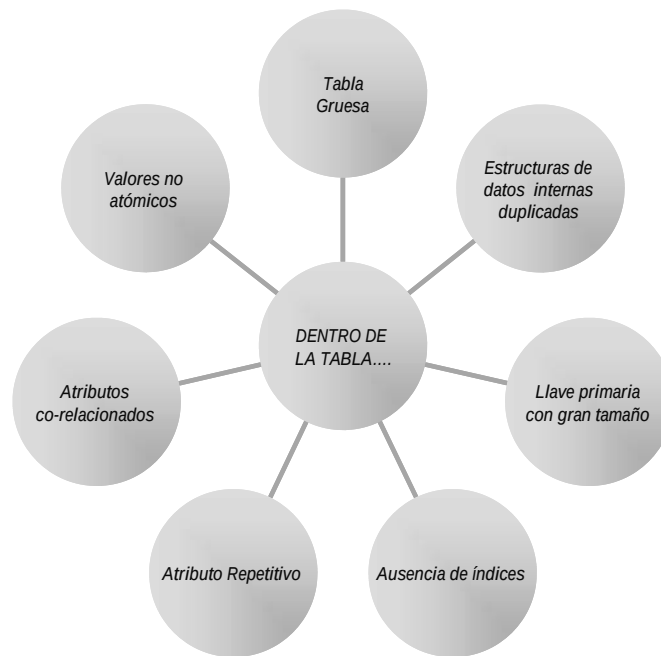


Figura 19 Taxonomía de bad smells BDR arquitectónicos- dentro de tablas

La catalogación de los aspectos entre tablas se constituye como se observa en la Figura 20.

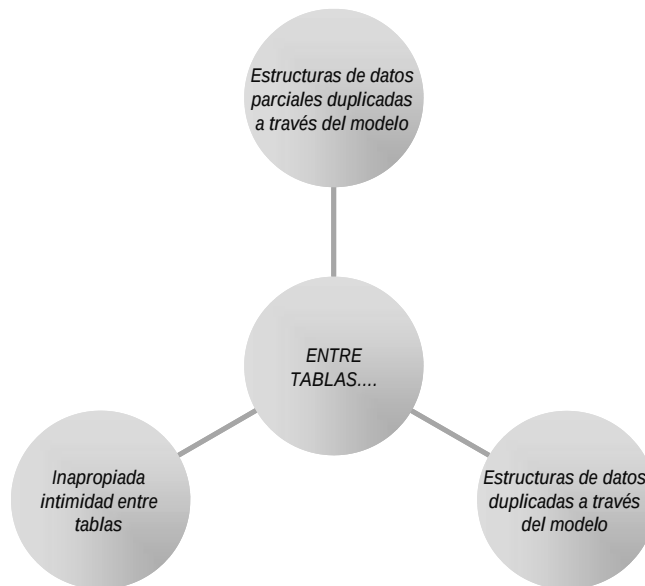


Figura 20 Taxonomía de bad smells BDR arquitectónicos- entre tablas

5.3.3 Aspectos de integridad

Los aspectos de integridad se catalogan teniendo en cuenta su presencia en el modelo, así:

- ↯ Dentro de tablas
- ↯ Entre tablas

La catalogación de los aspectos dentro de tablas se constituye como se observa en la Figura 21.



Figura 21 Taxonomía de bad smells BDR de integridad- dentro de tablas

La catalogación de los aspectos entre tablas se constituye como se observa en la Figura 22.

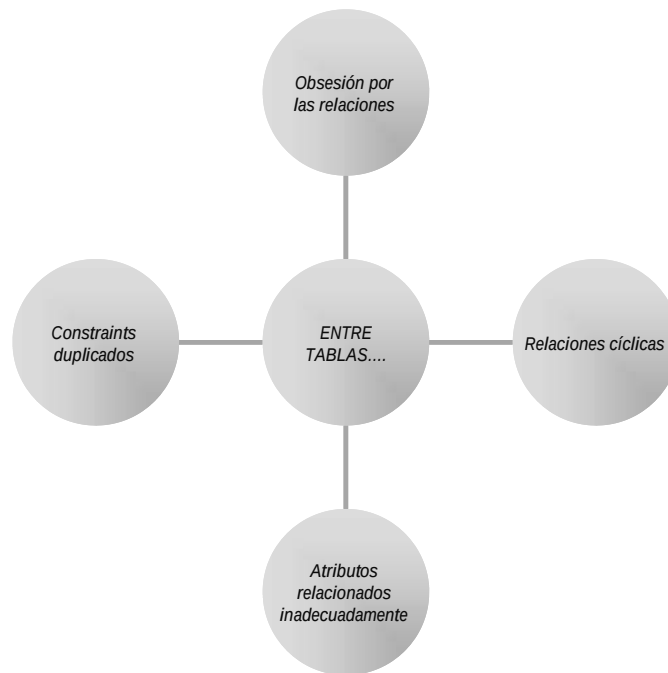


Figura 22 Taxonomía de bad smells BDR de integridad - entre tablas

5.3.4 Aspectos de Calidad

Los aspectos de calidad aplican al diseño en general, los bad smells vinculados a esta categoría se pueden identificar en la Figura 23.

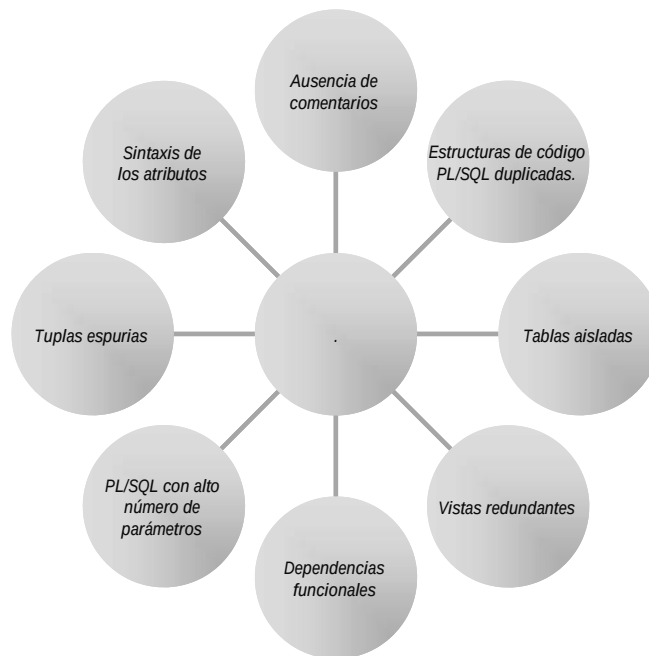


Figura 23 Taxonomía de bad smells BDR de calidad

6. ESTRATEGIA PARA DETECCIÓN DE BAD SMELLS EN BASES DE DATOS RELACIONALES

La construcción de una herramienta parte de la idea de: proporcionar un mecanismo automático para detección de bad smells en bases de datos relacionales. Ésta ha sido llamada “BadSmellsBDR” y se constituye como un framework que no solo apoya el desarrollo de éste trabajo; adicional se ha construido con la flexibilidad necesaria para soportar trabajos posteriores de automatización, en detección de bad smells en bases de datos relacionales y código java, entre otros. (Para ver requerimientos de la aplicación ver en el Anexo 4)

La aplicación implementa cuatro de los bad smells propuestos en 5.2, con el objeto de evidenciar la forma automática en la que se puede detectar su posible presencia, en modelos de bases de datos relacionales.

6.1. Mecanismo de Operación

La herramienta está construida en un modelo de tres capas, como se observa en la Figura 24, lo que proporciona movilidad para poder realizar actividades de detección de bad smells, dado que no debe contar con un ambiente especialmente configurado para ésta labor en particular.

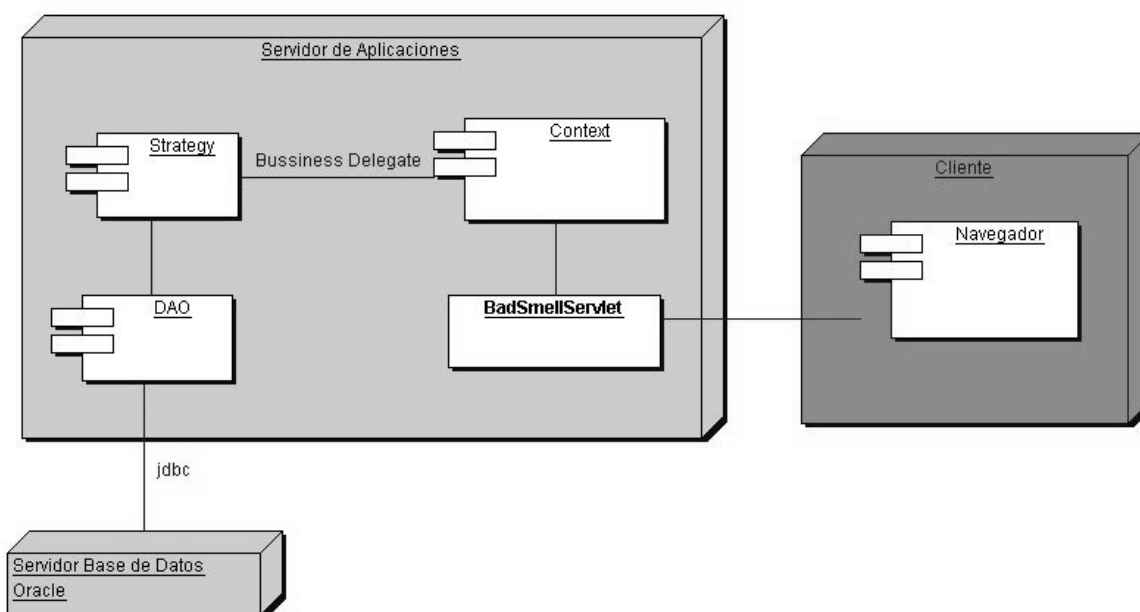


Figura 24 Arquitectura de “BadSmellsBDR”

Como se ha mencionado anteriormente, BadSmellsBDR es un framework que permite y permitirá soportar trabajos de detección automáticos de bad smells. Se ha utilizado como patrón marco MVC, con el propósito de separar claramente las responsabilidades de los componentes a través de la arquitectura de la aplicación.

Una vez identificados todos los paquetes y sus responsabilidades (Anexo 3), el patrón MVC se encuentra distribuido de la siguiente forma:

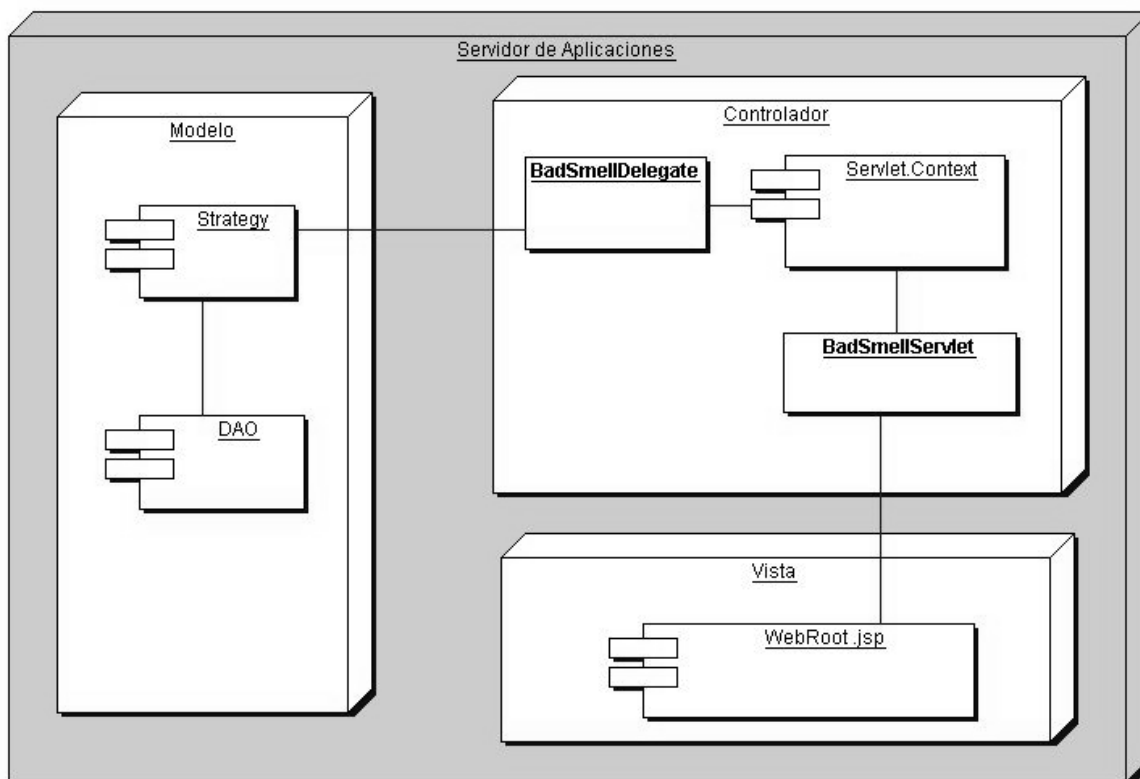


Figura 25 Patrón MVC en BadSmellsBDR

6.1.1 Detalle Técnico del Proceso de Detección de Bad Smells

Para iniciar el proceso de detección de bad smells es necesario tener cargada en la herramienta la configuración de la base de datos sobre la cual se realizará el análisis, para esto, el usuario debe proveer a la aplicación la ubicación del archivo .xml que contenga la siguiente estructura (Figura 26):

```

<?xml version="1.0" encoding="UTF-8"?>
<configuracionBadSmell>
  <configuracionBaseDatos>
    <urlConexion>jdbc:oracle:thin:@host:puertoBd:nombreBd</urlConexion>
    <usuario>usuario de conexión</usuario>
    <password>contraseña</password>
  </configuracionBaseDatos>
</configuracionBadSmell>

```

Figura 26 Parámetros de configuración para conexión a bases de datos –Archivo XML

Para comprender la forma como BadSmellsBDR detecta bad smells, a continuación se presenta el diagrama de secuencia de la aplicación.

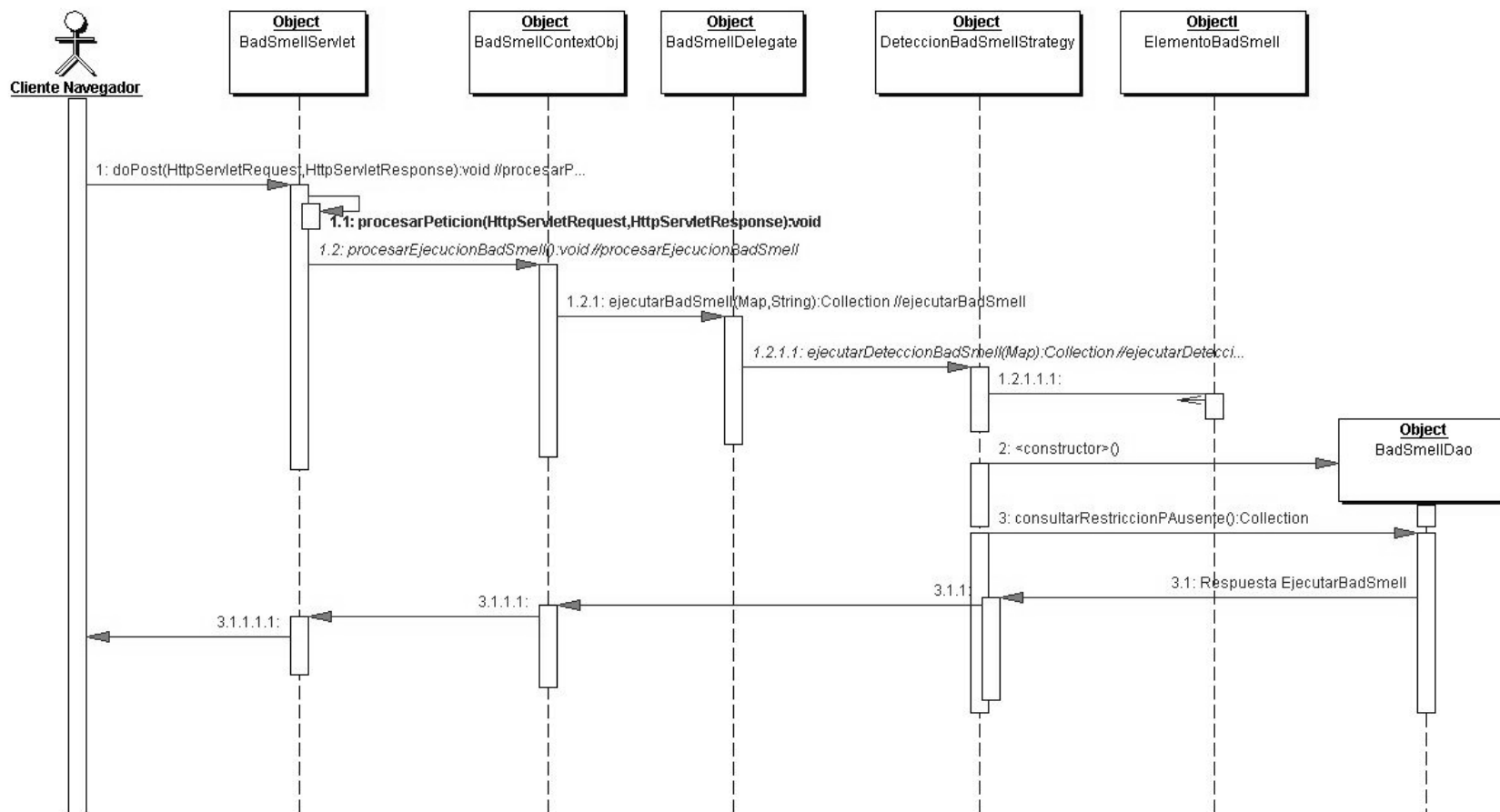


Figura 27 Diagrama de secuencia para detección de bad smells en BadSmellsBDR

6.1.2 Detalle Visual del Proceso de Detección de Bad Smells

De acuerdo con la descripción técnica detallada en 6.1.1, se puede observar como el proceso de detección de bad smells de la aplicación se ejecuta en el siguiente orden (Figura 28):

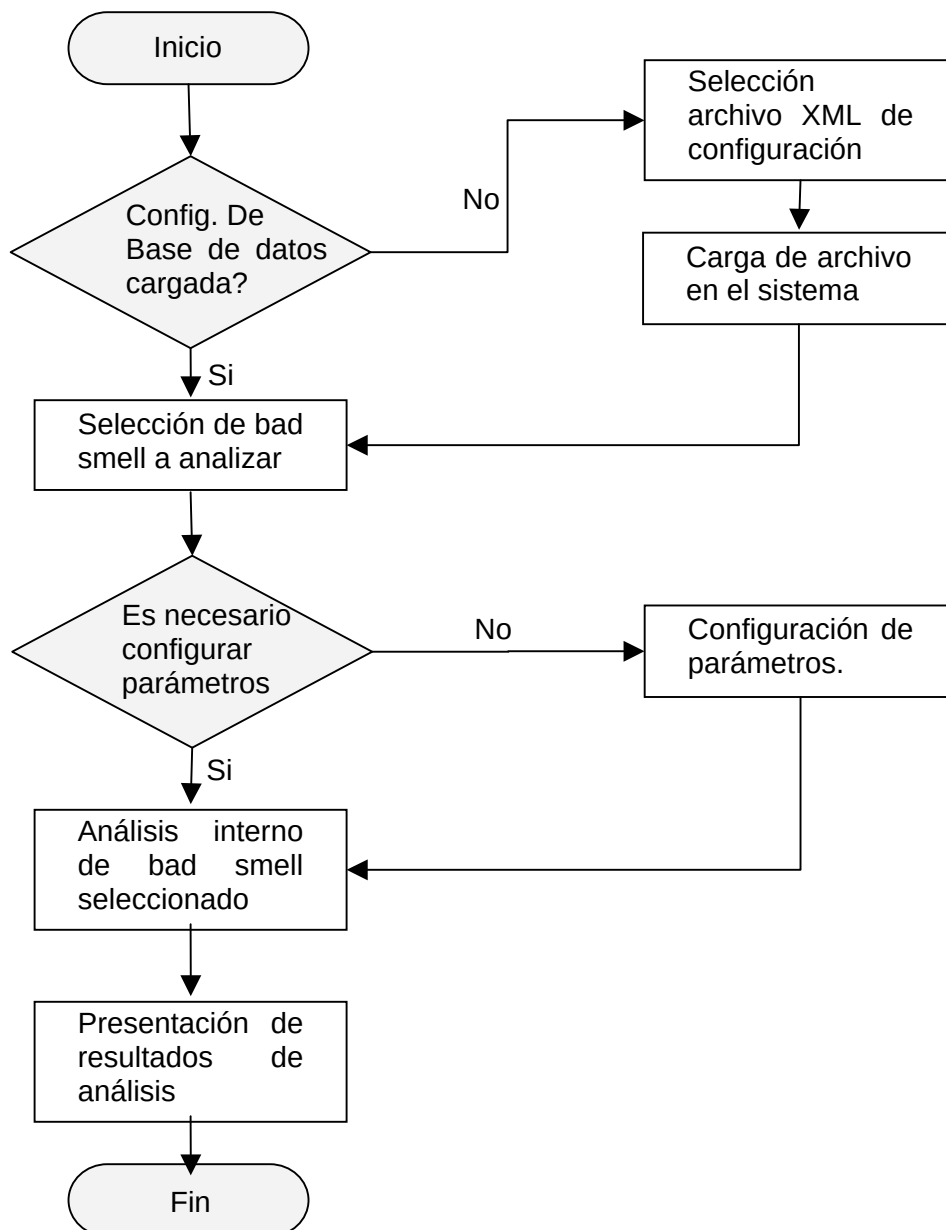


Figura 28 Orden en el cual visualmente se ejecuta la aplicación BadSmellsBDR

En el Anexo 5 se presenta un ejemplo de detección efectuado por la herramienta, para un caso de prueba muy corto, a través de un manual de usuario con el objeto de presentar visualmente el comportamiento de la aplicación.

6.2. Cómo incluir un nuevo Bad Smell a BadSmellsBDR

Como ya se ha mencionado BadSmellsBDR, ha sido construido como un framework, que permite a través de actividades puntuales, configurar nuevos bad smells para ser detectados.

El patrón MVC ha sido el marco de referencia para la construcción de la aplicación, por lo que la lista de chequeo para ajustar nuevos bad smell a BadSmellsBDR, se definirá en el mismo sentido.

Antes de iniciar los ajustes se debe tener claro:

- ⌘ Nombre de paquete característico del bad smell: Por lo general en minúsculas, lleva el mismo nombre del bad smell, para éste caso nombadsmell.
- ⌘ Sentencia sql: Al realizar análisis sobre un esquema determinado es necesario acceder al diccionario de datos de la base de datos (la aplicación utiliza para sus análisis esquemas configurados en bases de datos oracle ®), para éste caso sentenciasql
- ⌘ Item constante del bad smell: Sirve para identificar el bad smell a través de la aplicación, con base en éstos ítem el servlet sabe que context object llamar. Tienen un estándar: ITEM_MENU_BAD_SMELL_NOMBRE_BAD_SMELL

6.2.1 Ajustes al Modelo

El ajuste se debe realizar en el paquete co.edu.uniandes.badsmells.logica, de la siguiente forma.

Dentro del Paquete....	Ajuste
Dto	⌘ Se debe crear un nuevo paquete con el nombre <u>nombadsmell</u>. ⌘ Dentro del nuevo paquete, crear una clase <u>Respuestanombadsmell.java</u>, que implemente la interface <u>ElementoBadSmell.java</u>
Dao	⌘ En la clase <u>BadSmellDao.java</u>: Incluir una variable de clase “private static final <u>SQL_NOMBADSMELL</u>” = <u>sentenciasql</u> ⌘ Crear un nuevo método <u>consultarnombadsmell</u>. (guiarse en estructura por los ya escritos).
strategy	⌘ Se debe crear un nuevo paquete con el nombre <u>nombadsmell</u>. ⌘ Dentro del nuevo paquete, crear una clase <u>nombadsmellStrategy.java</u>, que implemente la interface <u>DeteccionBadSmellStrategy.java</u>

6.2.2 Ajustes al Controlador

El ajuste se debe realizar en el paquete co.edu.uniandes.badsmells.presentacion, de la siguiente forma.

Dentro del Paquete....	Ajuste
delegate	En la clase badSmellDelegate.java, se debe modificar el constructor, para agregar en el Map el nuevo objeto <u>BadSmellStrategy</u> (ver implementación).
context	- Se debe crear un nuevo paquete con el nombre <u>nombadsmell</u>. - Dentro del nuevo paquete, crear una clase <u>nombadsmellContextObj.java</u>, que implemente la interface BadSmellContextObj.java
servlet	En la clase badSmellServlet.java, se debe modificar el método init, para agregar en el Map el nuevo objeto <u>nombadsmellContextObj</u> (ver implementación).
constantes	- Dentro de la clase ConstantesPresentacion.java, agregar la variable de clase: <pre>public static final String ITEM_MENU_BAD_SMELL_NOMBRE_BAD_SMELL=<u>nombads mell</u>;</pre>

6.2.3 Ajustes a la Vista

El ajuste se debe realizar en el directorio WebRoot, de la siguiente forma.

Dentro del Paquete....	Ajuste
badsmells	- Si el bad smell requiere parámetros dados por el usuario: Se debe crear un nuevo paquete con el nombre <u>nombadsmell</u>. - Dentro del nuevo paquete, crear un archivo inicioBadSmell.jsp, que solicite los parámetros requeridos y el botón implemente onclick="procesar()".
nostree	En el archivo tree_nodes.jsp, agregar la opción del menú (ver implementación estándar)

7. VALIDACIÓN DE LA ESTRATEGIA DE DETECCIÓN

Ese capítulo presenta los resultados del proceso de validación de la estrategia propuesta para la detección de los bad smells: Tabla gruesa, Tablas aisladas, Llave primaria larga y Restricciones principales ausentes.

7.1. *Caso de Estudio*

Para efectuar la validación, se seleccionó una base de datos relacional de tamaño considerable, que por su dimensión permite que: la actividad de detección esté al alcance de la inspección humana, sin ser tan evidente.

7.1.1 Marco Conceptual del caso seleccionado

[TMFM88] En la industria de telecomunicaciones TM Forum®, es una entidad constituida en 1988 por grandes proveedores de servicios de telecomunicaciones e información. Desde entonces ésta entidad se ha enfocado en proporcionar a la industria lineamientos, soluciones estratégicas y prácticas para mejorar la administración de la información y servicios de telecomunicaciones.

TM Forum ®, ha proporcionado el framework “eTOM” (Figura 29) donde propone los sistemas que debe soportar una empresa proveedora de servicios de telecomunicaciones.

Modelo eTOM

Telecom Operating Map



Figura 29 Modelo eTOM - TM Forum ®

Sistemas OSS (Operating Support Systems) soportan de acuerdo al modelo eTOM, el área de “Operaciones” donde su responsabilidad se centra en:

- ☞ La gestión de elementos
- ☞ La gestión de redes de comunicaciones e información
- ☞ La gestión de servicios y operaciones
- ☞ La gestión del negocio
- ☞ Desarrollo de recursos (Aplicación, computación y red) y administración
- ☞ Desarrollo de la cadena de aprovisionamiento y administración

7.1.2 Caso de estudio seleccionado

Sistema OSS que actualmente se encuentra en operación, de una empresa de la industria de telecomunicaciones. El diagrama entidad relación en su totalidad no se presenta por su tamaño sin embargo, se presentará parcialmente a través de los casos particulares de estudio.

Características del sistema:

- ☞ Cuenta con 518 tablas.
- ☞ Maneja 10 interfaces con otros sistemas.

- ⌘ Ha estado en operación aproximadamente 10 años.
- ⌘ Se encuentra sobre un motor de Oracle ® 9i.

7.2. Mecanismo de detección por intuición humana

La estrategia para validar la detección de bad smells por la intuición humana se implementa a través de una encuesta, en la cuál se busca recoger las respuestas de los expertos a través de un grupo de inquietudes, de tal manera que a través de sus respuestas se puedan detectar consensos alrededor de los aspectos a través de los cuales se caracterizan los bad smells seleccionados.

La encuesta no busca ser la herramienta “formal” de validación, pero si busca una aproximación que permita tener un punto de referencia de la validez de la estrategia de detección planteada.

7.2.1 Descripción de la encuesta

Para introducir al experto en el tema de evaluación, la encuesta se estructura en dos partes:

- ⌘ Presentación, a través de la cual se expone el objetivo de la encuesta, a quién va dirigida y quién la dirige.
- ⌘ Indicaciones, a través de la cual se presenta el concepto de Martin Fowler acerca de los bad smells, una breve descripción de los bad smells a evaluar, aspectos importantes de los casos a analizar e indicaciones para diligenciar la encuesta.

Se colocan a consideración del experto dos casos, presentados a través de diagramas ER parciales del caso de estudio seleccionado.

Con el objeto de obtener un consenso de la presencia o no de los bad smells: tabla gruesa, tablas aisladas, llave primaria larga, restricción principal ausente, se realizan 11 preguntas a través de las cuales el experto debe indicar si se presentan o no las características suficientes para identificar un bad smell.

En el Anexo 6, se muestra la presentación de la encuesta y los textos completos.

7.2.2 Descripción de los casos de evaluación

Del caso de estudio seleccionado se obtuvieron dos diagramas ER parciales del modelo total, los casos pueden ser consultados en el Anexo 6.

7.3. Resultados Obtenidos

A continuación se presentan los resultados de las evaluaciones desarrolladas sobre el caso de estudio, se presentan los resultados obtenidos por intuición humana y los resultados reportados a través de la estrategia de detección automática.

Para el caso de los resultados por intuición humana, a continuación se relaciona la ficha de la encuesta.

Fecha de apertura	7 de julio de 2.006
Fecha de cierre	15 de julio de 2.006
Total Invitados	10
Total encuestas diligenciadas	4

Cabe anotar que:

- ☞ La encuesta fue realizada a través de invitación a expertos, por lo que las respuestas recibidas parten del hecho que están diligenciadas por personas con amplios conocimientos en el manejo de base de datos, ingeniería de software y/o manejo de proyectos de reingeniería, lo que por su grado de experiencia da confiabilidad a los resultados.

Los resultados se confrontan por porcentaje de respuestas que identifican la presencia de bad smell en de acuerdo a cada una de las preguntas.

Para cada planteamiento de la encuesta se presenta la pregunta, la intención, y los resultados obtenidos.

7.3.1 Caso – Tabla gruesa

☞ Pregunta:

Considera usted que en la tabla FNX_CLIENTES se evidencia (Caso1):

- Alto acoplamiento de la información relacionada con el cliente
- Se podrían presentar anomalías de actualización
- Opción 1 y 2
- Ninguna de las anteriores

☞ Intención:

Es posible la evidencia del bad smell tabla gruesa, con la presencia de tablas que:

: cuentan con un gran número de atributos, donde es probable el alto acoplamiento de la información

: Por causa de un diseño inadecuado, se evidencia la presencia de anomalías de actualización.

: cuentan con un gran número de atributos nulos.

☞ Resultados obtenidos

Resultado intuición humana	Resultado BadSmellsBDR		
75% Hay posible presencia	Resultado Ejecución		
	Nombre Elemento	Tipo	Causa BadSmell
	FNX_CLIENTES	TABLA	La tabla FNX_CLIENTES tiene 22 atributos y el máximo estimado es 20

7.3.2 Caso – Tabla gruesa

¿ Pregunta:

La definición de la tabla FNX_ORDENES_TRABAJO relaciona EQUIPO_ID, AREA_OPERATIVA_ID, SUBZONA_ID, AREA_TRABAJO_ID, RUTA_TRABAJO_ID, INFLUENCIA_ID como llaves foráneas. Considera usted que (Caso 2):

- Tiene un gran número de atributos que pueden ser organizados a través de nuevas relaciones
- Alto acoplamiento de la información relacionada con el ordenes de trabajo
- La información solicitada es la mínima requerida para crear ordenes de trabajo completas
- A través del tiempo, la tabla asegura la existencia de los datos necesarios para dar información de lo ocurrido con una orden de trabajo

¿ Intención:

Es posible la evidencia del bad smell tabla gruesa, con la presencia de tablas que:

: cuentan con un gran número de atributos, donde es probable el alto acoplamiento de la información

: Por causa de un diseño inadecuado, se evidencia la presencia de anomalías de actualización.

: cuentan con un gran número de atributos nulos.

¿ Resultados obtenidos

Resultado intuición humana	Resultado BadSmellsBDR		
	Resultado Ejecución		
	Nombre Elemento	Tipo	Causa BadSmell
100% Hay posible presencia	FNX_AQRS_TEMP	TABLA	La tabla FNX_AQRS_TEMP no tiene relación con otras tablas del esquema

7.3.3 Caso – Tabla aislada

¿ Pregunta:

Con base en la definición de las tablas FNX_AQRS y FNX_AQRS_TEMP de los casos 1 y 2, respectivamente. Considera usted que la tabla FNX_AQRS_TEMP aporta en el buen diseño de la base de datos (Caso 2)

- Falso
- Verdadero

¿ Intención:

Es posible la evidencia del bad smell tablas aisladas, con la presencia de tablas que:

: No tienen relaciones a través de la base de datos.

: Tablas que no aportan al modelo de datos.

: Duplicidad de información con otras tablas del modelo.

: aisladas, y que no aportan a tareas de auditoria

Resultados obtenidos

Resultado intuición humana	Resultado BadSmellsBDR		
100% Hay posible presencia	Resultado Ejecución		
	Nombre Elemento	Tipo	Causa BadSmell
	FNX_AQRS_TEMP	TABLA	La tabla FNX_AQRS_TEMP no tiene relación con otras tablas del esquema

7.3.4 Caso – Tabla aislada

Pregunta:

La tabla FNX_LOG_AUDITA_OPERACIONES apoya la administración de la base de datos (Caso 2):

- Falso
- Verdadero

Intención:

Es posible la evidencia del bad smell tablas aisladas, con la presencia de tablas que:

- : No tienen relaciones a través de la base de datos.
- : Tablas que no aportan al modelo de datos.
- : Duplicidad de información con otras tablas del modelo.
- : aisladas, y que no aportan a tareas de auditoria

Resultados obtenidos

Resultado intuición humana	Resultado BadSmellsBDR					
50% Hay posible presencia	Resultado Ejecución					
	<table><thead><tr><th>Nombre Elemento</th><th>Tipo</th><th>Causa BadSmell</th></tr></thead><tbody><tr><td>FNX_LOG_AUDITA_OPERACIONES</td><td>TABLA</td><td>La tabla FNX_LOG_AUDITA_OPERACIONES no tiene relación con otras tablas del esquema.</td></tr></tbody></table>	Nombre Elemento	Tipo	Causa BadSmell	FNX_LOG_AUDITA_OPERACIONES	TABLA
Nombre Elemento	Tipo	Causa BadSmell				
FNX_LOG_AUDITA_OPERACIONES	TABLA	La tabla FNX_LOG_AUDITA_OPERACIONES no tiene relación con otras tablas del esquema.				

7.3.5 Caso – Llave primaria larga

Pregunta:

Sobre cuáles de las siguientes tablas aplica el bad smell "llave primaria de gran tamaño":
 Seleccione las más representativas

- FNX_CONCEPTOS_ORDENES
- FNX_ORDENES TRABAJOS
- FNX_LOG_CONFIGURACION_IDENTIF
- FNX_EJECUTIVOS_CLIENTES
- FNX_LOG_EQUIPOS

Intención:

Es posible la evidencia del bad smell llave primaria larga, con la presencia de tablas que:
 : tienen asociada una llave primaria compuesta por un alto número de atributos del total que compone la tabla
 : Tablas transaccionales que manejan un alto número de accesos, y por su llave primaria la consulta se torna lenta en cuanto al tiempo de respuesta

Resultados obtenidos

Resultado intuición humana	Resultado BadSmellsBDR		
	Resultado Ejecución		
	Nombre Elemento	Tipo	Causa BadSmell
FNX_CONCEPTOS_ORDENES (50%)	COOR_PK	LLAVE PRIMARIA	La tabla FNX_CONCEPTOS_ORDENES tiene asociada la llave primariaCOOR_PK con 11 atributos y el máximo estimado es 5
FNX_ORDENES TRABAJOS (50%)	LGCI_PK	LLAVE PRIMARIA	La tabla FNX_LOG_CONFIGURACION_IDENTIF tiene asociada la llave primariaLGCI_PK con 7 atributos y el máximo estimado es 5
FNX_LOG_CONFIGURACION_IDENTIF (50%)	LGEQ_PK	LLAVE PRIMARIA	La tabla FNX_LOG_EQUIPOS tiene asociada la llave primariaLGEQ_PK con 7 atributos y el máximo estimado es 5
FNX_EJECUTIVOS_CLIENTES (0%)			
FNX_LOG_EQUIPOS (50%)			

7.3.6 Caso – Llave primaria larga

Pregunta:

Sobre cuál de las siguientes tablas considera usted que la llave primaria definida impacta el desempeño de las consultas realizadas a las bases de datos: Seleccione las más representativas

- FNX_CONCEPTOS_ORDENES
- FNX_ORDENES TRABAJOS
- FNX_LOG_CONFIGURACION_IDENTIF
- FNX_EJECUTIVOS_CLIENTES
- FNX_LOG_EQUIPOS

Intención:

Es posible la evidencia del bad smell llave primaria larga, con la presencia de tablas que:
 : tienen asociada una llave primaria compuesta por un alto número de atributos del total que compone la tabla
 : Tablas transaccionales que manejan un alto número de accesos, y por su llave primaria la consulta se torna lenta en cuanto al tiempo de respuesta

Resultados obtenidos

Resultado intuición humana	Resultado BadSmellsBDR		
	Resultado Ejecución		
	Nombre Elemento	Tipo	Causa BadSmell
FNX_CONCEPTOS_ORDENES (75%)	COOR_PK	LLAVE PRIMARIA	La tabla FNX_CONCEPTOS_ORDENES tiene asociada la llave primariaCOOR_PK con 11 atributos y el máximo estimado es 5
FNX_ORDENES TRABAJOS (25%)	LGCI_PK	LLAVE PRIMARIA	La tabla FNX_LOG_CONFIGURACION_IDENTIF tiene asociada la llave primariaLGCI_PK con 7 atributos y el máximo estimado es 5
FNX_LOG_CONFIGURACION_IDENTIF (50%)	LGEQ_PK	LLAVE PRIMARIA	La tabla FNX_LOG_EQUIPOS tiene asociada la llave primariaLGEQ_PK con 7 atributos y el máximo estimado es 5
FNX_EJECUTIVOS_CLIENTES (0%)			
FNX_LOG_EQUIPOS (50%)			

7.3.7 Caso – Llave primaria larga

Pregunta:

Para una tabla transaccional como FNX_LOG_CONFIGURACION_IDENTIF, considera usted que es más eficiente (Caso 1):

- La llave primaria definida
- Relacionar en una nueva tabla los valores de la llave primaria y establecer una integridad referencial con FNX_LOG_CONFIGURACION_IDENTIF, a través de un secuencial asignado en la nueva tabla

Intención:

Es posible la evidencia del bad smell llave primaria larga, con la presencia de tablas que:
: tienen asociada una llave primaria compuesta por un alto número de atributos del total que compone la tabla

: Tablas transaccionales que manejan un alto número de accesos, y por su llave primaria la consulta se torna lenta en cuanto al tiempo de respuesta

Resultados obtenidos

Resultado intuición humana	Resultado BadSmellsBDR		
75% Hay posible presencia	Resultado Ejecución		
	Nombre Elemento	Tipo	Causa BadSmell
	LGCI_PK	LLAVE PRIMARIA	La tabla FNX_LOG_CONFIGURACION_IDENTIF tiene asociada la llave primaria LGCI_PK con 7 atributos y el máximo estimado es 5

7.3.8 Caso – Detección Bad Smells

Pregunta:

Al analizar la tabla FNX_SOLICITUDES, cuáles bad smells puede identificar:

- Tabla gruesa
- Tabla aislada
- Llave primaria de gran tamaño
- Ausencia de restricciones principales

Intención:

Dada la definición proporcionada para cada uno de los bad smells, buscar un consenso entre los expertos para identificar aquellos que aplican a la tabla.

Resultados obtenidos

Resultado intuición humana	Resultado BadSmellsBDR		
Tabla gruesa (75%)	Resultado Ejecución		
	Nombre Elemento	Tipo	Causa BadSmell
	FNX_SOLICITUDES	TABLA	La tabla FNX_SOLICITUDES tiene 33 atributos y el máximo estimado es 20
Ausencia de restricciones principales (75%)	No reportado		

7.3.9 Caso – Detección Bad Smells

Pregunta:

Al analizar la tabla FNX_INF_INALAMBRICAS_ANU, cuáles bad smells puede identificar:

- Tabla gruesa
- Tabla aislada
- Llave primaria de gran tamaño
- Ausencia de restricciones principales

Intención:

Dada la definición proporcionada para cada uno de los bad smells, buscar un consenso entre los expertos para identificar aquellos que aplican a la tabla.

Resultados obtenidos

Resultado intuición humana	Resultado BadSmellsBDR											
Tabla gruesa (75%)	<table><tr><th colspan="3">Resultado Ejecución</th></tr><tr><th>Nombre Elemento</th><th>Tipo</th><th>Causa BadSmell</th></tr><tr><td>FNX_INF_INALAMBRICAS_ANU</td><td>TABLA</td><td>La tabla FNX_INF_INALAMBRICAS_ANU tiene 25 atributos y el máximo estimado es 20</td></tr></table>			Resultado Ejecución			Nombre Elemento	Tipo	Causa BadSmell	FNX_INF_INALAMBRICAS_ANU	TABLA	La tabla FNX_INF_INALAMBRICAS_ANU tiene 25 atributos y el máximo estimado es 20
Resultado Ejecución												
Nombre Elemento	Tipo	Causa BadSmell										
FNX_INF_INALAMBRICAS_ANU	TABLA	La tabla FNX_INF_INALAMBRICAS_ANU tiene 25 atributos y el máximo estimado es 20										
Ausencia de restricciones principales (100%)	<table><tr><th colspan="3">Resultado Ejecución</th></tr><tr><th>Nombre Elemento</th><th>Tipo</th><th>Causa BadSmell</th></tr><tr><td>FNX_INF_INALAMBRICAS_ANU</td><td>TABLA</td><td>La tabla FNX_INF_INALAMBRICAS_ANU no tiene asociada una llave primaria</td></tr></table>			Resultado Ejecución			Nombre Elemento	Tipo	Causa BadSmell	FNX_INF_INALAMBRICAS_ANU	TABLA	La tabla FNX_INF_INALAMBRICAS_ANU no tiene asociada una llave primaria
Resultado Ejecución												
Nombre Elemento	Tipo	Causa BadSmell										
FNX_INF_INALAMBRICAS_ANU	TABLA	La tabla FNX_INF_INALAMBRICAS_ANU no tiene asociada una llave primaria										
Tabla aislada (50%)	No reportado											

7.3.10 Caso – Detección Bad Smells

Pregunta:

Al analizar la tabla FNX_REQUERIMIENTOS_TRABAJOS, cuáles bad smells puede identificar:

- Tabla gruesa
- Tabla aislada
- Llave primaria de gran tamaño
- Ausencia de restricciones principales

Intención:

Dada la definición proporcionada para cada uno de los bad smells, buscar un consenso entre los expertos para identificar aquellos que aplican a la tabla.

Resultados obtenidos

Resultado intuición humana	Resultado BadSmellsBDR		
Tabla gruesa (75%)	Resultado Ejecución		
	Nombre Elemento	Tipo	Causa BadSmell
	FNX_REQUERIMIENTOS	TABLA	La tabla FNX_REQUERIMIENTOS tiene 21 atributos y el máximo estimado es 20
Ausencia de restricciones principales (50%)	No reportado		

Para observar los resultados de la encuesta, ver Anexo 6.

7.4. Resultados Obtenidos

Al analizar las pruebas efectuadas a través de la encuesta, se observa, que hay cierto grado de correspondencia entre los resultados obtenidos a través de la intuición humana y los resultados automáticos obtenidos a través de "BadSmellsBDR".

- 3 Para el bad smell "tabla gruesa", al evaluar la coincidencia de las preguntas de acuerdo a la intuición humana vs. la herramienta, presenta el 80% de correspondencia de resultados similares y en todos los casos con una alta probabilidad el bad smell fue identificado.

Sin embargo, resulta interesante el hecho que, al preguntar en la encuesta si es posible, a través de un modelo matemático, determinar el número máximo de atributos que debe tener una tabla en el diseño relacional de la base de datos en la cual está inmersa, la respuesta esta dividida a favor y en contra por partes iguales.

- 3 Para el bad smell "tabla aislada", al evaluar la coincidencia de las preguntas de acuerdo a la intuición humana vs. la herramienta, se presenta el siguiente comportamiento:
 - Al realizar la evaluación sobre una tabla de auditoria, no resulta claro determinar si una tabla con ésta responsabilidad debe o no estar vinculada a través de relaciones en el modelo. Esto se evidencia en el hecho que no hay un consenso si es correcto o no, que la tabla se encuentre aislada. Por lo que el porcentaje de probabilidad se encuentra dividido por partes iguales entre los encuestados.
 - En casos diferentes a responsabilidades de auditoria el consenso es inminente con un 100% de identificación del bad smells en los casos correspondientes.

- 3 Para el bad smell "llave primaria larga", al hacer que se identifique sobre un conjunto de tablas el bad smell existe consenso ante la ausencia del mismo, en el caso en que la tabla actúa como intermedia para partir una relación m:m, y la tabla no maneja información adicional.

Sin embargo, para aclarar la identificación del bad smell se realizó dos preguntas que indagan lo mismo pero de diferente forma, se presenta los siguientes casos sobre éstas tablas mismas:

- Se seleccionaron tablas donde la llave primaria se compone de más de 5 atributos, para estas tablas al solicitar se identifique el bad smells, no existe un consenso claro.
 - Al plantear las mismas tablas, que el punto anterior, y se solicita se identifique las tablas donde la llave primaria impacta el desempeño de las consultas realizadas a la base de datos, se identifica que: sobre una tabla con la llave primaria compuesta por 11 atributos el bad smell tiene un consenso de 75%, sin embargo para tablas con 5 a 7 atributos en su llave primaria, hay un consenso positivo aunque no rotundo de 54%.
- 3 Para el bad smell "ausencia de restricciones principales", al evaluar la coincidencia de las preguntas de acuerdo a la intuición humana vs. la herramienta, presenta el 100% de correspondencia de resultados.

8. CONCLUSIONES

Este documento describió un proyecto de investigación dirigido al mantenimiento de bases de datos relacionales. Con base en el estudio de sanas prácticas de diseño de bases de datos relacionales, se analizaron los elementos que facilitarían la identificación de malas prácticas en el diseño. Como resultado se propone un conjunto de 29 bad smells, proporcionando una estrategia formal, que permite fortalecer el análisis de éste esquema de persistencia en proyectos de reingeniería.

El definir para cada bad smell: la descripción formal, los síntomas para detectarlo, qué hacer para su corrección, problemas que genera su presencia y ventajas de su solución; hace de la propuesta no solo un marco de referencia, también provee los elementos necesarios, para iniciar el proceso de detección y determinar de una forma preliminar los refactoring necesarios a aplicar.

El trabajo se fortalece a través de una propuesta taxonómica, bajo la cual se organizan los bad smells por categorías relacionadas, lo que permite una mayor comprensión del alcance de cada uno, las relaciones entre ellos y facilita, de acuerdo al interés con que se inicia el proceso de refactoring, la detección de malas prácticas de diseño por aspectos: arquitectónicos, de integridad o de calidad.

Además de la propuesta taxonómica también se desarrolló una herramienta, llamada BadSmellsBDR, para hacer una prueba de concepto de la propuesta. Ésta herramienta cumple con dos propósitos fundamentales: el primero, demostrar que los bad smells propuestos son susceptibles de ser detectados automáticamente. el segundo, proporcionar un framework construido con la flexibilidad necesaria para soportar trabajos posteriores de automatización, que permita adicionar con relativa facilidad mecanismos de detección de bad smells no solo para bases de datos relacionales, sino también para otros tipos de programación, como Java.

Los resultados de la encuesta, aunque no se constituyen en una herramienta de validación formal, indica que los bad smells y su detección automática es viable.

Por último, la recopilación de información referente a: reingeniería, refactoring y las mejores prácticas en el diseño de bases de datos relacionales, en un solo marco teórico se constituye en un punto de referencia para consulta o trabajos futuros, lo que permite iniciar investigaciones desde una base sólida de información, con el objeto de facilitar la profundización de cualquier propuesta.

9. TRABAJO FUTURO

Este trabajo es la base para iniciar trabajos interesantes, que apoyen el problema de detección en el refactoring de bases de datos relacionales. Teniendo como precedente que el área de trabajo de esta propuesta no había sido abordada con la suficiente formalidad, son varias las direcciones que pueden tomar trabajos futuros, con el objeto de complementar y mejorar éste.

A continuación se mencionan trabajos que pueden fortalecer la presente propuesta:

- 3 Complementar los bad smells propuestos, formalizando los ítems de: síntomas y qué hacer, a través de uno o varios modelos formales (Ej. modelos matemáticos).
- 3 El bad smell “dependencias funcionales”, proporciona el cierre de F+ [SILB02], y éste a su vez complementa el “qué hacer” de varios de los bad smells definidos. La labor de identificar automáticamente éste bad smell es compleja, sin embargo, el insumo que proporciona para apoyar la detección de algunos bad smells es importante.
- 3 Fortalecer el proceso de detección de los bad smells propuestos, verificando sus características: estructurales, de integridad, calidad, relacionales, y sus responsabilidades; implementando modelos de detección formales (métricas de software, estudio de grafos, axiomas, predicados, algoritmos, entre otros.) que permitan establecer umbrales, donde los resultados indiquen la posible presencia del bad smell analizado.
- 3 Integrar los bad smells definidos, a la herramienta “BadSmellsBDR”, implementando las clases y tareas necesarias para que la detección sea guiada por el modelo formal definido para cada bad smell.
- 3 Integrar análisis dinámicos a los procesos de detección, puede hacer más efectivo el proceso de detección de los bad smells. Pues la validación de esta propuesta, solo contempla análisis estáticos.
- 3 Se consideró el proceso de detección de cada bad smell como una preocupación individual, sin embargo, al plantear la definición de algunos bad smells nacían dependencias, como el caso de “tabla gruesa” >> “tabla con alta responsabilidad”. Es posible, como en el caso anterior, que se pueda establecer un orden de ejecución para su detección, con el objeto de identificar su presencia con mayor efectividad o se pueda establecer ante la presencia de un bad smell, la probabilidad de ocurrencia de otro.
- 3 No es conveniente mantener la tarea de detección aislada de las actividades que conforman el proceso de refactoring, por lo que es interesante establecer una línea de continuidad, de tal forma que una vez se tienen identificados los bad smells, con el apoyo de la propuesta de Ambler [AMBL02], se determine que refactorings se debe aplicar a un bad smell identificado.

BIBLIOGRAFÍA

- [MFOW99] FOWLER, Martin. Refactoring: Improving the Design of Existing Code. Addison-Wesley, 1.999.
- [BFLO05] FLORIAN, Beatriz Eugencia. Detección de Bad Smells en Java utilizando métricas de software. Universidad de los Andes, 2.005.
- [ARNO93] ARNOLD, Robert & HOHNER Software Reengineering, .IEEE Computer Society Press. International Conference on Software Maintenance 1993, USA
- [WSEIG] <http://www.sei.cmu.edu/str/indexes/glossary/reengineering.html>. Software Engineering Institute. Carnegie Mellon. Fecha Consulta: noviembre 2005.
- [CHIK92] CHIKOFSKY, Elliot. Reverse Engineering and Design Recovery: A Taxonomy. IEEE Computer Society Press. 1992.
- [ARNO96] ARNOLD, Robert. Software Change Impact Analysis, IEEE International Conference on Software Maintenance 1996, USA
- [KAZM99] KAZMAN, Rick, CARRERE, S. Jeromy. Playing Detective: Reconstructurin Software Architecture from Available Evidence. Journal of automated Software Engineering. Pág. 100 - 140. 1999.
- [KAZM98] KAZMAN, Rick, CARRERE, S. Jeromy y WOODS, S.G. Requeriments for integrating Software Architecture and Reengineering Models: CORUMII. WCRE 98. 1998
- [PSEI95] SEI, Software Engineering Institute. Carnegie Mellon University. Perspectives on Legacy system Reengineering. 1995
- [BERG01] BERGEY, john, O'BRIEN, Liam, SMITH, Dennis. SEI, Software Engineering Institute. Carnegie Mellon University. Options Analysis for Reengineering (OAR): A Method for Mining Legacy Assets. 2001
- [MENS04] MENS, Tom, TOURWE, tom. A survey on Software refactoring. Transactions on Software Engineering. IEEE computer Society Press. Vol 30 N. 2, Febrero 2004
- [AIEM92] ANSI/IEEE P1219/D16. Support For Software Maintenance.Standard. 1992.
- [ALEX77] CRHISTOPHER, Alexander, Sara Ishikawa, Murray Silverstein, otros. A pattern language. Oxford University Press. New York. 1977.
- [GAMM94] GAMMA, Erich. HELM, Richard. JOHNSON, Ralph. Design Patterns. Addison Wesley. 1994
- [WAKE00] WAKE, William C. Refactoring Workbook. Addison Wesley. 2000.
- [MANT03] MANTYLA, Mika. Bad Smells in Software – a taxonomy and an emprirical study. 2.003.
- [PSEI88] SEI, Software Metrics. Carnegie Mellon University. 1988
- [BATI92] C. Batini, S. Ceri and S. Navathe. Conceptual Database Design : an Entity-Relationship Approach. Benjamin Cummings, 1992.

- [CHIA94] R.H.L Chiang, T-M Barron and V.C Storey. Reverse Engineering of Relational Databases: Extraction of an EER Model from a Relational Database. Data and Knowledge Engineering, 12, pp. 107-142 (1994).
- [AGSD03] Rakesh Agarwal, Ajit Sarangi and Swati Das. Reengineering of database Intensive application. Publicación avalada por Infosys Technologies Limited, India. 2003.
- [AMBL02] Scott W. Ambler, Pramodkumar J. Sadalage. Refactoring Databases Evolutionary Database Design. Addison Wesley. Febrero 2002.
- [BARB00] Aziz Barbar. A User Driven Method for Database Reverse Engineering. I3S Laboratory, University of Nice. 2000.
- [HAIN93] J-L. Hainaut, C. Tonneau, M. Joris, M. Chandelon. SCHEMA TRANSFORMATION TECHNIQUES FOR DATABASE REVERSE ENGINEERING. Institut d'Informatique, University of Namur, 1993.
- [ANDE94] Andersson, Martin. Extracting an entity relationship schema from a relational database through reverse engineering. Database laboratory, Department of Computer Science Swiss Federal Institute of Technology.1994.
- [SILB02] Abraham Silberschats, Henry F. Korth, S. Sudarshan. Fundamentos de bases de datos. Mac Graw Hill. España 2002
- [TMFM88] Consultado, Julio de 2.006. <http://www.tmforum.org>
- [CANG05] ANGARITA, Carlos. Detección de bad smells en aplicaciones java a partir de una representación XML del código fuente, Universidad de los Andes, 2.005.

ANEXOS

Anexo 1 Proceso de reingeniería en sistemas legados

El SEI realizó una propuesta del framework para llevar a cabo el proceso de reingeniería completo [PSEI95] para sistemas legados, sin embargo, a continuación se hace una referencia general al marco, puesto que proporciona fundamentos para comprender que no solo el proceso de reingeniería se debe llevar como una actividad, sino por el contrario se debe desarrollar como un proyecto pues su alcance no solo llega a sistemas legados, en cambio cubre cualquier tipo de proceso de reingeniería.

El framework descrito es una referencia para la planeación de proyectos de reingeniería, para identificar, describir, entender, evaluar, dirigir y caracterizar factores de reingeniería relacionados con los proyectos.

A continuación se observa la Figura 30, donde se describe el entorno y los elementos a tener en cuenta en el proceso de reingeniería.

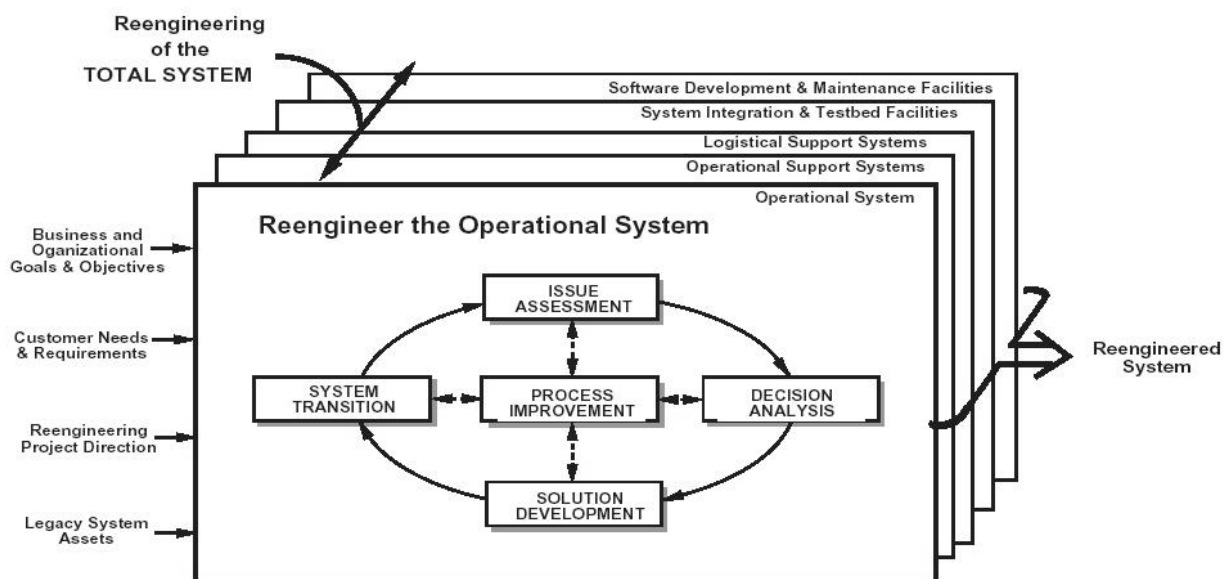


Figura 30 Descripción del proceso completo de reingeniería.

Existen 5 tipos genéricos de elementos que deben ser cuidadosamente considerados cuando se evalúa el inicio de un proceso de reingeniería:

- 3 *Sistema operativo*: Se debe evaluar operacionalmente el sistema que es típicamente el foco de los esfuerzos en reingeniería y compromete el core de la funcionalidad del sistema que se encuentra en producción.
- 3 *Operación de los sistemas soportados*: El sistema puede tener interacción con otros sistemas organizacionales y sin embargo estos otros sistemas no son físicamente parte del sistema al cual se le está realizando reingeniería.
- 3 *Sistemas de soporte logístico*: Sistemas offline que soportan dispositivos y elementos logísticos de la operación de la organización.

- 3 Integración con sistemas y facilidad de pruebas: Se debe tener presente que sistemas deben ser integrados, sean nuevos o modificados y las pruebas con el software que experimento la reingeniería.
- 3 Facilidades de evolución del software: Si el sistema requiere reingeniería de software, es manejable su adaptación con sistemas legacy y permite el desarrollo de nuevos componentes para su integración.

Fases del proceso de Reingeniería

Con base en los elementos anteriormente detectados, el proyecto de reingeniería inicia un proceso de 5 fases que debe ser igualmente aplicado a cada uno de los elementos identificados, para determinar si son o no afectados durante el proceso de reingeniería. Figura 31.

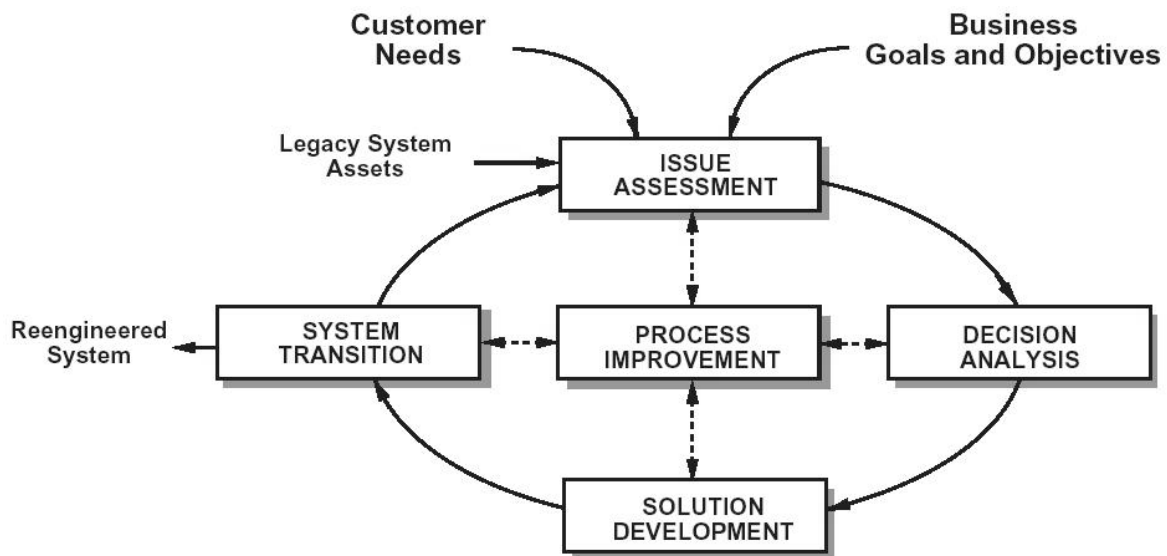


Figura 31 Fases a evaluar al iniciar un proyecto de reingeniería

En un proyecto en particular de reingeniería se debe especificar un conjunto de tareas a realizar en un Work Brakdown structure (WBS) o en un documento de planeación. Existen dos tipos genéricos de estos documentos (1) un plan de valoración (2) Plan del proyecto de reingeniería.

Realizar una valoración.

Actividad inicial en la cual se establece el alcance y esfuerzo necesario en el proceso de reingeniería, las actividades están centradas en desarrollar un plan preliminar como guía para dirigir las fases iniciales de la reingeniería Figura 32.

El objetivo, es obtener rápidamente un nivel de entendimiento de la arquitectura de los sistemas, el diseño, características operacionales y funcionalidad.

Entregables:

- ⌘ Un verdadero alcance del esfuerzo que se requiere en la reingeniería.
- ⌘ Metas y objetivos explícitos de la reingeniería
- ⌘ Descripción cuantitativa y cualitativa de los sistemas legados (si aplica).
- ⌘ La viabilidad de realizar reingeniería a los sistemas legados (si aplica).
- ⌘ Soluciones alternativas de reingeniería.

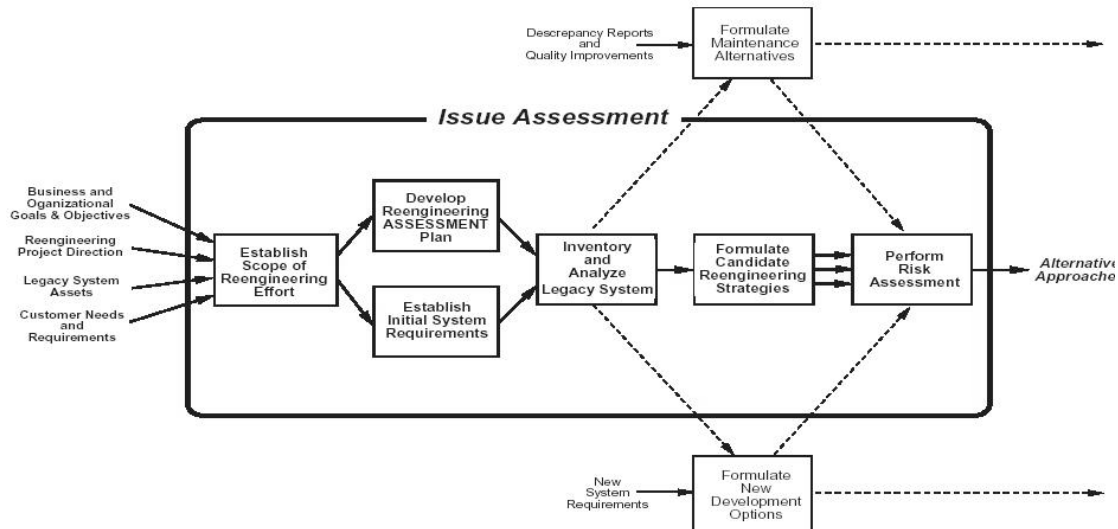


Figura 32 Actividades de la etapa de valoración

Analizar la decisión

Para tomar una adecuada decisión es necesario contar con todos los componentes técnicos, económicos, programáticos y políticos de la organización que pueden influenciar y afectar el proceso de reingeniería. Una parte integral de la toma de decisión es realizar un análisis costo / beneficio / riesgo y evaluar otras estrategias de reingeniería frente a nuevos desarrollos o alternativas de mantenimiento. Figura 33

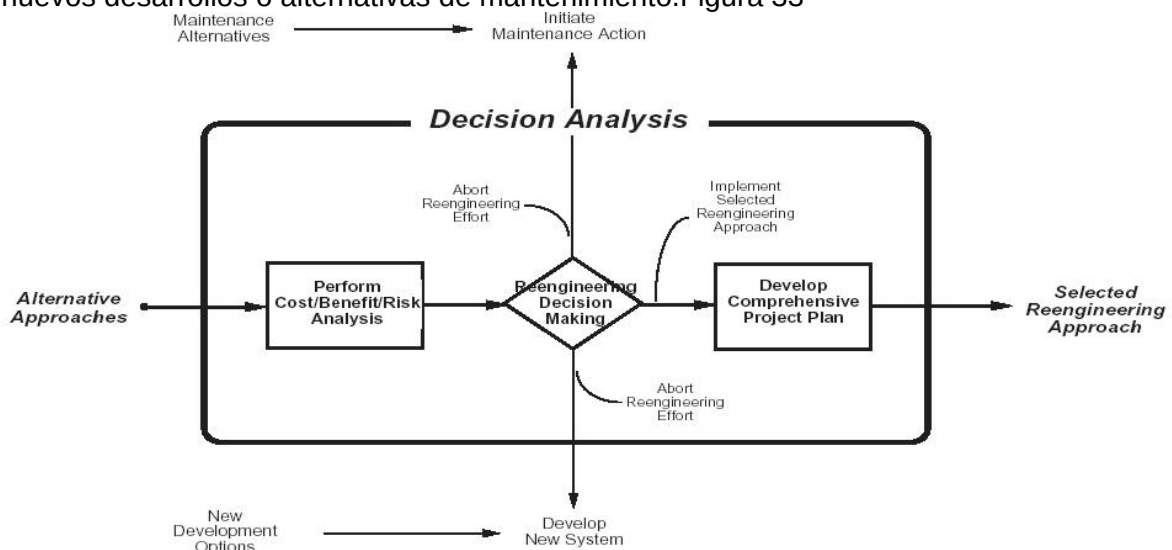


Figura 33 Etapas en el análisis de la decisión.

Si luego del proceso del análisis de la decisión, la reingeniería es viable la fase final es preparar un plan adecuado para todo el proyecto de reingeniería para su aprobación e implementación.

El plan cubre:

- ⌘ Tareas a realizar
- ⌘ Establecer un cronograma para las tareas.
- ⌘ Evaluar los riesgos.
- ⌘ Establecer los recursos requeridos.
- ⌘ Establecer entregables
- ⌘ Establecer los productos a liberar.

Entregables:

- ⌘ Comprensión y análisis de los datos realizado para el proceso de la toma de decisiones
- ⌘ Una estrategia de reingeniería validada y aprobada y bases técnicas.
- ⌘ La proyección del impacto del proceso de reingeniería.
- ⌘ Estimación de costos, recursos, tiempo y riesgos que envuelve el proceso.
- ⌘ Beneficios proveídos luego del proceso de reingeniería.
- ⌘ Plan de reingeniería.

Desarrollar una solución

Esta fase constituye el core de todo el esfuerzo de reingeniería. La actividad inicial de esta fase consiste en obtener y validar en detalle cada uno de los requerimientos del sistema sobre el cual se realizará la reingeniería. Figura 34

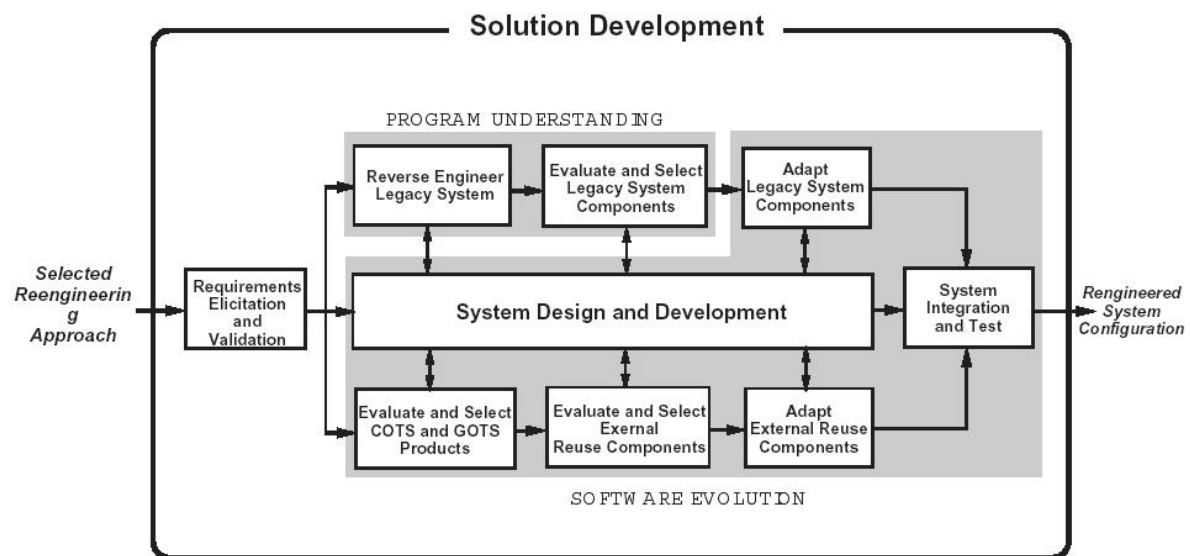


Figura 34 Etapas en el desarrollo de la solución propuesta en reingeniería

El proceso de ingeniería en reversa, se enfoca a entender el sistema existente y obtener una comprensión total del mismo con la ayuda de herramientas de software.

Entregables:

- ↯ Un sistema con el proceso de reingeniería completo y los requerimientos cubiertos.
- ↯ Sistema probado y demostrado.
- ↯ Un sistema con costos, tiempos y manejo de riesgos de acuerdo a lo planeado.

Evaluar la transición de los sistemas

La fase se enfoca a la liberación del sistema dentro de su ambiente de producción. Es una fase clave para el éxito del proyecto de reingeniería. Se preocupa por evaluar y preparar la instalación del sistema con base en listas de chequeo, entrenar apropiadamente al usuario y preparar la infraestructura de la organización para recibir el sistema.

El trabajo de reingeniería se entiende como completo cuando el sistema esta totalmente implantado y se acepta que es capaz de reemplazar el sistema legado en su totalidad. Figura 35

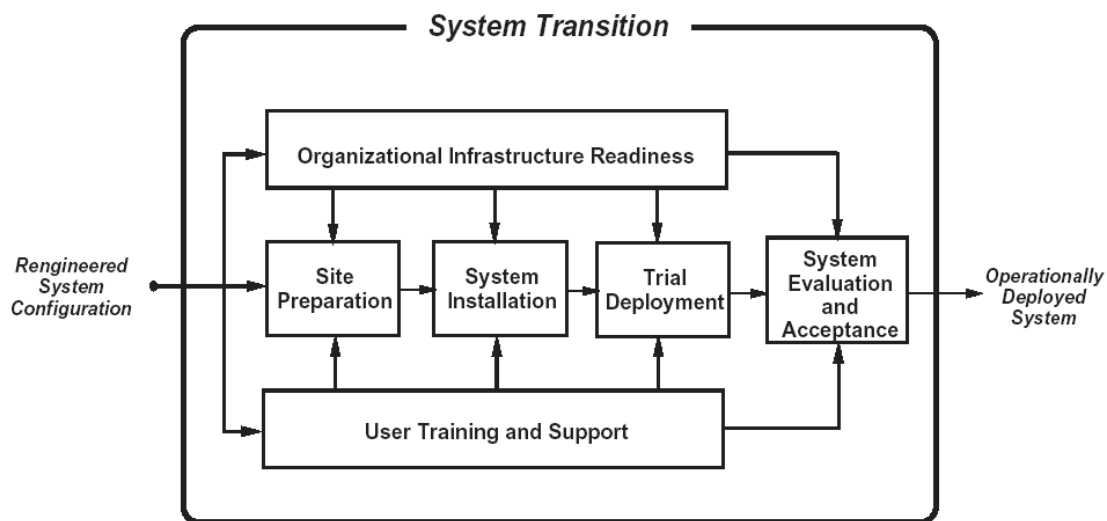


Figura 35 Actividades en la etapa de transición

Entregables:

- ↯ Un sistema compatible con la totalidad de su entorno de operación.
- ↯ Sistema operacionalmente validado y aceptado por el usuario.

Proceso de mejora

Se planean nuevas versiones del sistema. Como un punto de unión entre el proyecto que se culmina y un posible próximo proyecto. Esto permite visualizar la forma en que el sistema evolucionará. Figura 36

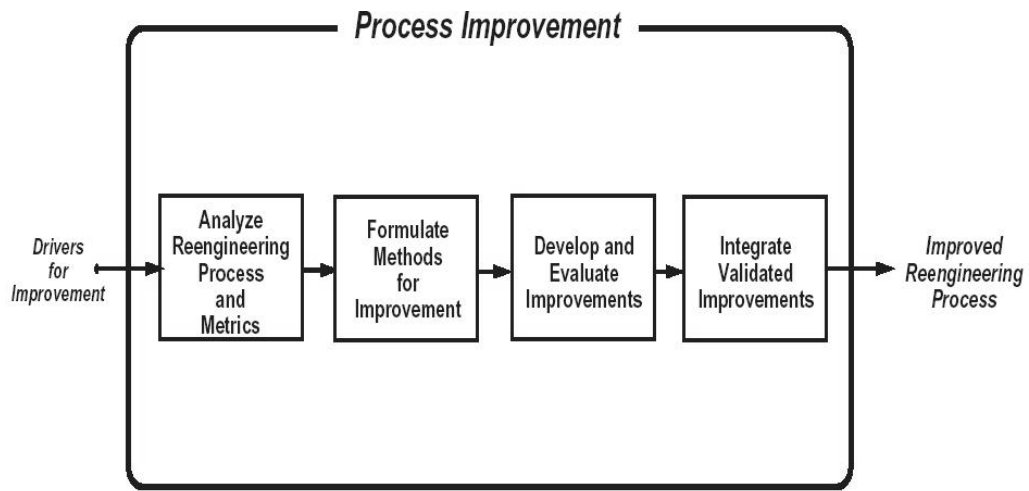


Figura 36 Actividades de la etapa de mejora.

Anexo 2 Propuesta de Bad Smells de Fowler

Fowler [MFOW99] presenta 22 bad smells para programación orientada a objetos, como se muestra a continuación.

Bad Smell	Descripción
Duplicated code	Estructuras que se encuentran duplicadas en la aplicación. Tanto sintácticas como semánticas.
Long Method	Métodos largos en términos de líneas de código por lo cual su responsabilidad es alta.
Large Class	Clases con muchos atributos y métodos, como consecuencia la responsabilidad de la clase es alta.
Long Parameter List	Métodos con una gran cantidad de parámetros.
Divergent change	Clases que deben ser modificadas permanentemente para adaptarse ante cambios de requerimientos no funcionales
Shotgun surgery	Para cambios pequeños en una clase se deben modificar otras clases que igualmente requieren cambios pequeños para manejar el cambio.
Feature Envy	Métodos que tienen más relación con una clase que con la clase en la cual se encuentra localizado.
Data Clumps	Conjunto de datos que por lo general se deben manejar juntos y se encuentran distribuidos, pueden convertirse en una nueva clase.
Primitive obsession	En ocasiones se utiliza con bastante frecuencia primitivas en vez de crear clases que aunque pequeñas agrupan un conjunto de datos primitivos.
Switch Statements	En lugar de hacer uso de polimorfismo, se utiliza con frecuencia expresiones condicionales.
Parallel inheritance hierarchies	Existen jerarquías paralelas de clases y éstas pueden ser extendidas. Cuando se realiza una subclase de una también se debe hacer una subclase por las otras.
Lazy Class	Clases que no tienen responsabilidades reales en la ejecución del sistema y pueden ser removidas.
Speculative Generality	Código innecesario que ha sido creado para soportar cambios futuros en la aplicación.
Temporary Field	Variables de instancia que son utilizadas solo en ciertos casos.
Message Chains	Clases que se encuentran altamente acopladas, por lo cual una clase siempre esta haciendo uso de otras.
Middle Man	Clases que delegan gran parte de su responsabilidad a otras clases.
Inappropriate intimacy	Clases que se encuentran altamente acopladas
Alternative classes with different interfaces	Clases o métodos que tienen diferentes firmas pero tienen la misma responsabilidad.
Incomplete library class	Librerías que tienen parte del trabajo que deberían tener asociado, por lo cual el uso posterior de la librería no es

	posible.
Data Class	Clases sin responsabilidades, contiene muchos datos pero manejan poca lógica sobre los mismos.
Refused Bequest	Subclases que no realizan el uso de las propiedades y atributos que heredan. O que no soportan todos los métodos y datos heredados.
Comments	No identifican bad smells pero pueden ayudar a identificar estructuras de código pobre que requieren de comentarios.

Anexo 3 Diseño de la aplicación BadSmellsBDR.

Para apoyar la descripción del diseño, se utiliza el diagrama de paquetes como herramienta que permite identificar como se encuentra organizado BadSmellsBDR y la dependencia de cada uno de sus paquetes.

Como parte de la organización principal de paquetes a continuación se presenta el diagrama que estructura a nivel macro la aplicación.

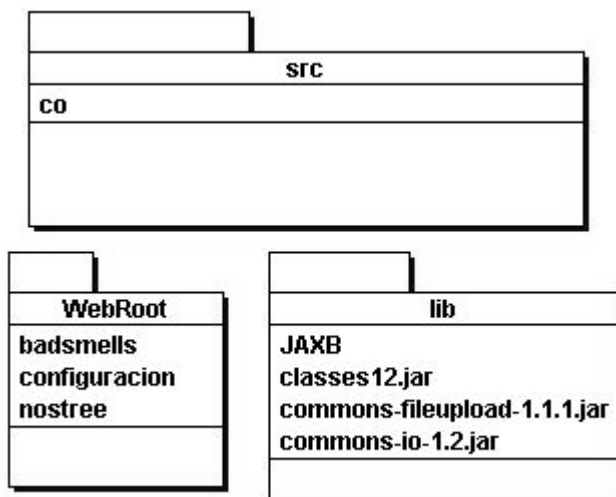


Figura 37 Paquetes marco de BadSmellsBDR

Nombre del paquete	Propósito
<i>src.co.edu.uniandes.badsmells</i>	Paquete en el que se encuentra la arquitectura de la aplicación.
<i>WebRoot</i>	Paquete que mantiene los jsp necesarios para manejar la interfaz gráfica con el usuario. En badsmells se encuentra las páginas .jsp requeridas para parametrizar y dar respuesta al análisis de cada uno de los badsmells. En configuración se encuentra las páginas .jsp requeridas para parametrizar y cargar la configuración de base de datos. En nostree se mantiene la definición del árbol de opciones para utilizar la aplicación. Las páginas .jsp están vinculadas al archivo config.css lo que permite modificar la interfaz gráfica fácilmente.
<i>lib</i>	Mantiene la definición de las librerías particulares necesarias para la operación de la aplicación.

El paquete *src.co.edu.uniandes.badsmells*, como ya se mencionó mantiene la arquitectura core de la aplicación por lo cual se describe a continuación.

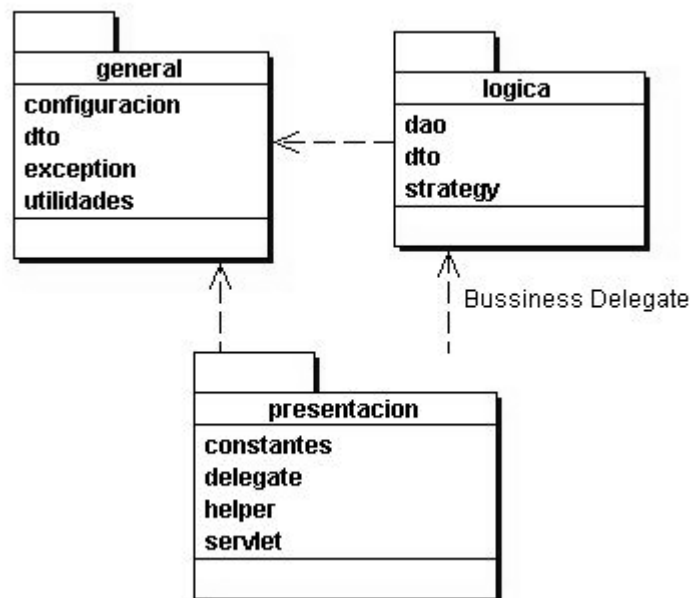


Figura 38 Estructura del paquete src.co.edu.uniandes.badsmells de BadSmellsBDR

Nombre del paquete	Propósito
General	Proporciona las clases necesarias para que la aplicación maneje sus propias excepciones, utilidades de log, lectura de archivos XML y validación del mismo a través de dto.
Logica	Mantiene las clases que se constituyen como punto de acceso para acceder a las funcionalidades propias del modelo de la aplicación. Y las clases del modelo.
Presentacion	En este paquete se encuentran centralizadas todas las clases que manejan la interfaz gráfica con el usuario y las clases controladoras que dan viabilidad a la comunicación entre el modelo y la vista.

Cada uno de los paquetes, cuenta con su propia estructura de la siguiente manera:

☞ Paquete general

- o Cuenta con, un paquete de manejo de excepciones que maneja de forma separada las excepciones de lógica y de interfaz gráfica.
- o El paquete de utilidades permite el registro de log de eventos.
- o Los paquetes configuración y dto, permiten carga de un archivo XML con la información necesaria para la conexión a la base de datos y su validación.

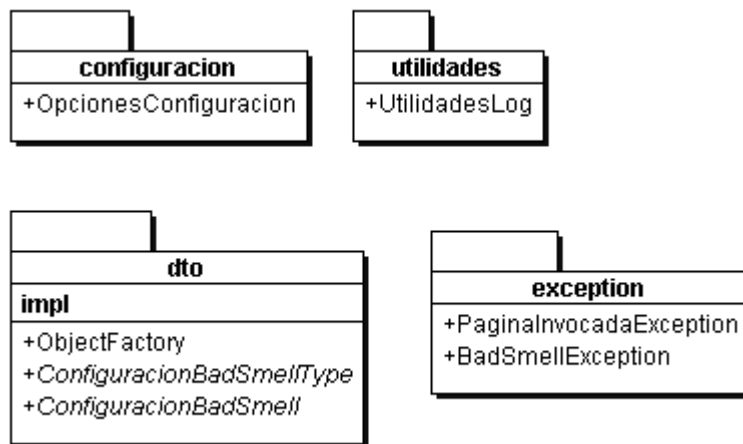


Figura 39 Estructura del paquete general de BadSmellsBDR

3 Paquete logica

- o El modelo se ve reflejado en los paquetes dto y dao. Por su parte dto, tiene la responsabilidad de definir de una forma uniforme la manera como se organiza la información luego del análisis de un bad smell. Dao, implementa el DAO, a través de ésta clase se puede extraer análisis interesantes de los bad smells que se desean identificar de la base de datos, solo se encarga de extraer información de la base de datos, sin embargo si se desea realizar análisis particulares sobre las consultas, se debe trabajar con las clases dispuestas en el paquete strategy.
- o El paquete strategy, permite determinar la manera como se interpretan los parámetros para realizar el análisis del bad smell seleccionado, ordena la ejecución del análisis y da respuesta de los bad smells que se identifican a través del DAO y se analizan a través de la aplicación.

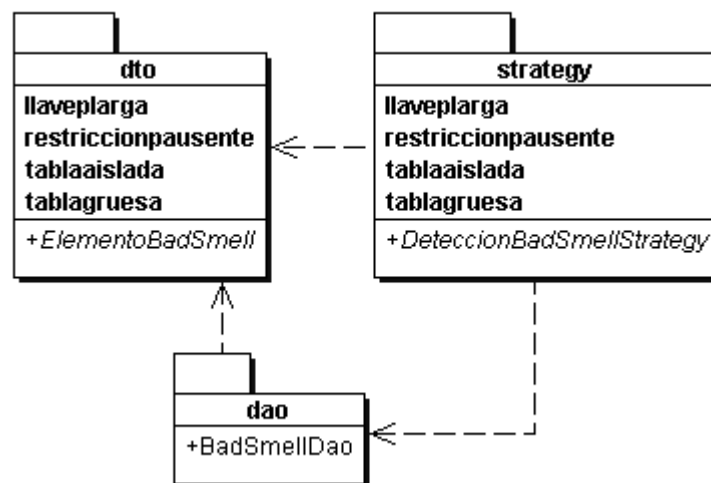
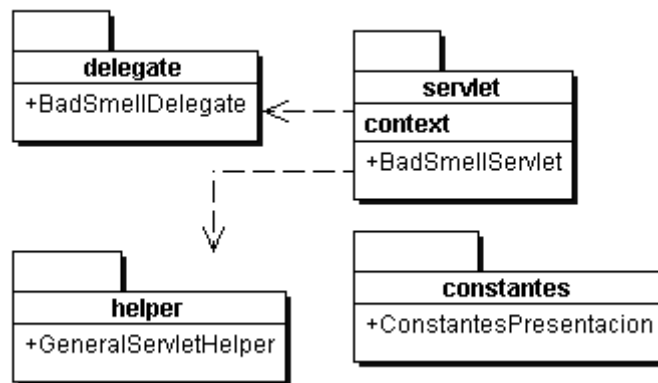


Figura 40 Estructura del paquete logica de BadSmellsBDR

⌘ Paquete presentación

- o El paquete constantes, permite definir los Bad Smells que la aplicación soporta en el menú.
- o La vista es soportada por el paquete servlet y helper, el cual se encarga de leer los parámetros proporcionados por los usuarios para la ejecución de los Bad Smells. Y realizar el llamado a las páginas correspondientes de acuerdo al Context Object's para la ejecución de los BadSmells específicos.
- o El controlador se distribuye entre los paquetes servlet.context y delegate, el primero define la estructura uniforme bajo la cual se deben crear cada uno de los BadSmellContextObj para la atención particular de cada uno de los bad smells que analiza la aplicación. Y el segundo, une la capa de presentación con la capa de lógica de negocio, recibe las peticiones de los Context Object's para realizar la ejecución de los BadSmells específicos haciendo un llamado a los Strategy correspondientes.



Técnicamente, se requiere montar la aplicación con las siguientes condiciones operativas:

- ⌘ Servidor Web con soporte para Servlets (Tomcat) o Servidor de aplicaciones con soporte para aplicaciones Web (JBoss, Sun Application Server) que soporten las siguientes especificaciones: o Servlet 2.3. o JSP 1.2
- ⌘ BadSmellsBDR, se implemento con j2sdk 1.4.2_09

Anexo 4 Requerimientos de la aplicación BadSmellsBDR

☞ La aplicación cuenta con los siguientes requerimientos funcionales:

N. req.	REQ_1		
Modulo	Configuración		
Nombre Requerimiento	Definir parámetros de configuración con la base de datos.		
Estado	Definido	Tipo	Funcional
Descripción	Permite establecer los parámetros de conexión necesarios para la conexión de la base de datos.		

Precondiciones/Entradas:

- El archivo de configuración debe corresponder al esquema descrito en el archivo configuracion.xsd
- Archivo de configuración .xml

Flujo de Eventos:

SISTEMA	USUARIO
	Selecciona la opción de “Configuración de la base de datos” y proporciona la ruta donde se encuentra el archivo requerido por el sistema.
Valida que el archivo de configuración cumpla con el esquema de descrito en configuracion.xsd. Carga en el sistema la información de: ☞ host:puerto:base de datos ☞ Usuario ☞ Contraseña El sistema informa del éxito de la operación.	

Flujos Alternativos:

- Si el archivo de .xml proporcionado por el usuario no cumple corresponde al esquema descrito en configuracion.xsd, genera un mensaje indicando al usuario que el archivo es inválido.
- Si el usuario proporcionado en el archivo .xml, es sys o system indica al usuario que no es posible establecer conexión con éstos usuarios.

Criterios de Aceptación

- El sistema valida adecuadamente el sistema proporcionado por el usuario.

Poscondiciones/Salidas:

- En el sistema se encuentran cargados los parámetros necesarios para establecer conexión a la base de datos.

N. req.	REQ_2		
Modulo	Bad Smells		
Nombre Requerimiento	Ejecución de bad smells		
Estado	Definido	Tipo	Funcional
Descripción			

Precondiciones/Entradas:

- En el sistema se encuentran cargados los parámetros necesarios para establecer conexión a la base de datos.

Flujo de Eventos:

SISTEMA	USUARIO
	Selecciona la opción de ejecución de bad smell deseado.
El sistema determinar el bad smell seleccionado por el usuario: ↻ Si el bad smell seleccionado requiere de parámetros dados por el usuario el sistema solicita los datos necesarios.	
	Si el sistema solicita datos, el usuario proporciona lo requerido.
El sistema se conecta a la base de datos, realiza las consultas necesarias, el análisis y presenta el resultado del análisis al usuario.	

Flujos Alternativos:

- Al selecciona el bad smell a ejecutar, si en el sistema no se encuentra cargado los parámetros de conexión de base de datos, se informa al usuario de éste evento.

Criterios de Aceptación

- Se presenta al usuario un reporte del análisis de bad smell en el esquema determinado por el usuario.

Poscondiciones/Salidas:

- Se presenta al usuario un reporte del análisis de bad smell en el esquema determinado por el usuario.

3 La aplicación cuenta con los siguientes requerimientos funcionales:

N. req.	REQN_1		
Nombre Requerimiento	Bases de datos		
Estado	Definido	Tipo	No Funcional.
Descripción	Se requiere que la aplicación soporte el análisis de esquemas de bases de datos relacionales, montadas sobre motores de bases de datos Oracle ®.		

N. req.	REQN_2		
Nombre Requerimiento	Aplicación Web		
Estado	Definido	Tipo	No Funcional.
Descripción	Se requiere que la aplicación se monte sobre una arquitectura web que permita al cliente acceder a las funcionalidades a través de un navegador.		

N. req.	REQN_3		
Nombre Requerimiento	Escalable		
Estado	Definido	Tipo	No Funcional.
Descripción	Se requiere que la aplicación sea construida bajo un modelo de desarrollo incremental, que permita adicionar nuevos bad smells con un bajo nivel de complejidad.		

Anexo 5 Manual de usuario de la aplicación BadSmellsBDR

INTRODUCCIÓN

El manual de usuario es uno de los documentos que más fortaleza le brinda al usuario de una aplicación, para poder administrar y conocer al máximo todo el potencial que posee la herramienta que utiliza.

Es por esto que se busca apoyar al usuario, para conocer rápidamente el alcance y los límites de esta herramienta, y guiarlo en el proceso de uso.

Cabe resaltar que ésta es una herramienta sencilla y fácil de manejar, que busca introducir al diseñador, y apoyar las etapas iniciales del proceso de refactoring de bases de datos relacionales. Se entrega un framework sobre el cual se pueden seguir adaptando bad smells con el objeto de fortalecer cada vez más el alcance de la herramienta.

ALCANCE DEL DOCUMENTO

Por la necesidad de detallar la forma que el usuario debe utilizar la aplicación, este documento presenta y entrega una descripción detallada del procedimiento a seguir para aprovechar sus funcionalidades.

DESCRIPCIÓN DE LOS MÓDULOS

La herramienta tiene como objeto mostrar la forma como el usuario, que esté realizando un proceso de refactoring puede apoyarse para detectar la posible presencia de bad smells en su modelo de bases de datos.

En el análisis que realiza ésta herramienta se involucran pocos bad smells, teniendo en cuenta que la aplicación tiene dos objetivos: 1. Mostrar que los bad smells propuestos sí son susceptibles de ser automatizados y, 2. Ser un framework que permite y permitirá trabajos futuros para permitir la detección automática de más bad smells. El segundo objetivo fue el que tuvo mayor impacto en la definición de la arquitectura de la herramienta.

¡Sea usted Bienvenido a BadSmellsBDR!

Interfaz Principal

En la interfaz principal usted puede encontrar el menú y el árbol de opciones, que le permiten realizar las operaciones de análisis soportadas hasta el momento por la aplicación para que identifique éstas opciones es importante que conozca adecuadamente las opciones del menú de la ventana principal, mostrada en la Figura 41.

Se tienen dos opciones en el menú:

- La primera, permite configurar la conexión a la base de datos que se desea analizar, reconocida por su nombre "Configuración".

- La segunda, contiene una lista de los bad smells que hasta el momento soporta la aplicación, reconocida por su nombre “*Bad Smells*”



Figura 41 Interfaz Principal de BadSmellsBDR

Descripción del Menú

Menú Configuración

La opción que en este menú se encuentran, es:

- **Configuración de Base de Datos**
Esta opción permite al usuario seleccionar la ruta del archivo de configuración por medio del cual se establecerá la conexión a la base de datos a analizar. El archivo de configuración debe ser .xml y tener la estructura descrita en la Figura 42, en general el archivo debe corresponder al schema que está descrito en el archivo configuracion.xsd (ver directorio descriptores).

```
<?xml version="1.0" encoding="UTF-8"?>
<configuracionBadSmell>
  <configuracionBaseDatos>
    <urlConexion>jdbc:oracle:thin:@host:puertoBd:nombreBd</urlConexion>
    <usuario>usuario de conexión</usuario>
    <password>contraseña</password>
  </configuracionBaseDatos>
</configuracionBadSmell>
```

Figura 42 Parámetros de configuración para conexión a bases de datos –Archivo XML

La interfaz por la que se solicita el archivo se observa en la Figura 43

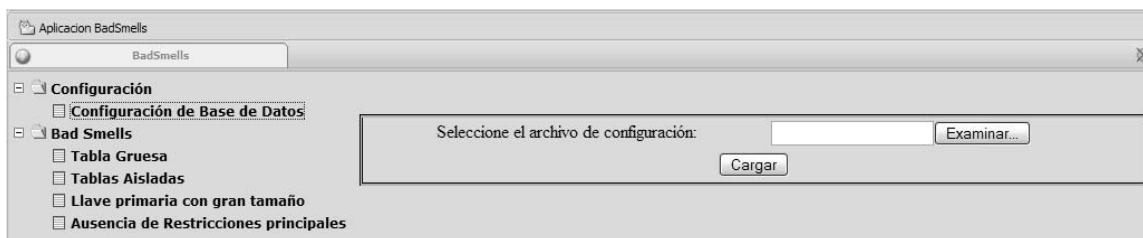


Figura 43 Solicitud de archivo de configuración para conexión a la base de datos.

Menú Bad Smells

Las opciones que en este menú se encuentran, son:

- **Tabla gruesa**
Permite detectar en el esquema de base de datos sobre el cual está conectado la presencia de posibles bad smell de “Tabla Gruesa”.
- **Tablas aisladas**
Permite detectar en el esquema de base de datos sobre el cual está conectado la presencia de posibles bad smell de “Tablas Aisladas”.
- **Llave primaria con gran tamaño**
Permite detectar en el esquema de base de datos sobre el cual está conectado la presencia de posibles bad smell de “Llave Primaria Larga”.
- **Ausencia de restricciones principales**
Permite detectar en el esquema de base de datos sobre el cual está conectado la presencia de posibles bad smell de “Ausencia de restricciones principales”.

Descripción de bad smells soportados

🔗 Bad Smells, Tabla gruesa.

Al seleccionar ésta opción el usuario indica a la aplicación que el bad smell a analizar es “tabla gruesa”, por lo que se solicita la información del número de atributos a partir del cual el usuario considera que es posible se presente éste bad smell.

La interfaz que solicita la información se observa en la Figura 44, una vez el usuario indica el número de atributos y presiona el botón “ejecutar”

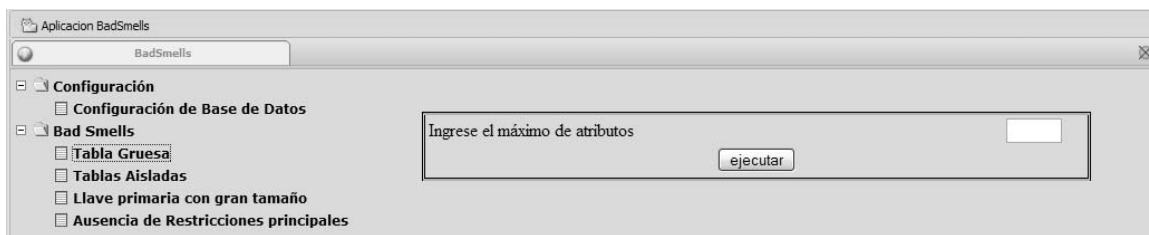


Figura 44 Solicitud de parámetros para bad smell “tabla gruesa”

La interfaz que solicita la información se observa en la Figura 44, una vez el usuario indica el número de atributos y presiona el botón “ejecutar”, se inicia el proceso de análisis de bad smell, al finalizar el análisis se presenta en pantalla los resultados como se observa en la Figura 45. Para conocer la forma de interpretar los resultados del análisis ir al tema “INTERPRETACIÓN DE LOS RESULTADOS DE ANÁLISIS”

Nombre Elemento	Tipo	Causa BadSmell
ESTUDIANTE	TABLA	La tabla ESTUDIANTE tiene 50 atributos y el máximo estimado es 10
PROF_PROFESOR	TABLA	La tabla PROF_PROFESOR tiene 48 atributos y el máximo estimado es 10
EST_SECRETARIA	TABLA	La tabla EST_SECRETARIA tiene 42 atributos y el máximo estimado es 10
GEN_PERMISOS	TABLA	La tabla GEN_PERMISOS tiene 23 atributos y el máximo estimado es 10
MATRICULAS_ACTIVASPS	TABLA	La tabla MATRICULAS_ACTIVASPS tiene 20 atributos y el máximo estimado es 10
PS_TMP_CARRERAS	TABLA	La tabla PS_TMP_CARRERAS tiene 19 atributos y el máximo estimado es 10
HORARIO_MATERIASRAI	TABLA	La tabla HORARIO_MATERIASRAI tiene 18 atributos y el máximo estimado es 10
PS_TMP_MATRICULAS	TABLA	La tabla PS_TMP_MATRICULAS tiene 18 atributos y el máximo estimado es 10

Figura 45 Resultado análisis bad smell tabla gruesa

🔗 Bad Smells, Tablas Aisladas.

Al seleccionar ésta opción el usuario indica a la aplicación que el bad smell a analizar es “tablas aisladas”, para éste análisis no se requiere información proporcionada por el usuario por lo cual se inicia inmediatamente el proceso de análisis y se presentan los resultados, como se observa en la Figura 46

Nombre Elemento	Tipo	Causa BadSmell
CANDIDATOS_EOLD	TABLA	La tabla CANDIDATOS_EOLD no tiene relación con otras tablas del esquema.
CANDIDATOS_POLD	TABLA	La tabla CANDIDATOS_POLD no tiene relación con otras tablas del esquema.
CARGA_SEMESTRAL	TABLA	La tabla CARGA_SEMESTRAL no tiene relación con otras tablas del esquema.
COBRO_SERVICIOS	TABLA	La tabla COBRO_SERVICIOS no tiene relación con otras tablas del esquema.
DEFINITIVA	TABLA	La tabla DEFINITIVA no tiene relación con otras tablas del esquema.
DEFINITIVAPREVIA	TABLA	La tabla DEFINITIVAPREVIA no tiene relación con otras tablas del esquema.
ERR_MATRICULAS_ACTIVAS	TABLA	La tabla ERR_MATRICULAS_ACTIVAS no tiene relación con otras tablas del esquema.

Figura 46 Resultado análisis bad smell tablas aisladas

🔗 Bad Smells, Llave primaria con gran tamaño

Al seleccionar ésta opción el usuario indica a la aplicación que el bad smell a analizar es “llave primaria larga”, por lo que se solicita la información del número máximo de atributos a partir del cual el usuario considera que es posible se presente éste bad smell.

La interfaz que solicita la información se observa en la Figura 44, una vez el usuario indica el número de atributos y presiona el botón “ejecutar”

Figura 47 Solicitud de parámetros para bad smell “llave primaria larga”

La interfaz que solicita la información se observa en la Figura 47, una vez el usuario indica el número máximo de atributos y presiona el botón “ejecutar”, se inicia el proceso de análisis de bad smell, al finalizar el análisis se presenta en pantalla los resultados como se observa en la Figura 48. Para conocer la forma de interpretar los resultados del análisis ir al tema “INTERPRETACIÓN DE LOS RESULTADOS DE ANÁLISIS”

Nombre Elemento	Tipo	Causa BadSmell
EST_MATERIAS_PK	LLAVE PRIMARIA	La tabla EST_MATERIAS tiene asociada la llave primariaEST_MATERIAS_PK con 5 atributos y el máximo estimado es 3
SYS_C007093	LLAVE PRIMARIA	La tabla MATRICULAS_ACTIVASPS tiene asociada la llave primariaSYS_C007093 con 4 atributos y el máximo estimado es 3

Figura 48 Resultado análisis bad smell tabla gruesa

🔗 Bad Smells, Ausencia de restricciones principales

Al seleccionar ésta opción el usuario indica a la aplicación que el bad smell a analizar es “ausencia de restricciones principales”, para éste análisis no se requiere información proporcionada por el usuario por lo cual se inicia inmediatamente el proceso de análisis y se presentan los resultados, como se observa en la Figura 49

Nombre Elemento	Tipo	Causa BadSmell
CANDIDATOS_EOLD	TABLA	La tabla CANDIDATOS_EOLDno tiene asociada una llave primaria
CANDIDATOS_POLD	TABLA	La tabla CANDIDATOS_POLDno tiene asociada una llave primaria
CARGA_SEMESTRAL	TABLA	La tabla CARGA_SEMESTRALno tiene asociada una llave primaria
COBRO_SERVICIOS	TABLA	La tabla COBRO_SERVICIOSno tiene asociada una llave primaria
DEFINITIVA	TABLA	La tabla DEFINITIVAno tiene asociada una llave primaria
DEFINITIVAPREVIA	TABLA	La tabla DEFINITIVAPREVIAno tiene asociada una llave primaria
ERR_MATRICULAS_ACTIVAS	TABLA	La tabla ERR_MATRICULAS_ACTIVASno tiene asociada una llave primaria

Figura 49 Resultado análisis bad smell tablas aisladas

Interpretación de los resultados del análisis

Los resultados para cualquier tipo de bad smell se presenta de forma uniforme, una vez ejecutado el proceso de análisis en la parte derecha de la pantalla se lista el resultado de los posibles bad smells detectados, dado los parámetros del usuario (en los casos en que son necesarios).

La lista está compuesta por tres columnas como se observa en la Figura 50, ésta son:

🔗 Nombre Elemento,

hace referencia al nombre del objeto de la base de datos sobre el cual se detectó un posible bad smell.

- 3 Tipo,
Indica el tipo de objeto que se lista en la columna “nombre elemento”
- 3 Causa BadSmell,
Especifica brevemente la causa por la cual se considera la posibilidad de existencia del bad smell analizado.

Resultado Ejecución		
Nombre Elemento	Tipo	Causa BadSmell
ESTUDIANTE	TABLA	La tabla ESTUDIANTE tiene 50 atributos y el máximo estimado es 10
PROF_PROFESOR	TABLA	La tabla PROF_PROFESOR tiene 48 atributos y el máximo estimado es 10
EST_SECRETARIA	TABLA	La tabla EST_SECRETARIA tiene 42 atributos y el máximo estimado es 10
GEN_PERMISOS	TABLA	La tabla GEN_PERMISOS tiene 23 atributos y el máximo estimado es 10

Figura 50 Resultados de análisis de bad smell

Anexo 6 Encuesta aplicada como elemento de validación

La encuesta ha sido estructurada para que su lectura sea simple y fácilmente accedida por los expertos.

Se utilizó como herramienta de apoyo para su diseño, Unit Command Climate Assessment and Survey System (UCCASS), para obtener mayor información acerca de la herramienta: <http://www.bigredspark.com/survey.html>.

A continuación se muestran las pantallas y textos de la encuesta, en el orden correspondiente de lectura.

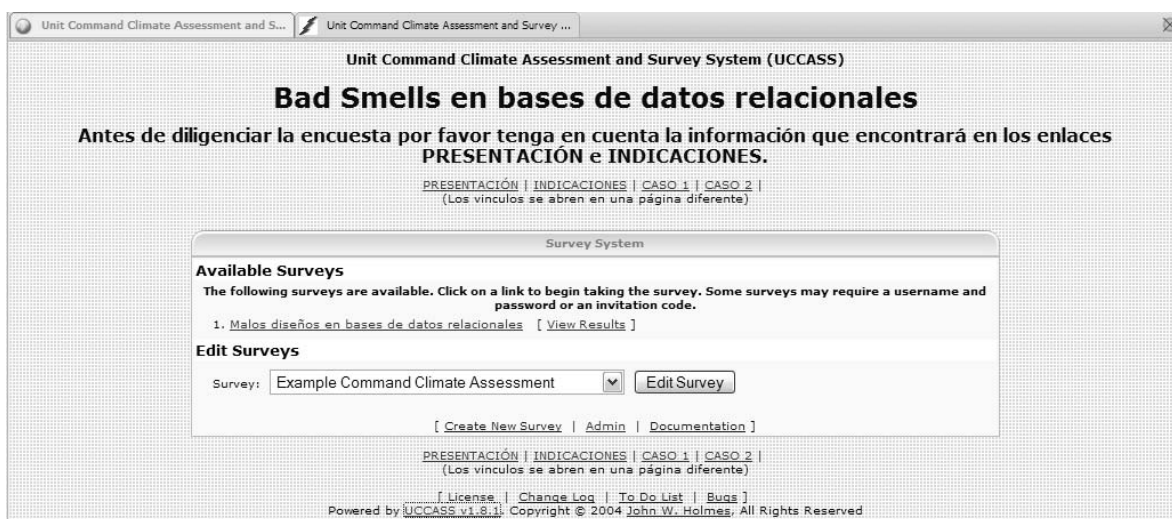


Figura 51 Página principal de la encuesta

3 Presentación de la encuesta

La primera información a la cuál el experto accede, se encuentra en una página de presentación en la cual se expone el objetivo de la encuesta y el perfil de los expertos (Figura 52).

Texto completo:

Las encuestas se utilizan como un mecanismo que permite, por medio de cuestionarios, recopilar información obteniendo así muestras representativas de datos que facilitan estudios, estadísticos y/o validadores.

¿Cuál es el objetivo de ésta encuesta?

Ésta encuesta está enmarcada dentro de la elaboración del trabajo de grado de la maestría de Ingeniería de Sistemas y Computación de la Universidad de los Andes, denominado "Bad Smells en bases de datos relacionales: Una propuesta taxonómica".

El trabajo propone un conjunto de Bad Smells (malos diseños), que pueden ser detectados en el momento de realizar reingeniería al diseño de bases de datos

relacionales, los clasifica en una taxonomía por categorías relacionadas lo que permitirá detectarlos, clasificarlos y comprenderlos con mayor agilidad.

Como estrategia de detección automática de éstos bad smells se ha desarrollado una herramienta, que se constituye como el marco de trabajo para ésta y futuras investigaciones.

Con el objeto de validar la herramienta y los resultados arrojados, se han seleccionado cuatro bad smells incluidos en la investigación como elementos facilitadores para validar el comportamiento de la herramienta vs. la experiencia y conocimiento de la comunidad de arquitectos/diseñadores de bases de datos relacionales.

¿A quién va dirigida?

A arquitectos, diseñadores y desarrolladores de bases de datos relacionales para quienes la mantenibilidad de sus bases de datos sea importante.

A arquitectos, diseñadores y desarrolladores de bases de datos relacionales que estén interesados en el aseguramiento de calidad de sus aplicativos.

A docentes con amplia experiencia en el diseño de bases de datos relacionales y/o ingeniería de software.

A investigadores en calidad de software, reingeniería de software y refactoring.

En general a toda la comunidad interesada en la optimización de los diseños de bases de datos que desean contar con mecanismos y herramientas que permiten analizar e identificar malas prácticas en el diseño de bases de datos relacionales.

¿Quién dirige la encuesta?

Mi nombre es Angela Paola Ramírez Pinilla. Soy ingeniera de sistemas de y actualmente me encuentro adelantando mis estudios de maestría en ingeniería de sistemas y computación en la Universidad de los Andes (Bogotá, Colombia).

Como parte de mi proyecto de investigación estoy interesada en proporcionar a la comunidad mecanismos de detección de malas prácticas en el diseño de bases de datos relacionales. Si desean compartir o iniciativas relacionadas con el proyecto con gusto atenderé sus comentarios. angel-ra@uniandes.edu.co , angpao@gmail.com.

Indicaciones

2.006

Indicaciones

La siguiente información a la cuál el experto accede, se encuentra en una página de indicaciones en la cual se presentan los conceptos de bad smells, la descripción de los bad smells a detectar e información sobre el caso de estudio (Figura 53).

Texto completo:

Contenido

Qué son Bad Smells (malas prácticas)

Bad Smells a evaluar

Aspectos importantes de los casos

Indicaciones para diligenciar la encuesta

Qué son bad smells?

El refactoring como actividad se ha desarrollado desde los años 1970's, sin embargo el entusiasmo reciente se centra en los beneficios que trae a los desarrolladores con el

planteamiento de mejores formas para detectar dónde aplicarlo, a través de la propuesta de Bad Smells realizada por Martín Fowler, los bad smells surgen como un mecanismo de detección de malas prácticas de diseño en la reingeniería.

En su propuesta Martín Fowler, definen el término Bad Smell como aquella mala práctica de programación, posteriormente Wake fortalece la definición de bad smell como aquella señal de alerta de problemas potenciales localizados en el código que aunque en ocasiones son fáciles de identificar no necesariamente son fáciles de arreglar.

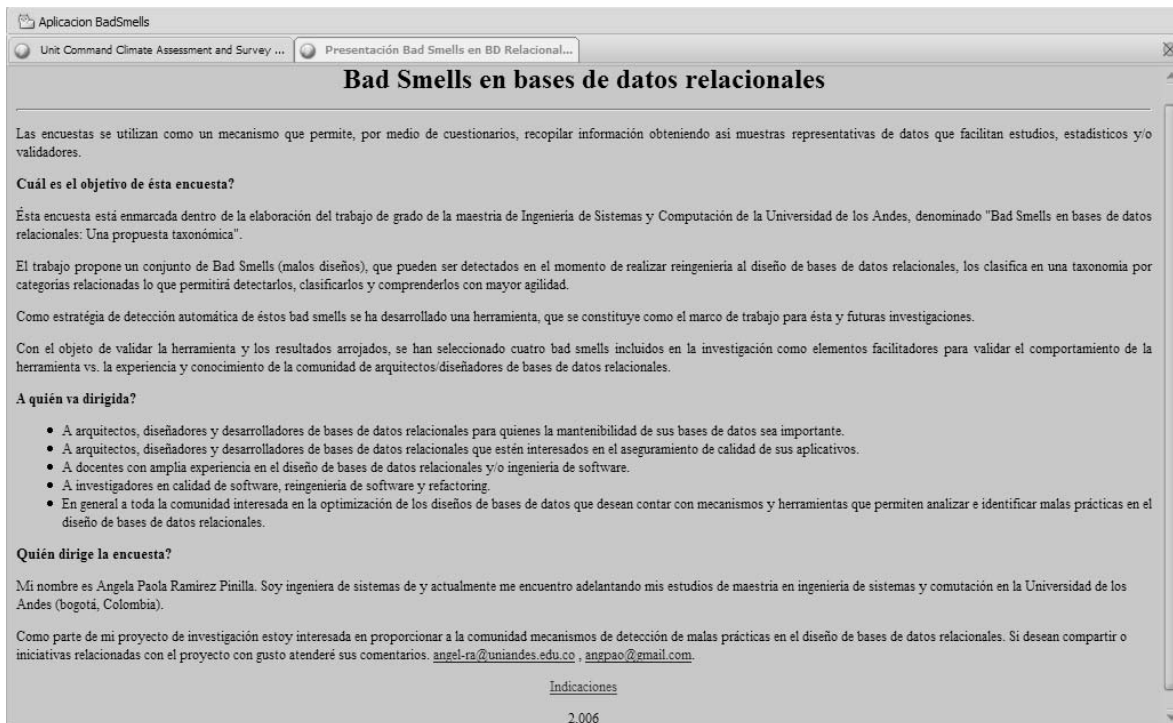


Figura 52 Página de presentación de la encuesta

Bad Smells a evaluar

En el marco de ésta investigación, se han definido bad smells aplicados al diseño de bases de datos relacionales, a continuación se relacionan 4 bad smells que serán utilizados como base para validar el comportamiento de la herramienta.

Nombre bad Smell	Descripción
Tabla Guesa	Apoya en la detección de malos diseños donde la tabla cuenta con un gran número de atributos, por lo que es probable que exista un alto acoplamiento de la información o indicar que se encuentra mal normalizada. La presencia de éste bad smell, puede indicar: ↻ Anomalías de actualización, por causa de un inadecuado diseño. ↻ Mantenibilidad, de la información en el tiempo.

Tabla Aislada	Apoya en detección de tablas que no tienen relaciones a través de la base de datos, normalmente las tablas con estas características son utilizadas con propósitos de auditoría, es por esto importante determinar cual es la razón de ser de la tabla. Se debe determinar si: ☞ Si la tabla es utilizada con propósitos de auditoría. ☞ Si se evidencia dependencias funcionales no relacionadas con el resto del modelo. En caso que a través de la tabla se pueda modelar dependencias funcionales, se debe verificar que dichas dependencias no se estén reflejadas a través de otras tablas y dependencias en la base de datos. La presencia de éste bad smell puede indicar: ☞ Ausencia de dependencias funcionales completas y/o necesarias a través de la base de datos. ☞ Dependencias funcionales necesarias en la base de datos que no están reflejándose en el modelo de datos.
Llave primaria de gran tamaño	Permite detectar llaves primarias que hacen lenta la consulta sobre tablas. Las llaves primarias permiten acceder más rápido a la información asociada a las tuplas de una tabla, sin embargo si están compuestas por un gran número de atributos con relación al total de atributos que están relacionados a una tabla, las consultas se hacen lentas a medida que el tamaño de la base de datos aumenta. La presencia de éste bad smell puede indicar: ☞ Bajo desempeño de la base de datos en la ejecución de tablas con éste bad smell.
Ausencia de restricciones principales	Apoya en detección de tablas con posible duplicidad de información. Las llaves primarias son un elemento fundamental en la definición de la base de datos puesto que evitan a nivel de tabla información redundante y a nivel del modelo de la base de datos permite establecer relaciones únicas evitando así tablas colgantes y espurias. La presencia de éste bad smell puede indicar: ☞ Redundancia de datos. ☞ Tablas colgantes. ☞ Tablas espurias.

Aspectos importantes de los casos

Los casos que apoyan la encuesta están elaborados con base en un sistema de gestión de instalación de servicios, dónde:

- ☞ Se administra la información de los usuarios,
- ☞ Las solicitudes de de servicio
- ☞ Las características de sus solicitudes, y
- ☞ Las ordenes de trabajo que se generan alrededor del proceso de instalación y administración de los servicios.

Estos casos son un reflejo parcial del esquema completo ER del diseño de la base de datos tomada como caso de estudio, sin embargo para efectos de análisis es interesante mencionar algunos datos que giran alrededor del sistema que soporta la base de datos.

- ☞ El registro usuarios en el sistema mensualmente es aproximadamente de 1.000
 - ☞ Un usuario por lo general realiza una solicitud que incluye dos o más requerimientos de trabajo
 - ☞ En el tiempo un usuario puede tener varias solicitudes asociadas
- La atención a quejas y reclamos de los usuarios se realiza sobre un pedido en particular

Indicaciones para diligenciar la encuesta

La encuesta está compuesta por una serie de preguntas enfocadas a:

Explorar de forma muy breve la experiencia del encuestado (3 preguntas)

Validar con base en la experiencia y conocimientos del encuestado la herramienta (11 preguntas).

A partir de la cuarta pregunta, en la parte inferior o en el enunciado se indica cual debe ser el caso a revisar para responder la pregunta, en la parte superior e inferior de la encuestas siempre encontrará los vínculos a los casos correspondientes.

Gracias por su colaboración!!!

Comentarios

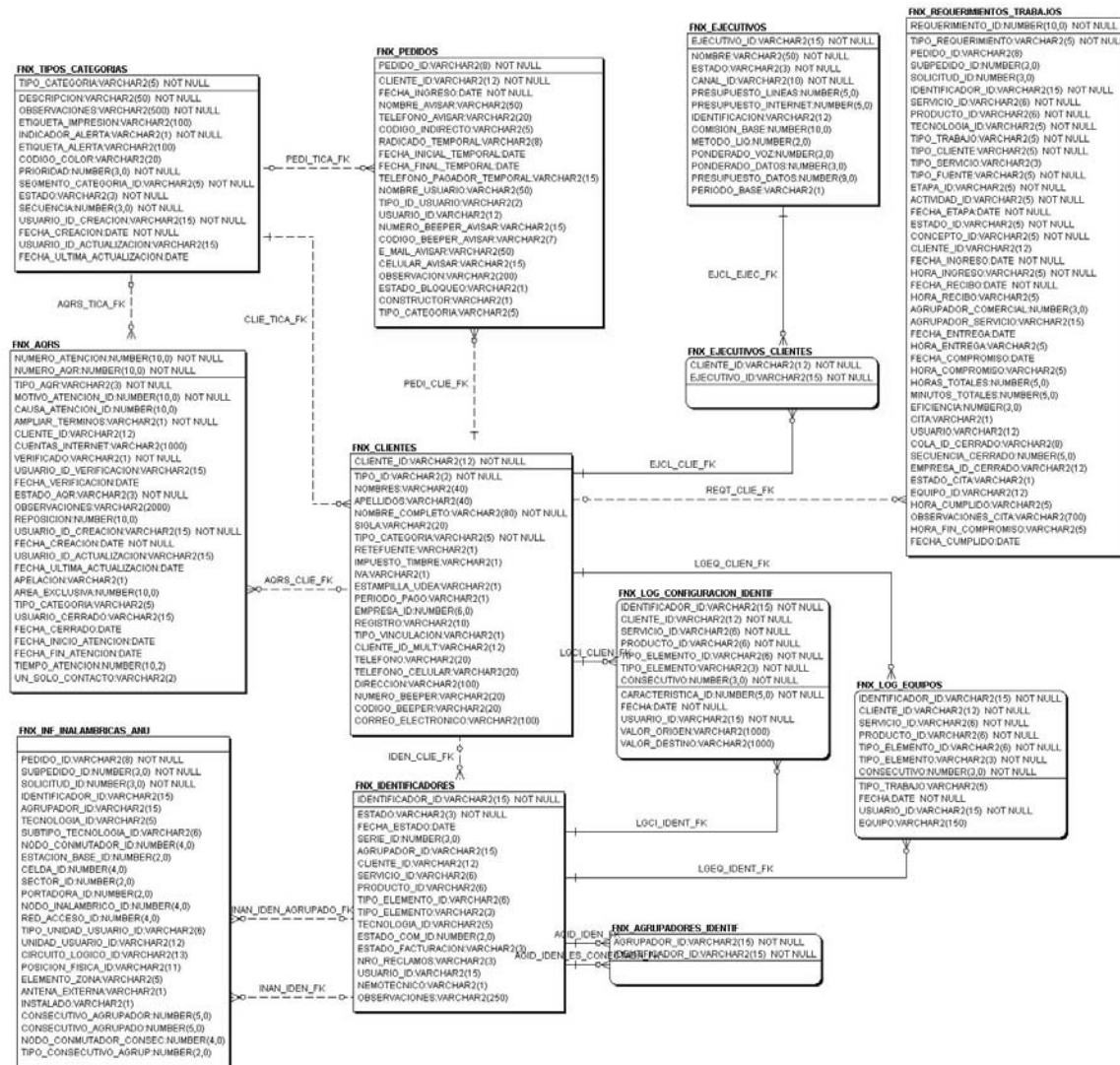


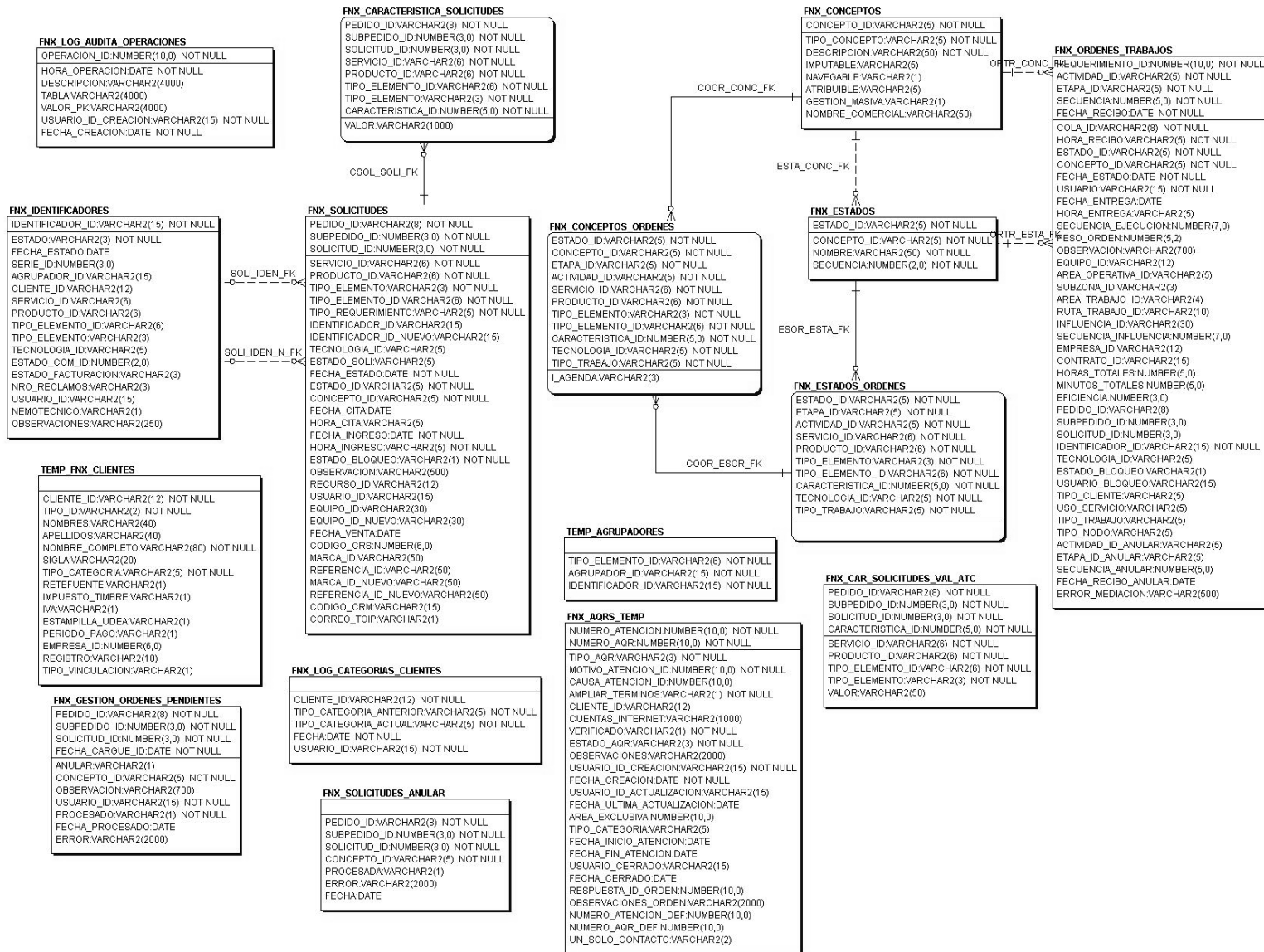
Figura 53 Página de indicaciones de la encuesta

3 Casos

Se coloca a consideración del experto dos casos, presentados a través de diagramas ER parciales del caso de estudio seleccionado, con el objeto de ser los instrumentos de análisis necesarios para que el experto tome decisiones sobre cada una de las preguntas.

A continuación se presenta el caso 1 y 2 respectivamente.





3 Cuestionario

La encuesta está compuesta por una serie de preguntas enfocadas a:

- 3 Explorar de forma muy breve la experiencia del encuestado (3 preguntas)
- 3 Validar con base en la experiencia y conocimientos del encuestado la herramienta (11 preguntas).

A partir de la cuarta pregunta, en la parte superior e inferior de la encuestas siempre encontrará los vínculos a los casos correspondientes. A continuación se presentan las preguntas.

Survey #28: Malos diseños en bases de datos relacionales

Page 1 of 5

1. [*] Ha recibido formación en buenas prácticas de diseño de bases de datos relacionales?

☐ Si ☐ No

2. [*] Cuantos años de experiencia tiene en el diseño de base de datos?

☐ Menos de 1 año
☐ 1 - 3 años
☐ 3 - 5 años
☐ 5 - 8 años
☐ 8 - 10 años
☐ 10 - 15 años
☐ Más de 15 años

Quit Survey - Do not save answers

<< Previous Page

Next Page >>

Page 2 of 5

3. [*] En qué tipo de formación recibió más conocimientos formales, bases teóricas y prácticas*Seleccione la formación donde recibió más conocimiento y tuvo más experiencia*

- ☐ Pregrado
- ☐ Diplomados
- ☐ Especializaciones
- ☐ Maestrias

4. [*] Considera usted que en la tabla FNX_CLIENTES se evidencia:*Caso 1*

- ☐ Alto acoplamiento de la información relacionada con el cliente
- ☐ Se podrían presentar anomalías de actualización
- ☐ Opción 1 y 2
- ☐ Ninguna de las anteriores

5. [*] Con base en la definición de las tablas FNX_AQRS y FNX_AQRS_TEMP de los casos 1 y 2, respectivamente. Considera usted que la tabla FNX_AQRS_TEMP aporta en el buen diseño de la base de datos.*Caso 2*

- ☐ Falso
- ☐ Verdadero

6. [*] La definición de la tabla FNX_ORDENES_TRABAJO relaciona EQUIPO_ID, AREA_OPERATIVA_ID, SUBZONA_ID, AREA_TRABAJO_ID, RUTA_TRABAJO_ID, INFLUENCIA_ID como llaves foráneas. Considera usted que:*Caso 2*

- ☐ Tiene un gran número de atributos que pueden ser organizados a través de nuevas relaciones
- ☐ Alto acoplamiento de la información relacionada con el ordenes de trabajo
- ☐ La información solicitada es la mínima requerida para crear ordenes de trabajo completas
- ☐ A través del tiempo, la tabla asegura la existencia de los datos necesarios para dar información de lo ocurrido con una orden de trabajo

Quit Survey - Do not save answers

<< Previous Page

Next Page >>

Page 3 of 5

7. [*] La tabla FNX_LOG_AUDITA_OPERACIONES apoya la administración de la base de datos.

Caso 2

- ☐ Falso
☐ Verdadero

8. [*] Sobre cuáles de las siguientes tablas aplica el bad smell "llave primaria de gran tamaño":

Seleccione las más representativas

- ☐ FNX_CONCEPTOS_ORDENES
☐ FNX_ORDENES_TRABAJOS
☐ FNX_LOG_CONFIGURACION_IDENTIF
☐ FNX_EJECUTIVOS_CLIENTES
☐ FNX_LOG_EQUIPOS

9. [*] Sobre cuál de las siguientes tablas considera usted que la llave primaria definida impacta el desempeño de las consultas realizadas a las bases de datos:

Seleccione las más representativas

- ☐ FNX_CONCEPTOS_ORDENES
☐ FNX_ORDENES_TRABAJOS
☐ FNX_LOG_CONFIGURACION_IDENTIF
☐ FNX_EJECUTIVOS_CLIENTES
☐ FNX_LOG_EQUIPOS

Quit Survey - Do not save answers

<< Previous Page

Next Page >>

Page 4 of 5

10. [*] Considera usted que es posible definir a través de un modelo matemático, el número máximo de atributos que debe tener una tabla en el diseño relacional de la base de datos en la cual está inmersa.

- ☐ Si
☐ No

11. [*] Para una tabla transaccional como FNX_LOG_CONFIGURACION_IDENTIF, considera usted que es más eficiente:

Caso 1

- ☐ La llave primaria definida
☐ Relacionar en una nueva tabla los valores de la llave primaria y establecer una integridad referencial con FNX_LOG_CONFIGURACION_IDENTIF, a través de un secuencial asignado en la nueva tabla

12. [*] Al analizar la tabla FNX_SOLICITUDES, cuáles bad smells puede identificar:

- ☐ Tabla gruesa
☐ Tabla aislada
☐ Llave primaria de gran tamaño
☐ Ausencia de restricciones principales

Quit Survey - Do not save answers

<< Previous Page

Next Page >>

Survey #28: Malos diseños en bases de datos relacionales

Page 5 of 5

13. [*] Al analizar la tabla FNX_INF_INALAMBRICAS_ANU, cuáles bad smells puede identificar:

☐ Tabla gruesa

☐ Tabla aislada

☐ Llave primaria de gran tamaño

☐ Ausencia de restricciones principales

14. [*] Al analizar la tabla FNX_REQUERIMIENTOS_TRABAJOS, cuáles bad smells puede identificar:

☐ Tabla gruesa

☐ Tabla aislada

☐ Llave primaria de gran tamaño

☐ Ausencia de restricciones principales

Quit Survey - Do not save answers << Previous Page Finish

Resultados de la encuesta

Survey ... Survey Results

☐ **4. [*] Considera usted que en la tabla FNX_CLIENTES se evidencia:**

Alto acoplamiento de la información relacionada con el cliente	- 2	50.00%
Se podrían presentar anomalías de actualización	- 1	25.00%
Opción 1 y 2	- 0	0.00%
Ninguna de las anteriores	- 1	25.00%
Total Answers	- 4	

☐ **5. [*] Con base en la definición de las tablas FNX_AQRS y FNX_AQRS_TEMP de los casos 1 y 2, respectivamente. Considera usted que la tabla FNX_AQRS_TEMP aporta en el buen diseño de la base de datos.**

Falso	- 4	100.00%
Verdadero	- 0	0.00%
Total Answers	- 4	

☐ **6. [*] La definición de la tabla FNX_ORDENES_TRABAJO relaciona EQUIPO_ID, AREA_OPERATIVA_ID, SUBZONA_ID, AREA_TRABAJO_ID, RUTA_TRABAJO_ID, INFLUENCIA_ID como llaves foráneas. Considera usted que:**

Tiene un gran número de atributos que pueden ser organizados a través de nuevas relaciones	- 4	100.00%
Alto acoplamiento de la información relacionada con el ordenes de trabajo	- 0	0.00%
La información solicitada es la mínima requerida para crear ordenes de trabajo completas	- 0	0.00%
A través del tiempo, la tabla asegura la existencia de los datos necesarios para dar información de lo ocurrido con una orden de trabajo	- 0	0.00%
Total Answers	- 4	

☐ **7. [*] La tabla FNX_LOG_AUDITA_OPERACIONES apoya la administración de la aplicación.**

Falso	- 2	50.00%
Verdadero	- 2	50.00%
Total Answers	- 4	

☐ 8. [*] Sobre cuáles de las siguientes tablas aplica el bad smell "llave primaria de gran tamaño":

FNX_CONCEPTOS_ORDENES	- 2	25.00%	25.00%
FNX_ORDENES_TRABAJOS	- 2	25.00%	25.00%
FNX_LOG_CONFIGURACION_IDENTIF	- 2	25.00%	25.00%
FNX_EJECUTIVOS_CLIENTES	- 0	0.00%	0.00%
FNX_LOG_EQUIPOS	- 2	25.00%	25.00%
Total Answers	- 8		

☐ 9. [*] Sobre cuál de las siguientes tablas considera usted que la llave primaria definida impacta el desempeño de las consultas realizadas a las bases de datos:

FNX_CONCEPTOS_ORDENES	- 3	37.50%	37.50%
FNX_ORDENES_TRABAJOS	- 1	12.50%	12.50%
FNX_LOG_CONFIGURACION_IDENTIF	- 2	25.00%	25.00%
FNX_EJECUTIVOS_CLIENTES	- 0	0.00%	0.00%
FNX_LOG_EQUIPOS	- 2	25.00%	25.00%
Total Answers	- 8		

☐ 10. [*] Considera usted que es posible definir a través de un modelo matemático, el número máximo de atributos que debe tener una tabla en el diseño relacional de la base de datos en la cual está inmersa.

Si	- 2	50.00%	50.00%
No	- 2	50.00%	50.00%
Total Answers	- 4		

☐ 11. [*] Para una tabla transaccional como FNX_LOG_CONFIGURACION_IDENTIF, considera usted que es más eficiente:

La llave primaria definida	- 1	25.00%	25.00%
Relacionar en una nueva tabla los valores de la llave primaria y establecer una integridad referencial con FNX_LOG_CONFIGURACION_IDENTIF, a través de un secuencial asignado en la nueva tabla	- 3	75.00%	75.00%
Total Answers	- 4		

☐ 12. [*] Al analizar la tabla FNX_SOLICITUDES, cuáles bad smells puede identificar:

Tabla gruesa	- 3	50.00%	50.00%
Tabla aislada	- 0	0.00%	0.00%
Llave primaria de gran tamaño	- 0	0.00%	0.00%
Ausencia de restricciones principales	- 3	50.00%	50.00%
Total Answers	- 6		

☐ 13. [*] Al analizar la tabla FNX_INF_INALAMBRICAS_ANU, cuáles bad smells puede identificar:

Tabla gruesa	- 1	14.29%	14.29%
Tabla aislada	- 2	28.57%	28.57%
Llave primaria de gran tamaño	- 0	0.00%	0.00%
Ausencia de restricciones principales	- 4	57.14%	57.14%
Total Answers	- 7		

☐ 14. [*] Al analizar la tabla FNX_REQUERIMIENTOS_TRABAJOS, cuáles bad smells puede identificar:

Tabla gruesa	- 3	50.00%	50.00%
Tabla aislada	- 0	0.00%	0.00%
Llave primaria de gran tamaño	- 0	0.00%	0.00%
Ausencia de restricciones principales	- 3	50.00%	50.00%
Total Answers	- 6		

Anexo 7 Explicación gráfica de bad Smells

Bad Smell – Tabla Gruesa

Name
Atributo 1
Atributo 2
.....
Atributo n-1
Atributo n

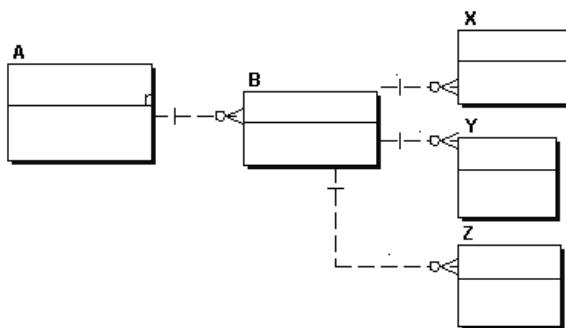
Para fortalecer la definición de éste bad smell, se debe realizar un estudio formal, de cual es el valor n, a partir del cuál se puede determinar la posibilidad de ocurrencia de éste bad smell.

Bad Smell – Tabla con alta responsabilidad



Se debe determinar si existe grupos de atributos que modelan diferentes responsabilidades del mundo, verificando posteriormente si realmente deben estar relacionados en la misma tabla

Bad Smell – Tabla con alto acoplamiento



Se debe determinar si hay relaciones de este estilo donde, por ejemplo:

- ↻ Tabla A representa empresa
- ↻ Tabla B descripción de un producto
- ↻ Tablas X, Y, Z, características asociadas a un producto

Varias empresas tienen asociado un mismo producto, con determinadas características. Sin embargo, se modifican las características de un producto, para satisfacer las necesidades de una empresa....en éste caso se puede presentar inconsistencia de información, dado que varias empresas hacen referencia a un mismo producto.

Bad Smell – Estructuras de datos internas duplicadas

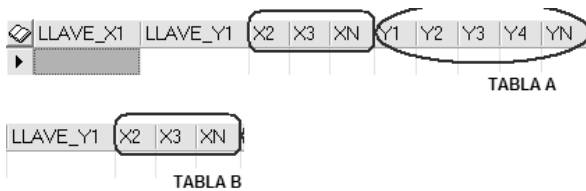
Name	DataType	Size	Not Null
LLAVE_X1	VARCHAR2	1	☐
LLAVE_Y1	VARCHAR2	1	☐
Y1	VARCHAR2	1	☐
Y2	VARCHAR2	4	☐
Y3	VARCHAR2	4	☐
Y4	VARCHAR2	4	☐

Si se identifican atributos como: y1, y2, y3 y y4. Dónde:

- ⌘ Conceptualmente manejan la misma información.
- ⌘ Su definición en tipo y tamaño es idéntica.
- ⌘ Son excluyentes

Se puede determinar la posibilidad de ocurrencia de éste bad smell.

Bad Smell – Estructuras de datos parcialmente duplicadas a través del modelo



Al validar tablas que:

- ⌘ Cuentan con atributos idénticos en tipo, tamaño y opcionalmente nombre.
- ⌘ Se debe verificar si se puede consolidar la información en una sola tabla

Bad Smell – Llave primaria con gran tamaño

Name	DataType	Size	Precision	Scale	Not Null	Primary Key
ESTADO_ID	VARCHAR2	5			☒	☒
CONCEPTO_ID	VARCHAR2	5			☒	☒
ETAPA_ID	VARCHAR2	5			☒	☒
ACTIVIDAD_ID	VARCHAR2	5			☒	☒
SERVICIO_ID	VARCHAR2	6			☒	☒
PRODUCTO_ID	VARCHAR2	6			☒	☒
TIPO_ELEMENTO	VARCHAR2	3			☒	☒
TIPO_ELEMENTO_ID	VARCHAR2	6			☒	☒
CARACTERISTICA_ID	NUMBER		5		☒	☒
TECNOLOGIA_ID	VARCHAR2	5			☒	☒
TIPO_TRABAJO	VARCHAR2	5			☒	☒
L_AGENDA	VARCHAR2	3			☐	☐

Se debe determinar si para la cantidad de atributos definidos para una tabla, la llave primaria se encuentra compuesta de un alto número de atributos.

Se puede determinar la posibilidad de ocurrencia de éste bad smell.

Bad Smell – Atributo repetitivo

SECUENCIAL	USUA_SECUENCIAL	FAC_M	PRG_M	MAT	CATE_SECUENCIAL	CODIGO
29	3755	I	I	18	72	1
31	1200	I	I	18	72	1
30	397	I	I	18	72	1
28	735	I	I	18	72	1
26	964	I	I	18	72	1
24	1437	I	I	18	72	1
21	4077	I	I	18	72	1
22	578	I	I	18	72	1
23	1452	I	I	18	72	1
283	3353	E	E	31	62	12
357	579	E	E	31	62	21
287	658	E	E	31	62	12
326	3002	E	E	31	62	21
282	989	E	E	31	61	11
356	2109	E	E	31	62	11
358	3411	E	E	31	62	21
360	496	E	E	31	62	31
359	3375	E	E	31	62	32
313	2877	E	E	33	62	2
341	2588	E	E	33	62	6

Si se puede establecer el dominio particular para un atributo o serie de atributos, Se puede determinar la posibilidad de ocurrencia de éste bad smell.

Bad Smell – Atributo repetitivo

SECUENCIAL	USUA_SECUENCIAL	FAC_M	PRG_M	MAT	CATE_SECUENCIAL	CODIGO
29	3755	I	I	18	72	1
31	1200	I	I	18	72	1
30	397	I	I	18	72	1
28	735	I	I	18	72	1
26	964	I	I	18	72	1
24	1437	I	I	18	72	1
21	4077	I	I	18	72	1
22	578	I	I	18	72	1
23	1452	I	I	18	72	1
283	3353	E	E	31	62	12
357	579	E	E	31	62	21
287	658	E	E	31	62	12
326	3002	E	E	31	62	21
282	989	E	E	31	61	11
356	2109	E	E	31	62	11
358	3411	E	E	31	62	21
360	496	E	E	31	62	31
359	3375	E	E	31	62	32
313	2877	E	E	33	62	2
341	2588	E	E	33	62	6

Si se puede establecer el dominio particular para un atributo o serie de atributos, Se puede determinar la posibilidad de ocurrencia de éste bad smell.

Bad Smell – Atributos co-relacionados

SECUENCIAL	USUA_SECUENCIAL	FAC_M	PRG_M	MAT	CATE_SECUENCIAL	LABO_SECUENCIAL
29	3755	I	I	18	72	9
31	1200	I	I	18	72	9
30	397	I	I	18	72	9
28	735	I	I	18	72	9
26	964	I	I	18	72	9
24	1437	I	I	18	72	9
21	4077	I	I	18	72	9
22	578	I	I	18	72	9
23	1452	I	I	18	72	9
283	3353	E	E	31	62	7
357	579	E	E	31	62	7
287	658	E	E	31	62	7
326	3002	E	E	31	62	7
282	989	E	E	31	61	7
356	2109	E	E	31	62	7
358	3411	E	E	31	62	7
360	496	E	E	31	62	7
359	3375	E	E	31	62	7
313	2877	E	E	33	62	7
341	2588	E	E	33	62	7

Si se puede establecer condiciones de dependencia funcional entre los atributos.

Se puede determinar la posibilidad de ocurrencia de éste bad smell.